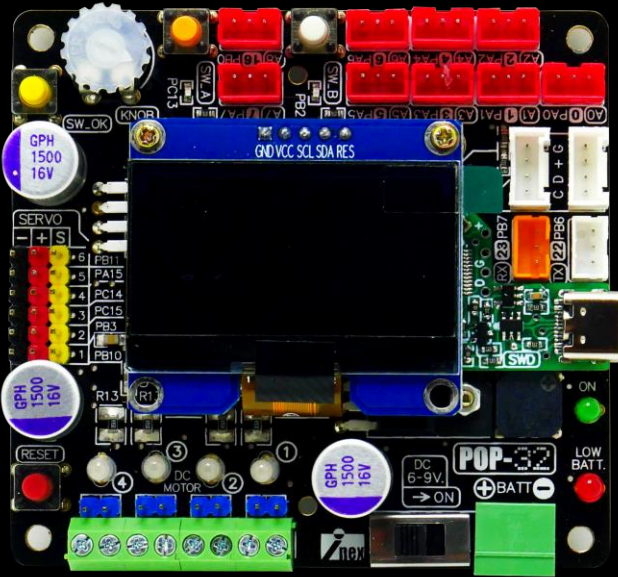
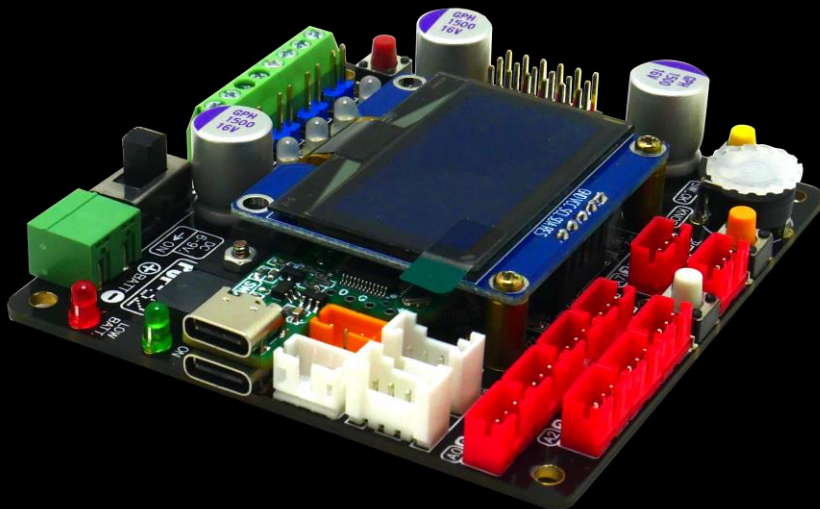


# POP-32



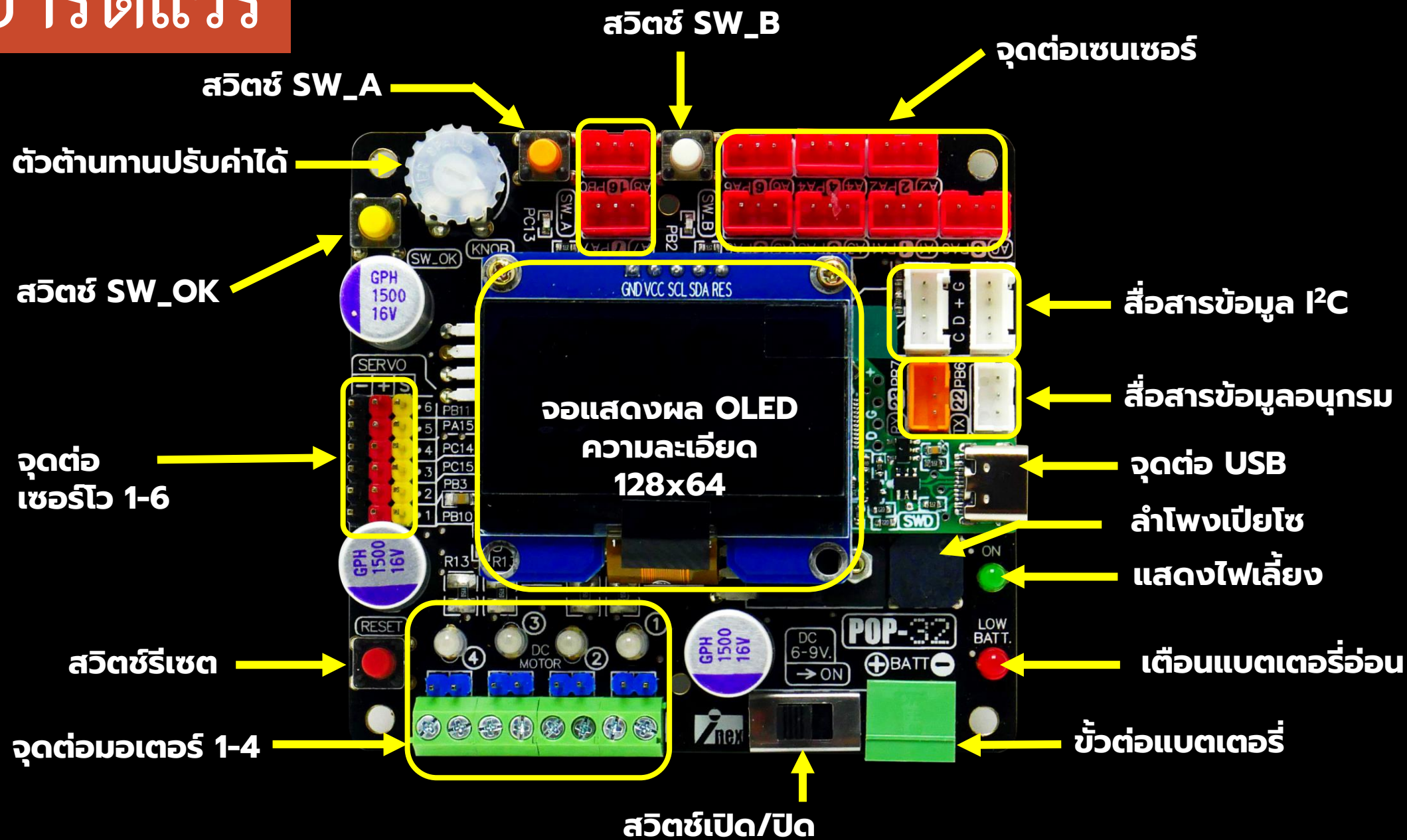
- ใช้ไมโครคอนโทรลเลอร์ 32 บิต เบอร์ STM32F103CBT6 ของ STMicroelectronics
- หน่วยความจำแฟลช 128KB
- หน่วยความจำแรม 20KB



# เอกสารที่ใช้ในการอบรม

<https://drive.google.com/drive/folders/1M0QmfqJlkK5WxmdZgHvEGYYXBXKms0Bb>

# ส่วนฮาร์ดแวร์



# ติดตั้งซอฟต์แวร์

PROFESSIONAL EDUCATION STORE

Search on Arduino.cc

SIGN IN

SOFTWARE CLOUD DOCUMENTATION COMMUNITY BLOG ABOUT

## Downloads

### Arduino IDE 2.3.2

The new major release of the Arduino IDE is faster and even more powerful! In addition to a more modern editor and a more responsive interface it features autocompletion, code navigation, and even a live debugger.

For more details, please refer to the [Arduino IDE 2.0 documentation](#).

Nightly builds with the latest bugfixes are available through the section below.

#### DOWNLOAD OPTIONS

**Windows** Win 10 and newer, 64 bits  
**Windows** MSI installer  
**Windows** ZIP file

**Linux** ApplImage 64 bits (X86-64)  
**Linux** ZIP file 64 bits (X86-64)

**macOS** Intel, 10.15: "Catalina" or newer, 64 bits  
**macOS** Apple Silicon, 11: "Big Sur" or newer, 64 bits

[Release Notes](#)

Help

1.เข้าไปดาวน์โหลดตัวติดตั้ง Arduino จากลิง <https://www.arduino.cc>

# ติดตั้งซอฟต์แวร์

PROFESSIONAL EDUCATION STORE

Search on Arduino.cc

SIGN IN

SOFTWARE CLOUD DOCUMENTATION COMMUNITY BLOG ABOUT

## Downloads

### Arduino IDE 2.3.2

The new major release of the Arduino IDE is faster and even more powerful! In addition to a more modern editor and a more responsive interface it features autocompletion, code navigation, and even a live debugger.

For more details, please refer to the [Arduino IDE 2.0 documentation](#).

Nightly builds with the latest bugfixes are available through the section below.

#### DOWNLOAD OPTIONS

**Windows** Win 10 and newer, 64 bits  
**Windows** MSI installer  
**Windows** ZIP file

**Linux** ApplImage 64 bits (X86-64)  
**Linux** ZIP file 64 bits (X86-64)

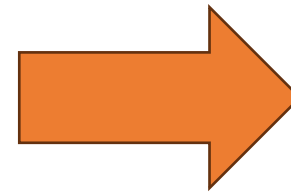
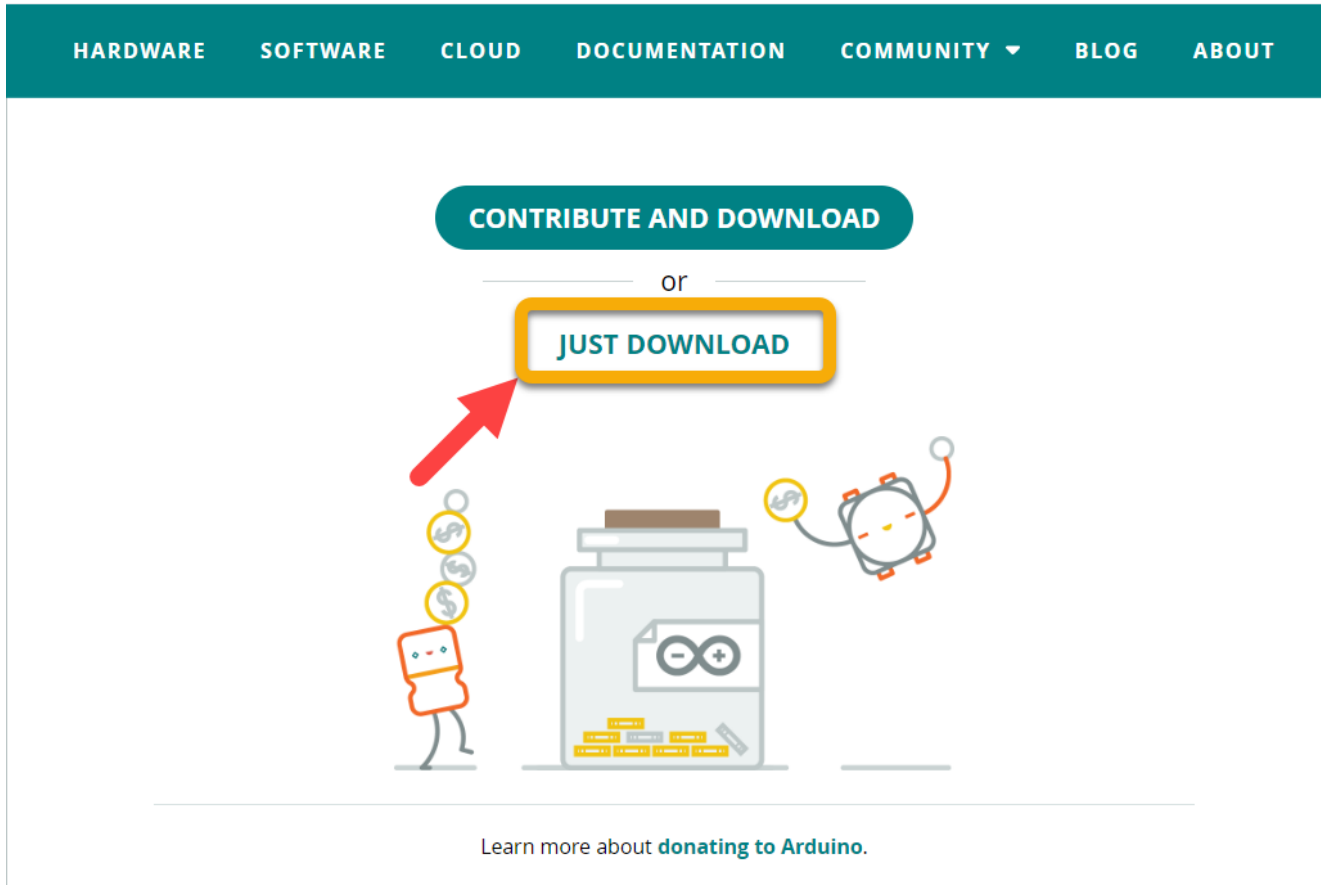
**macOS** Intel, 10.15: "Catalina" or newer, 64 bits  
**macOS** Apple Silicon, 11: "Big Sur" or newer, 64 bits

[Release Notes](#)

[Help](#)

2. ในที่นี้เลือกเป็นเวอร์ชัน 2.3.x

# ติดตั้งซอฟต์แวร์



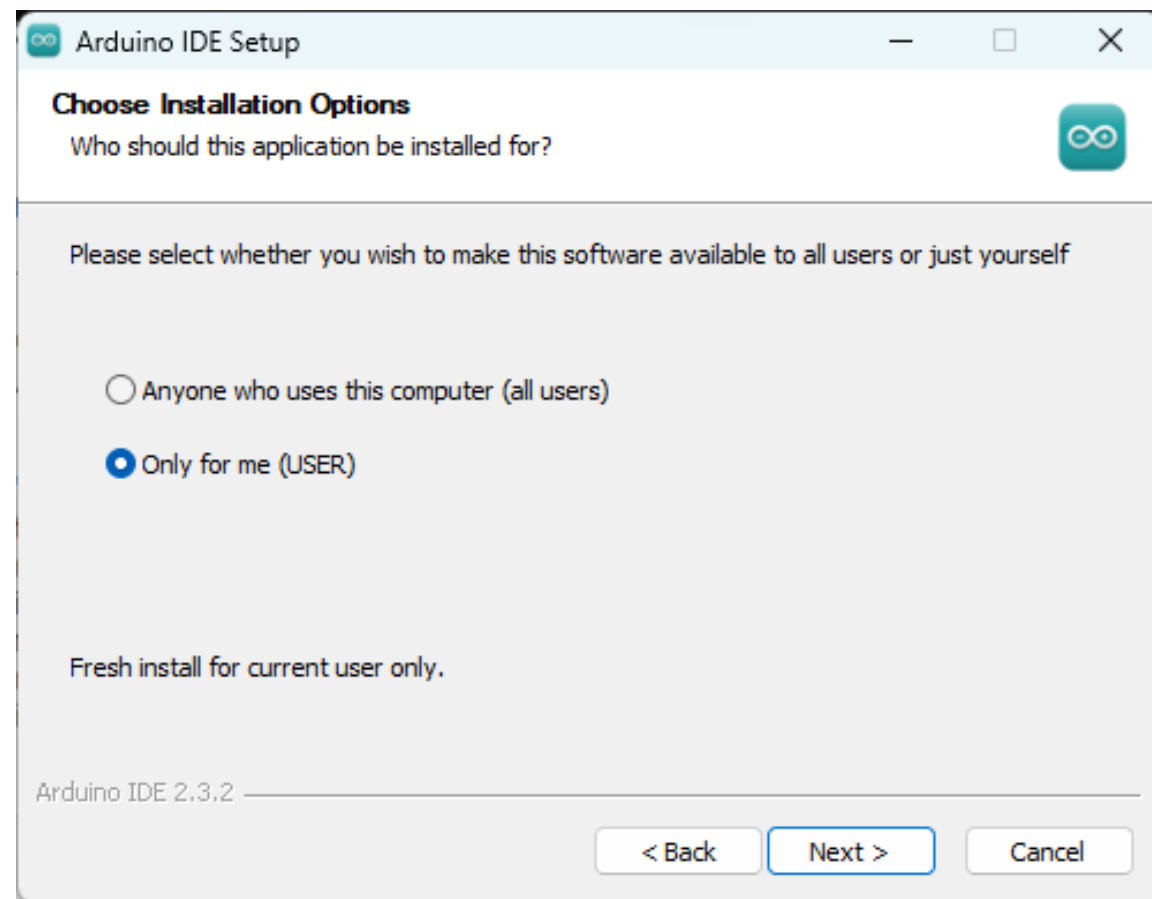
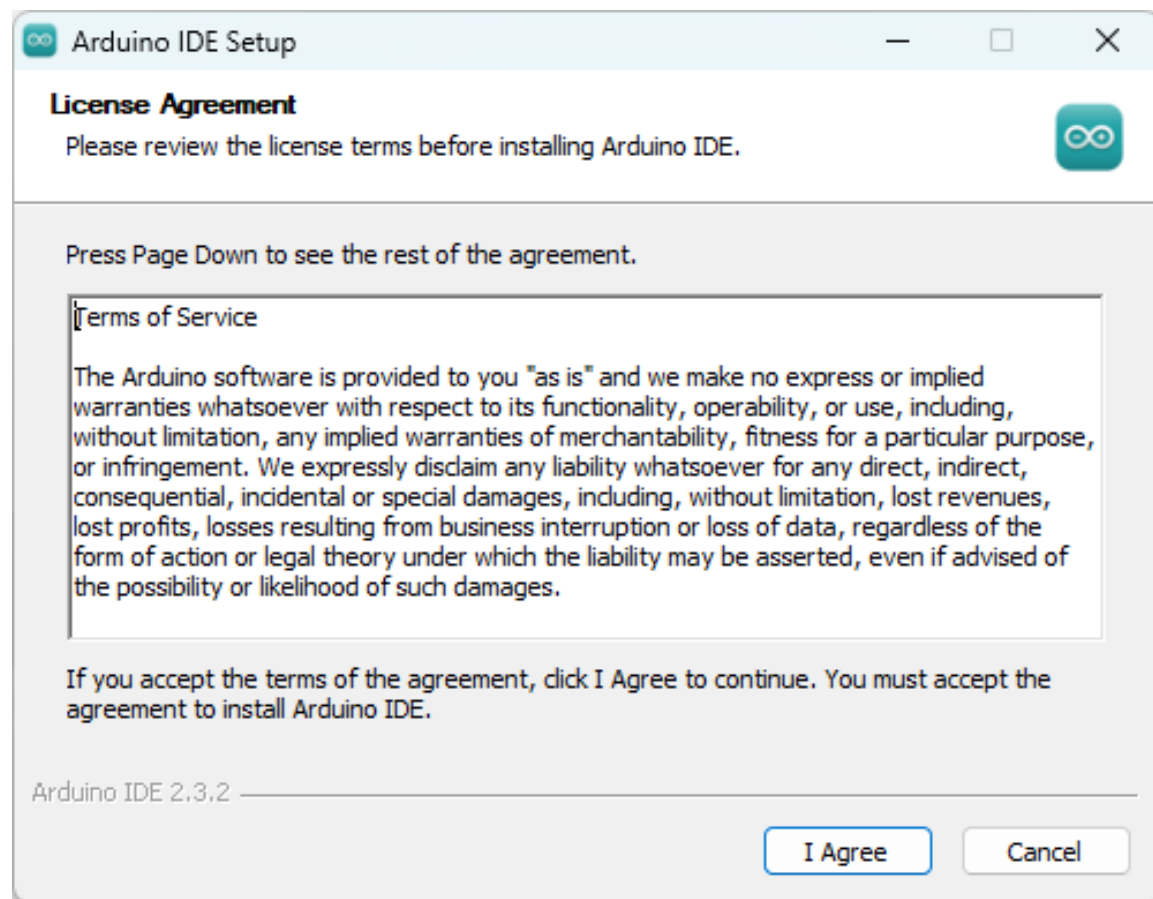
arduino-ide\_2.3.2  
\_Windows\_64bit.  
exe

3. เลือกตามขั้นตอนการดาวน์โหลด

4. รอจนกระทั่งตัวติดตั้ง Arduino ดาวน์โหลดเข้ามายังคอมพิวเตอร์

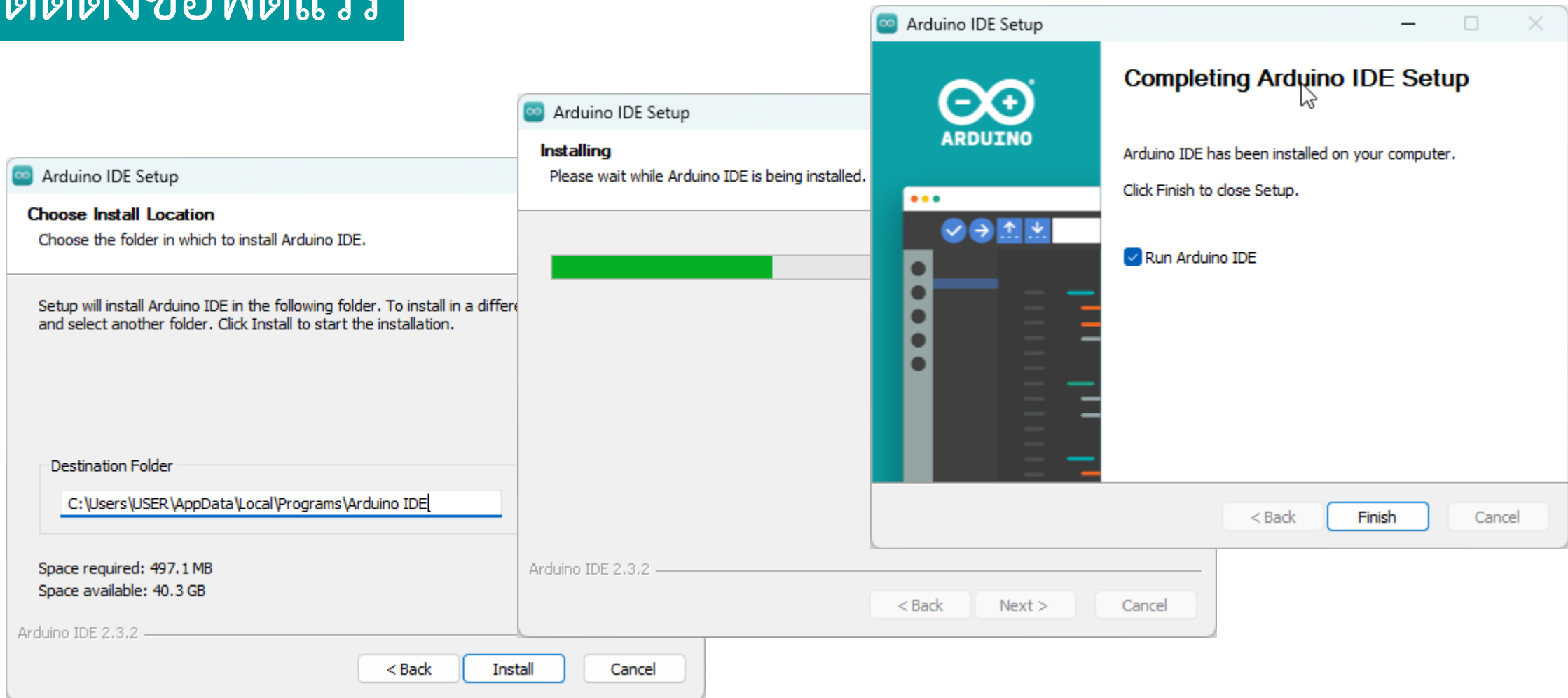


# ติดตั้งซอฟต์แวร์



5. ทำการติดตั้งโปรแกรม Arduino ลักษณะเช่นเดียวกับโปรแกรมทั่วไป

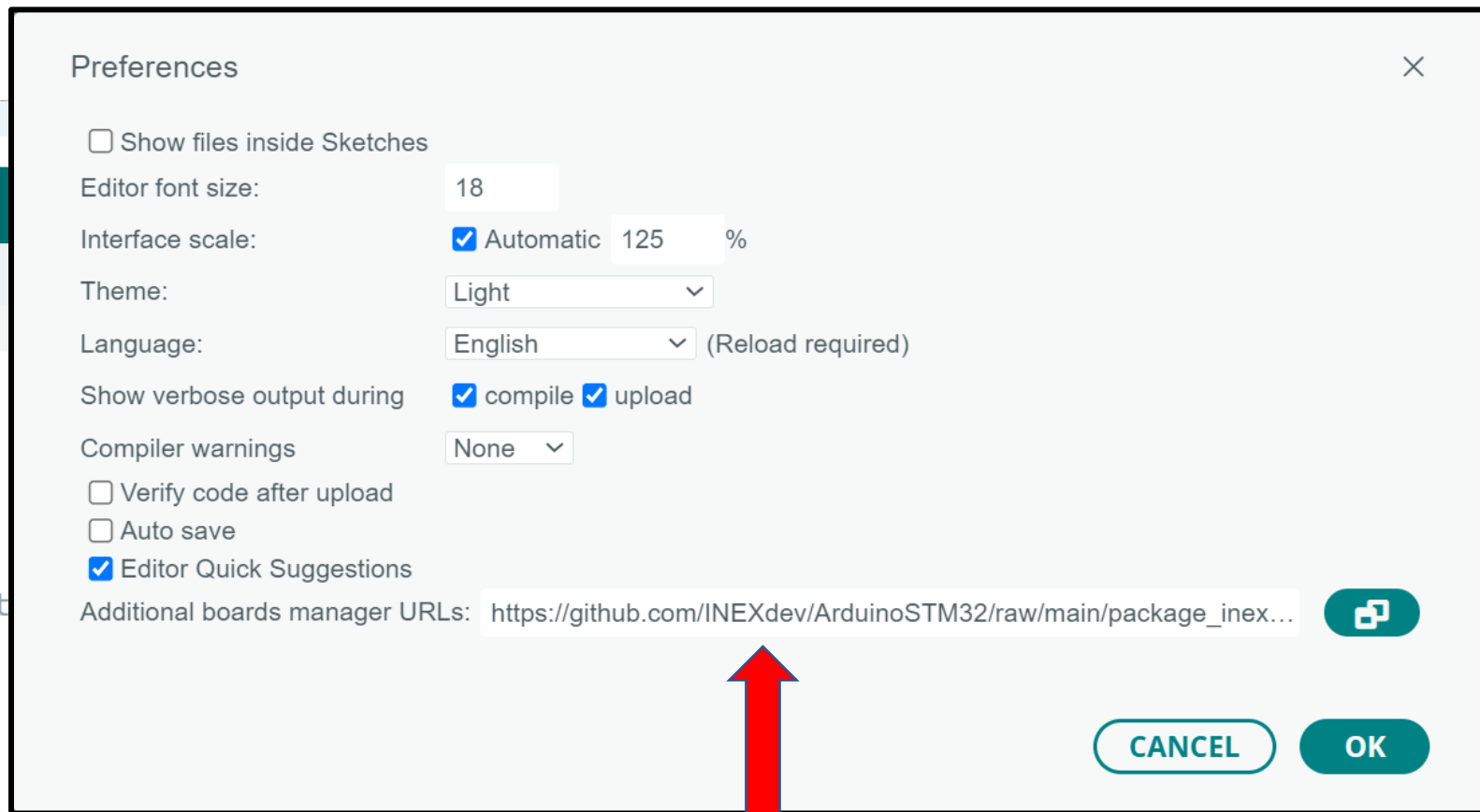
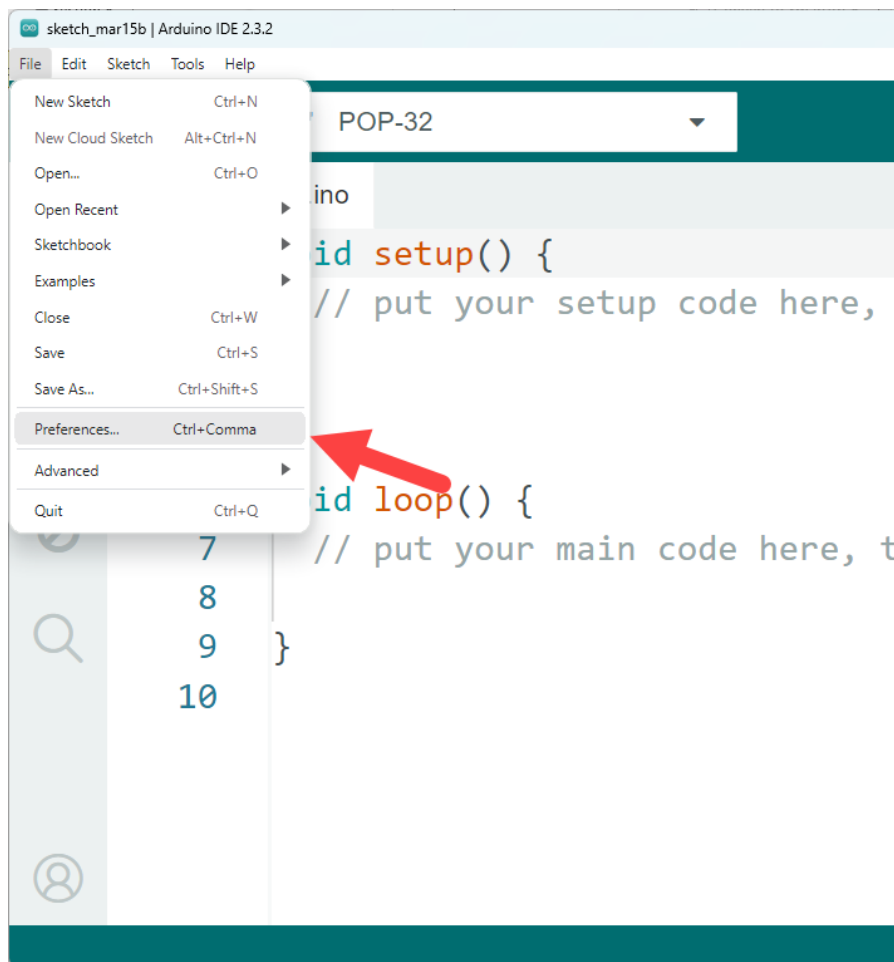
# ติดตั้งซอฟต์แวร์



6. หลังจากติดตั้งเสร็จสมบูรณ์เปิดโปรแกรม Arduino ขึ้นมา



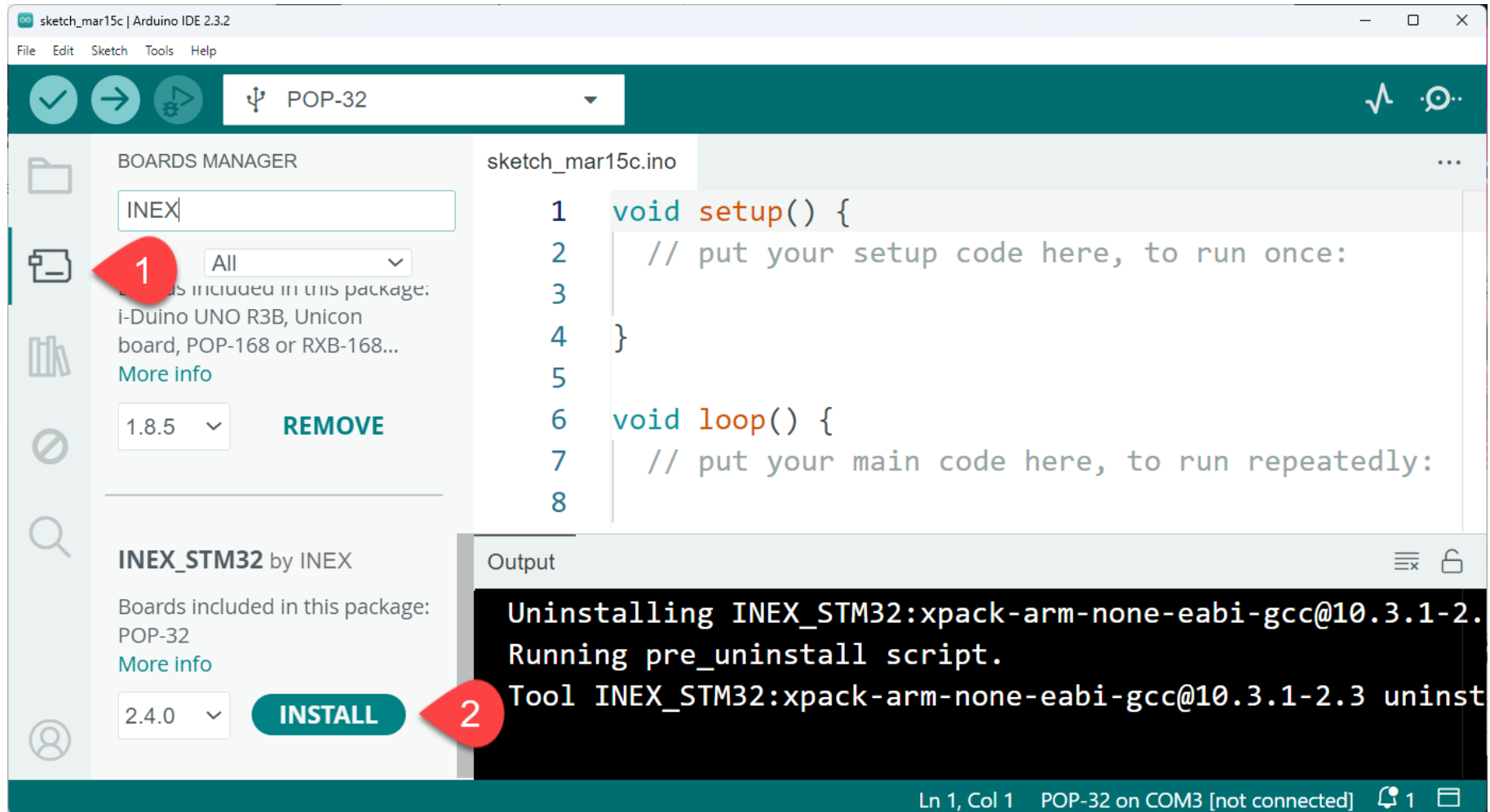
# ติดตั้งซอฟต์แวร์



[https://github.com/INEXdev/ArduinoSTM32/raw/main/package\\_inex\\_stm32\\_index.json](https://github.com/INEXdev/ArduinoSTM32/raw/main/package_inex_stm32_index.json)

7. ตั้งค่าโปรไฟล์บอร์ด INEX\_STM32 สำหรับบอร์ด POP-32

# ติดตั้งซอฟต์แวร์



## 8. ตั้งค่าสำหรับในส่วน Boards Manager

# ติดตั้งซอฟต์แวร์

```
Output
Installing INEX_STM32:CMSIS@5.7.0
Configuring tool.
INEX_STM32:CMSIS@5.7.0 installed
Installing platform INEX_STM32:stm32@2.4.0
Configuring platform.
Platform INEX_STM32:stm32@2.4.0 installed
```

9. รอจนกระทั่งติดตั้ง package เสร็จสมบูรณ์

# การอัปโหลดโปรแกรมผ่านช่อง สื่อสารอนุกรม

## ข้อดี

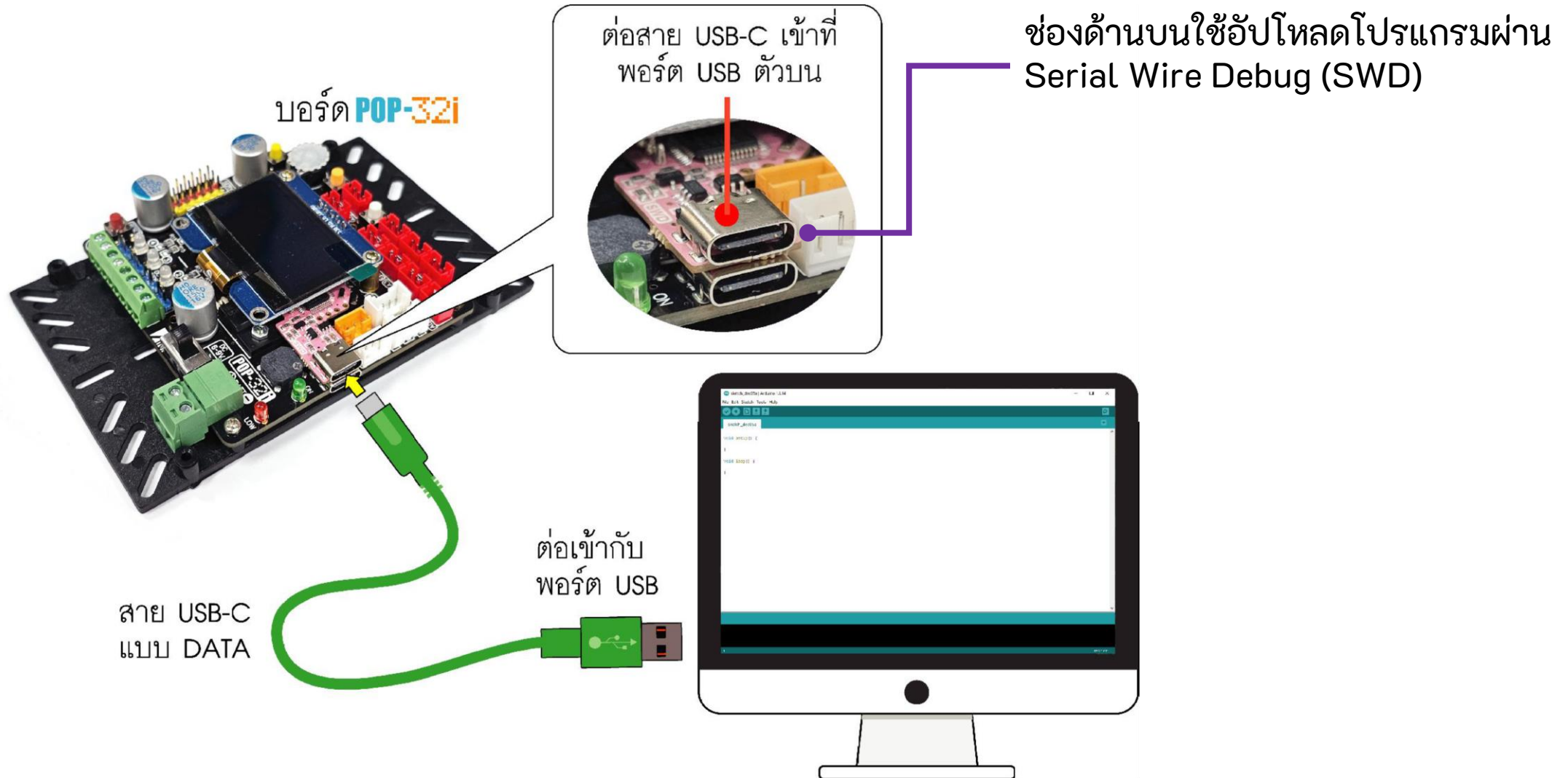
สามารถใช้งาน Serial Monitor ได้ทันทีโดยไม่ต้องเปลี่ยนช่อง USB

---

## ข้อเสีย

หากส่วน HID Bootloader บนบอร์ด POP-32 เกิดความเสียหาย ส่งผลให้ไม่สามารถอัปโหลดโปรแกรมผ่านช่องทางนี้ได้

# อัปโหลดโปรแกรมผ่านช่อง SWD



# ติดตั้ง STM32 Cube Programmer

ดาวน์โหลดโปรแกรม

<https://www.st.com/en/development-tools/stm32cubeprog.html>

## Get Software

|   | Part Number ▲    | General Description                    | Latest version ◆ | Download ◆ | All versions ◆   |
|---|------------------|----------------------------------------|------------------|------------|------------------|
| + | STM32CubePrg-Lin | STM32CubeProgrammer software for Linux | 2.13.0           | Get latest | Select version ▼ |
| + | STM32CubePrg-Mac | STM32CubeProgrammer software for Mac   | 2.13.0           | Get latest | Select version ▼ |
| + | STM32CubePrg-W32 | STM32CubeProgrammer software for Win32 | 2.13.0           | Get latest | Select version ▼ |
| + | STM32CubePrg-W64 | STM32CubeProgrammer software for Win64 | 2.13.0           | Get latest | Select version ▼ |

ขั้นตอนดาวน์โหลดจะต้องลงทะเบียน

## License Agreement

กดปุ่มยอมรับข้อตกลง



ACCEPT

Please indicate your acceptance or NON-acceptance by selecting "I ACCEPT" or "I DO NOT ACCEPT" as indicated below in the media.

BY INSTALLING COPYING, DOWNLOADING, ACCESSING OR OTHERWISE USING THIS SOFTWARE PACKAGE OR ANY PART THEREOF (AND THE RELATED DOCUMENTATION) FROM STMICROELECTRONICS INTERNATIONAL N.V, SWISS BRANCH AND/OR ITS AFFILIATED COMPANIES (STMICROELECTRONICS), THE RECIPIENT, ON BEHALF OF HIMSELF OR HERSELF, OR ON BEHALF OF ANY ENTITY BY WHICH SUCH RECIPIENT IS EMPLOYED AND/OR ENGAGED AGREES TO BE BOUND BY THIS SOFTWARE PACKAGE LICENSE AGREEMENT.

Under STMicroelectronics' intellectual property rights and subject to applicable licensing terms for any third-party software incorporated in this software package and applicable Open Source Terms (as defined here below), the redistribution, reproduction and use in source and binary forms of the software package or any part thereof, with or without modification, are permitted provided that the following conditions are met:



# Get Software

If you have an account on my.st.com, login and download the software without any further validation steps.

Login/Register

If you don't want to login now, you can download the software by simply providing your name and e-mail address in the form below and validating it.

This allows us to stay in contact and inform you about updates of this software.

**For subsequent downloads this step will not be required for most of our software.**

First Name:

Last Name:

E-mail address:

Please review our [Privacy Statement](#) that describes how we process your profile information and how to assert your personal data protection rights

☐ Please keep me informed about future updates for this software or new software in the same category

Download

กรอกชื่อและนามสกุล Email  
จากนั้นกดปุ่ม Download

# เปิด Email ที่ได้ลงทะเบียนไว้



## Start your software download

Hi teerawut

Please click on this button to validate your email address and start the download of the requested software:

Download now

← กดปุ่มเพื่อดาวน์โหลด

If you have any further issues, please send your request to our [online support](#) using the subject line: Software download issues.

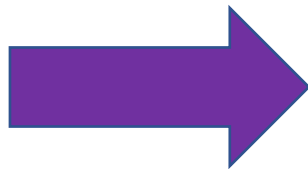
Thank you,

STMicroelectronics  
[www.st.com](http://www.st.com)

ได้ไฟล์ Zip จากนั้นแตกไฟล์



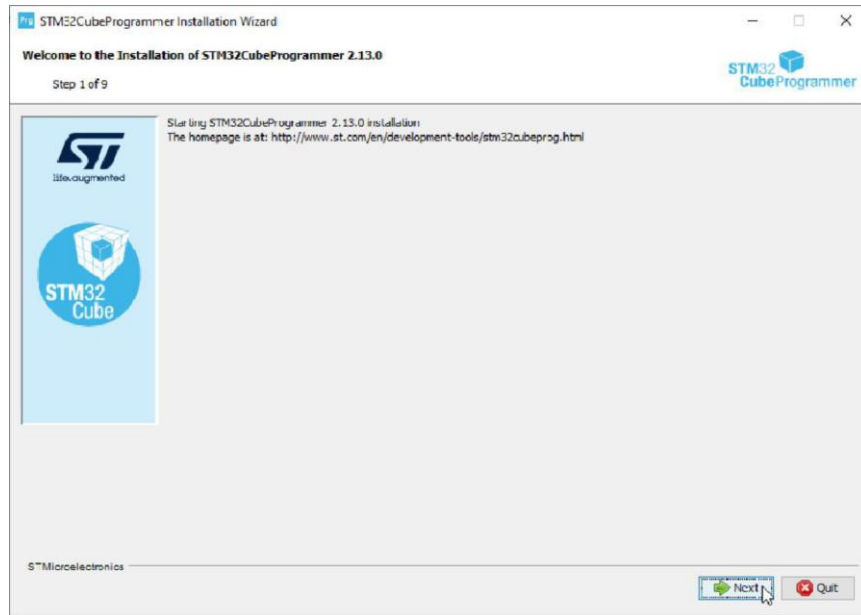
en.stm32cubeprog-win32-  
v2-13-0.zip



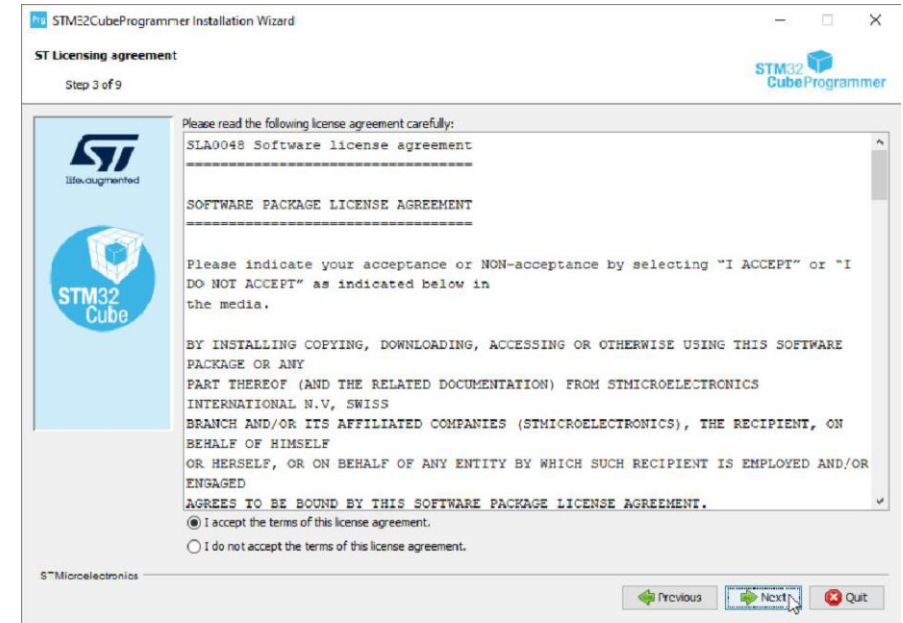
SetupSTM32CubeProgr  
ammer\_win32.exe

ดับเบิลคลิกเพื่อทำการติดตั้ง

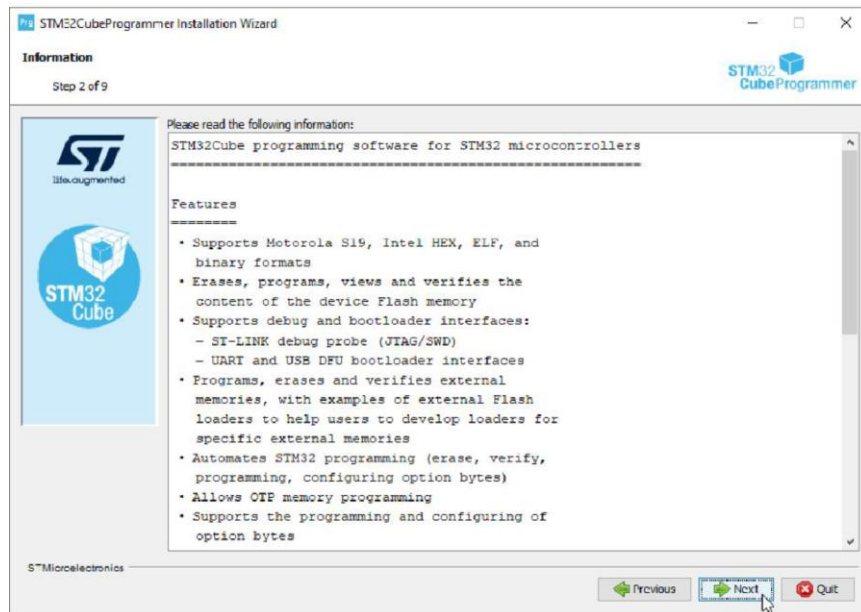
1



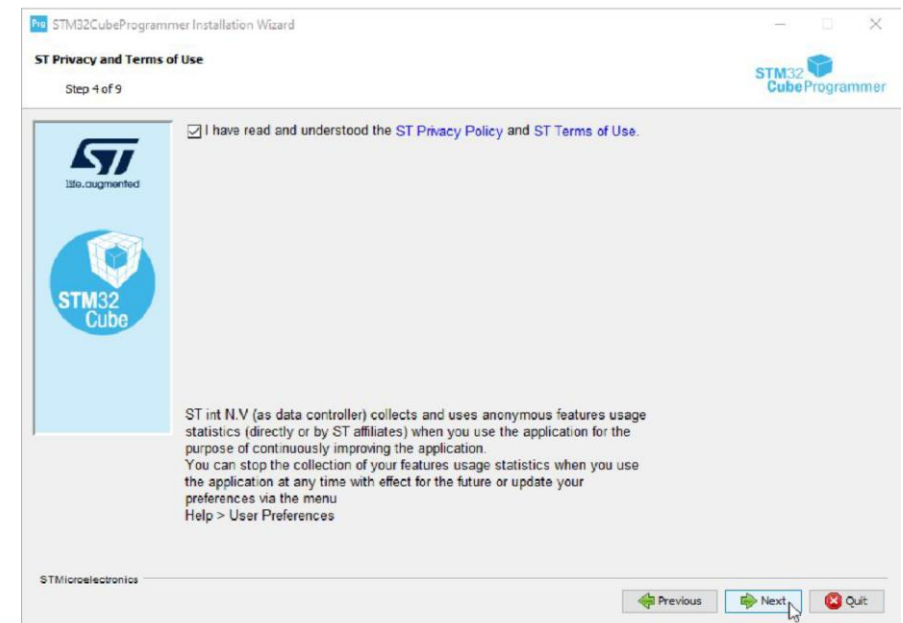
3



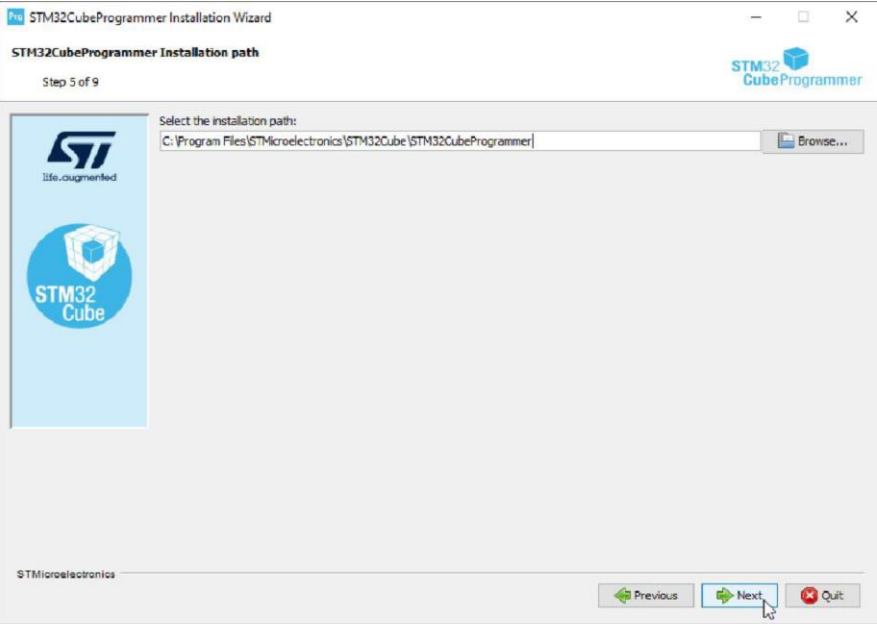
2



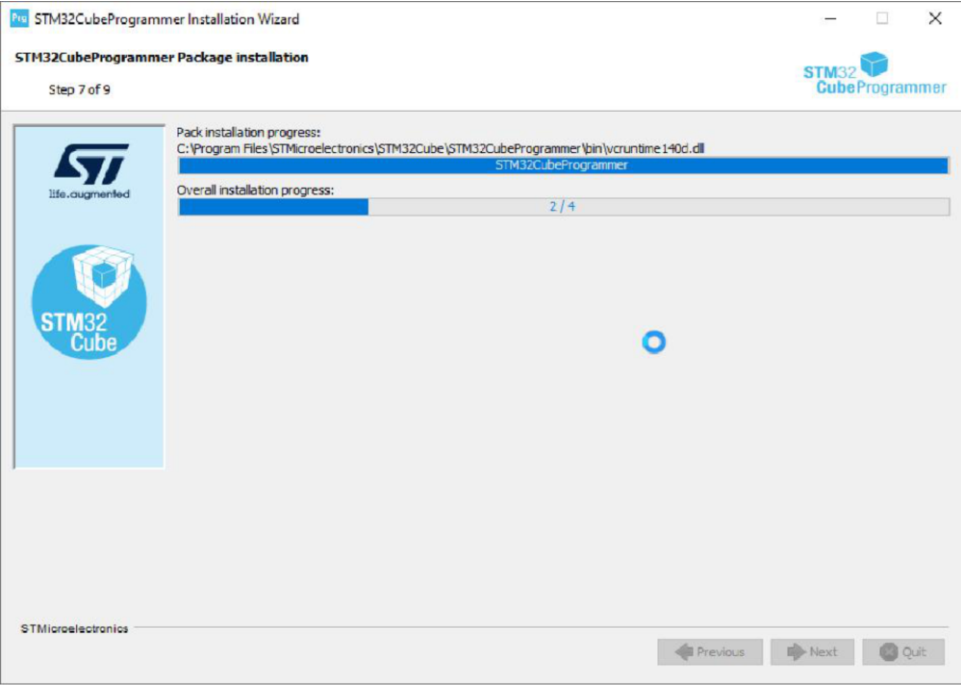
4



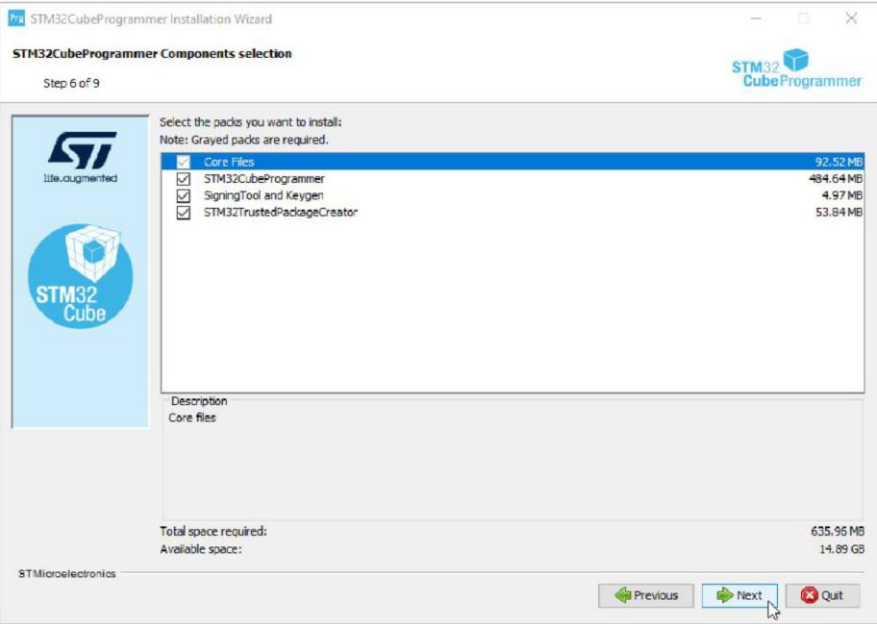
5



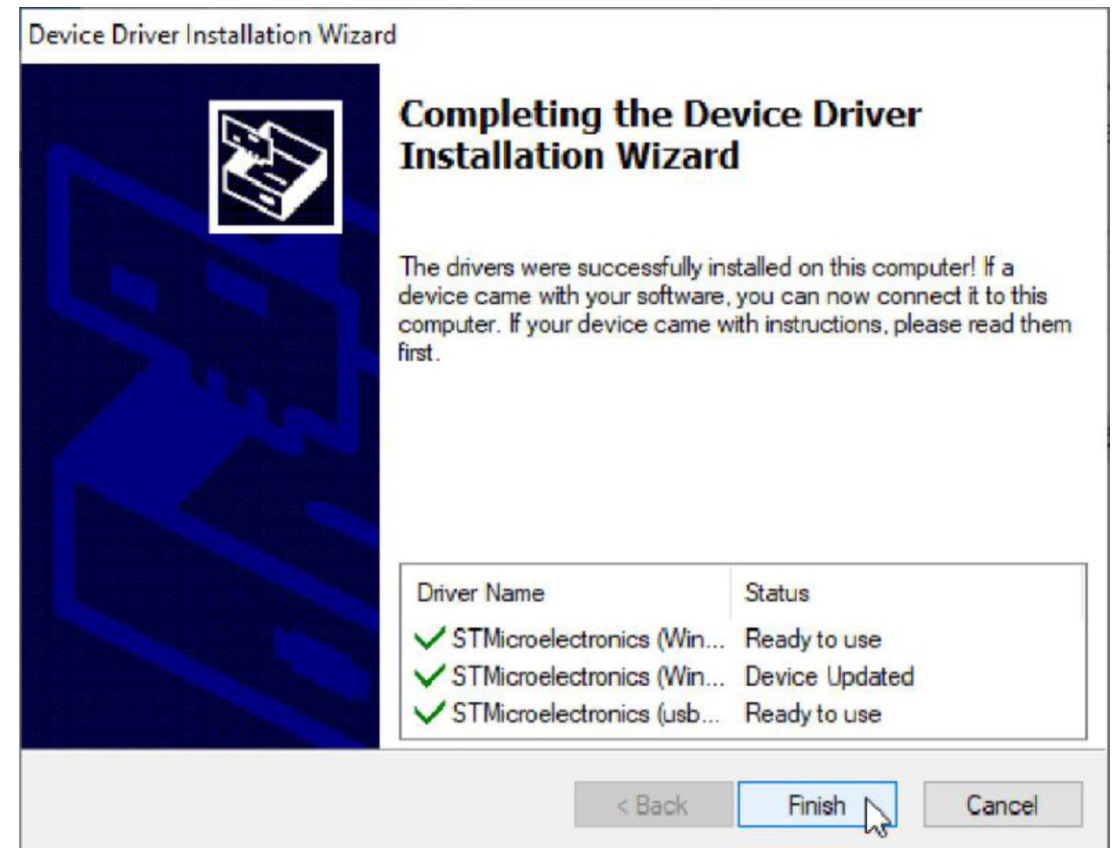
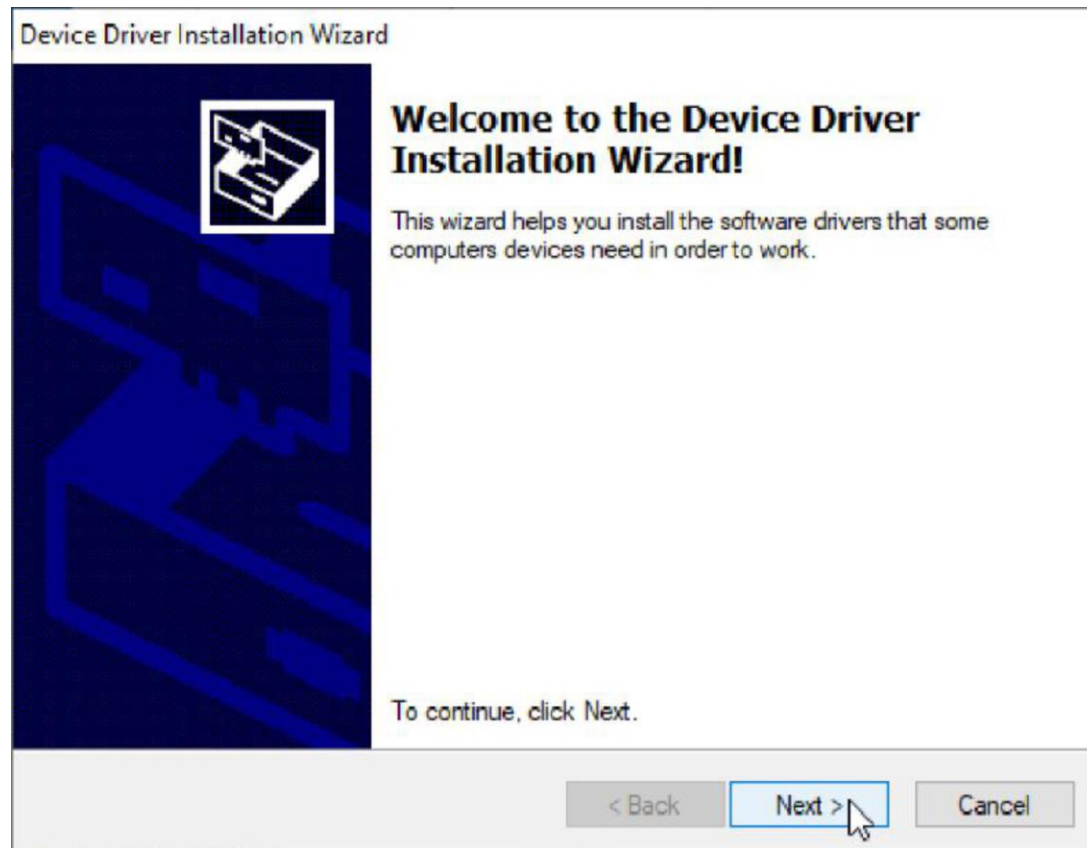
7



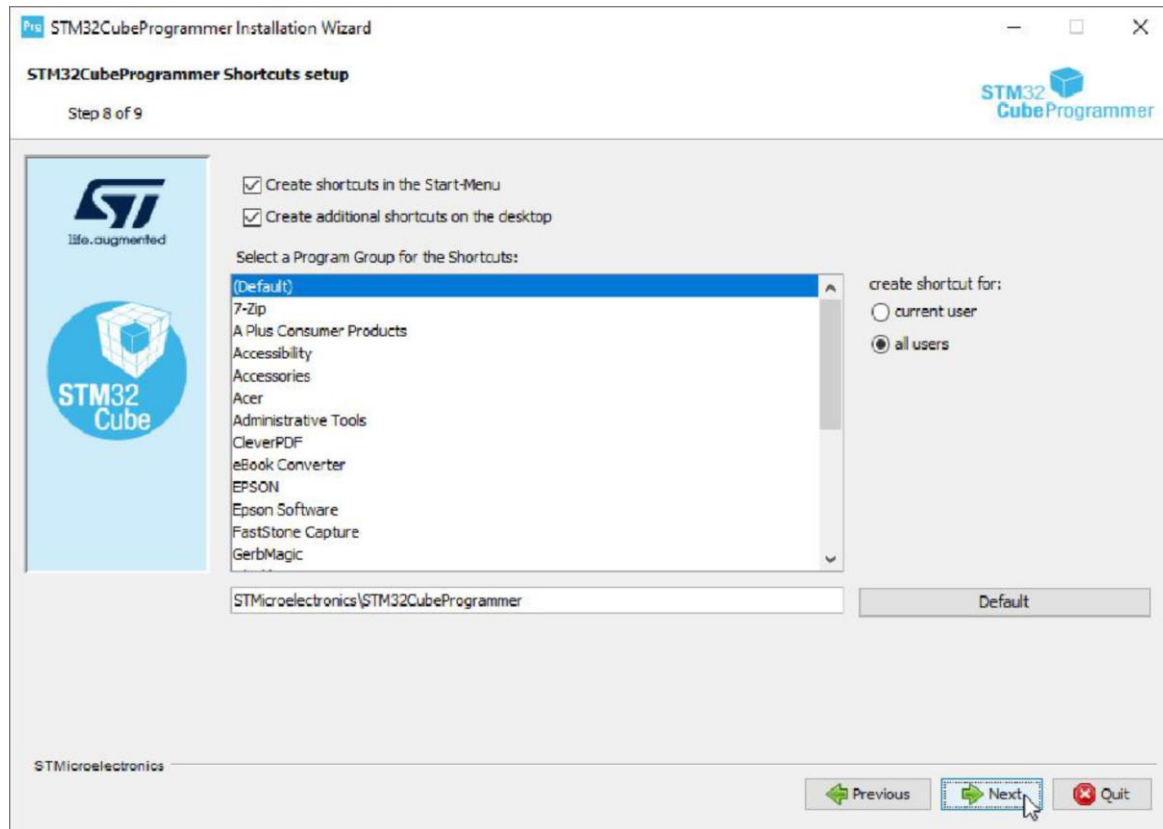
6



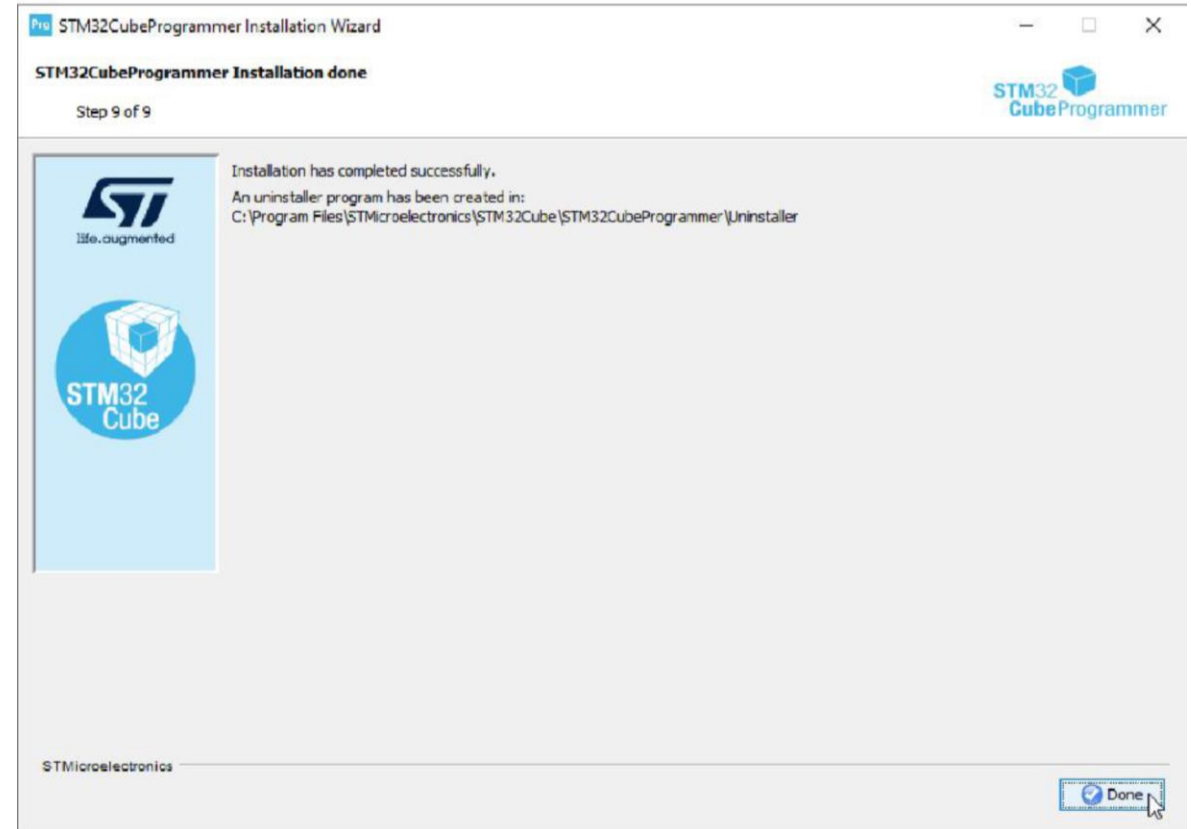
# ติดตั้ง Driver



8

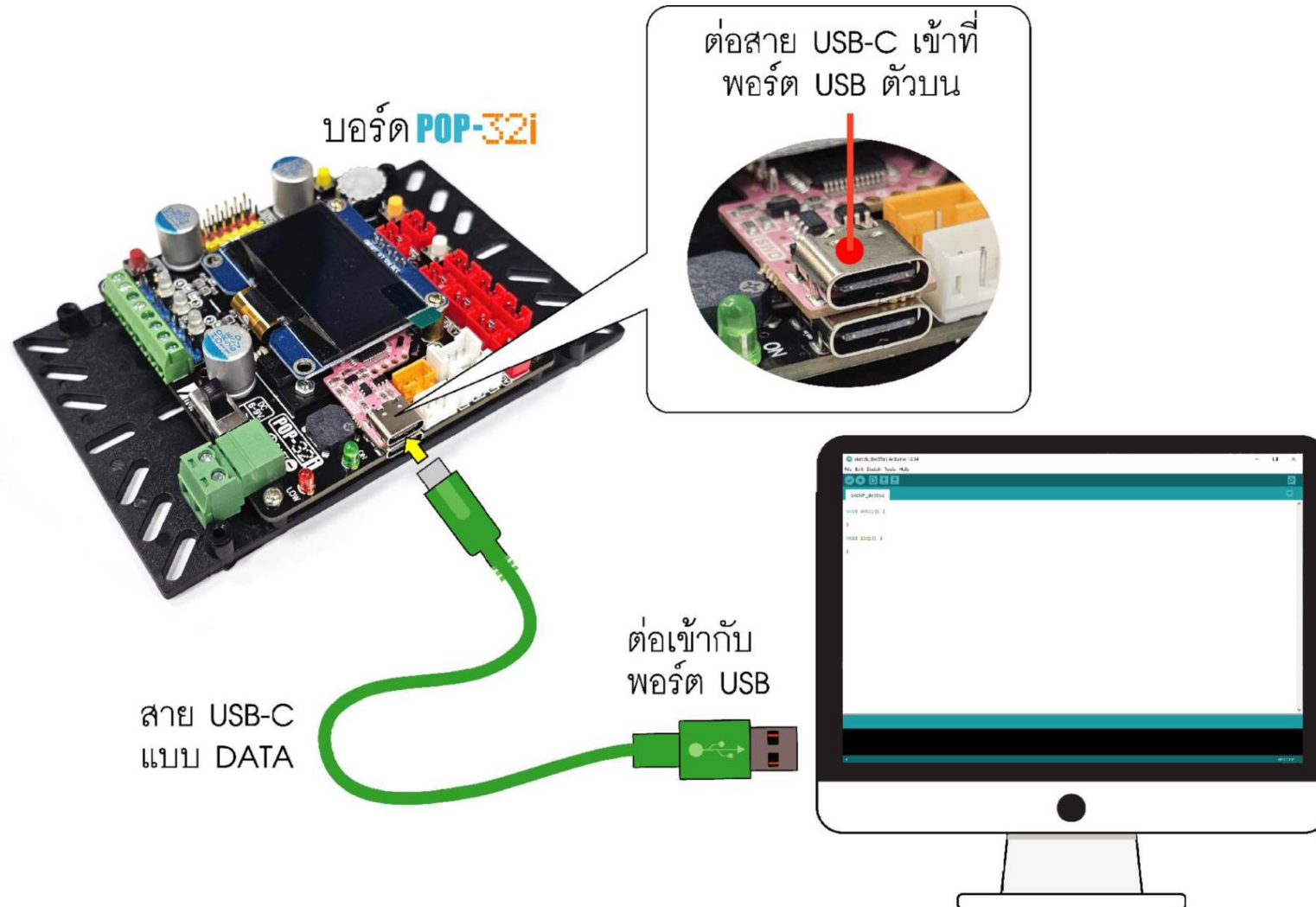


9





# ทดสอบอัปโหลดโปรแกรมไปยัง POP-32i





## แบตเตอรี่ Li-Po



แดง บวก  
ดำ ลบ



แดง บวก  
ดำ ลบ

- 2 เซล 7.4V
- กระแส 1100 mA
- จ่ายกระแส 30 เท่า
- ชาร์จ 5 เท่า

# รายละเอียดแบตเตอรี่ ลิเทียม-โพลิเมอร์



1 เซล 3.7V อนุกรมกัน 2 เซล = 7.4V

จ่ายไฟ 1100 mA ต่อเนื่องได้ประมาณ 1 ชั่วโมง

จ่ายกระแสชั่วขณะได้  $1100 \times 30 = 33000\text{mA}$  O\_o!

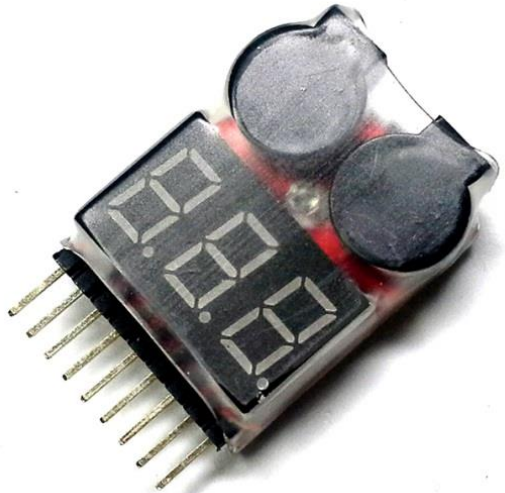
ชาร์จได้ 5 เท่า  $1100 \times 5 = 5500\text{mA}$  ใช้เวลาประมาณ 20 นาที

# ตรวจสอบแรงดันต่ำ

แสดงไฟและเตือน

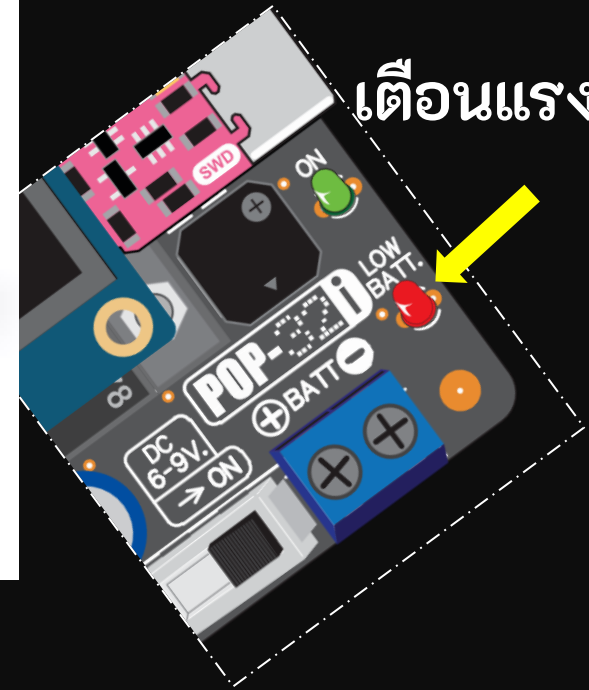


วัด Volt อย่างเดียว



วัดโวลต์และเตือน

เตือนแรงดันต่ำ



กะพริบเมื่อไหร่เปลี่ยนแบตเตอรี่ที่



# เครื่องชาร์จแบตเตอรี่

220V



Li-Po 7.4V (2 เซล) 1,100 mAh



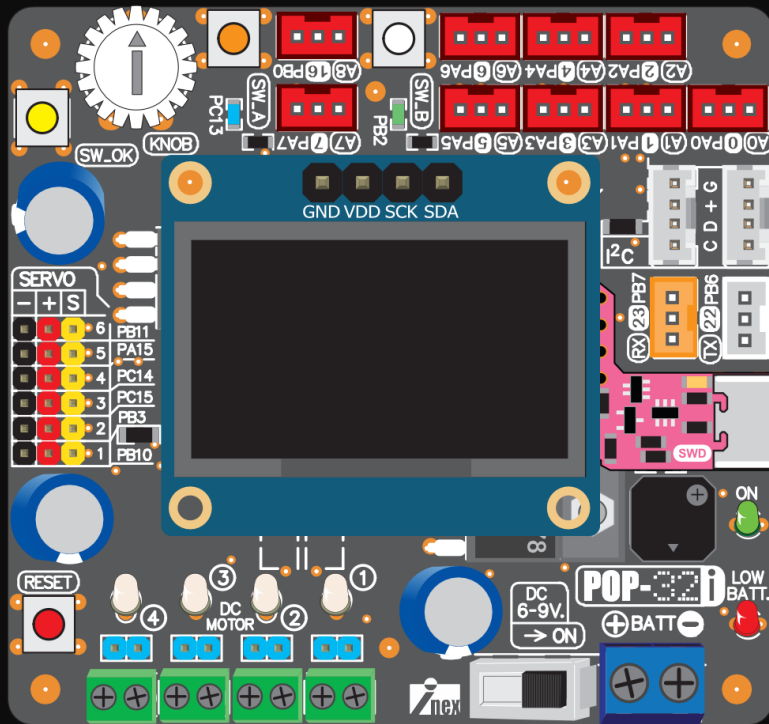
ชาร์จถ่าน Ni-MH  
ชาร์จแบตเตอรี่ Li-Po  
ชาร์จแบตเตอรี่รถยนต์

# เครื่องชาร์จแบตเตอรี่แบบ USB

ชาร์จแบตเตอรี่ Li-Po แบบ 2 เซล



# ขั้นตอนเชื่อมต่อกับคอมพิวเตอร์



1

ต่อสาย USB เข้ากับคอมพิวเตอร์

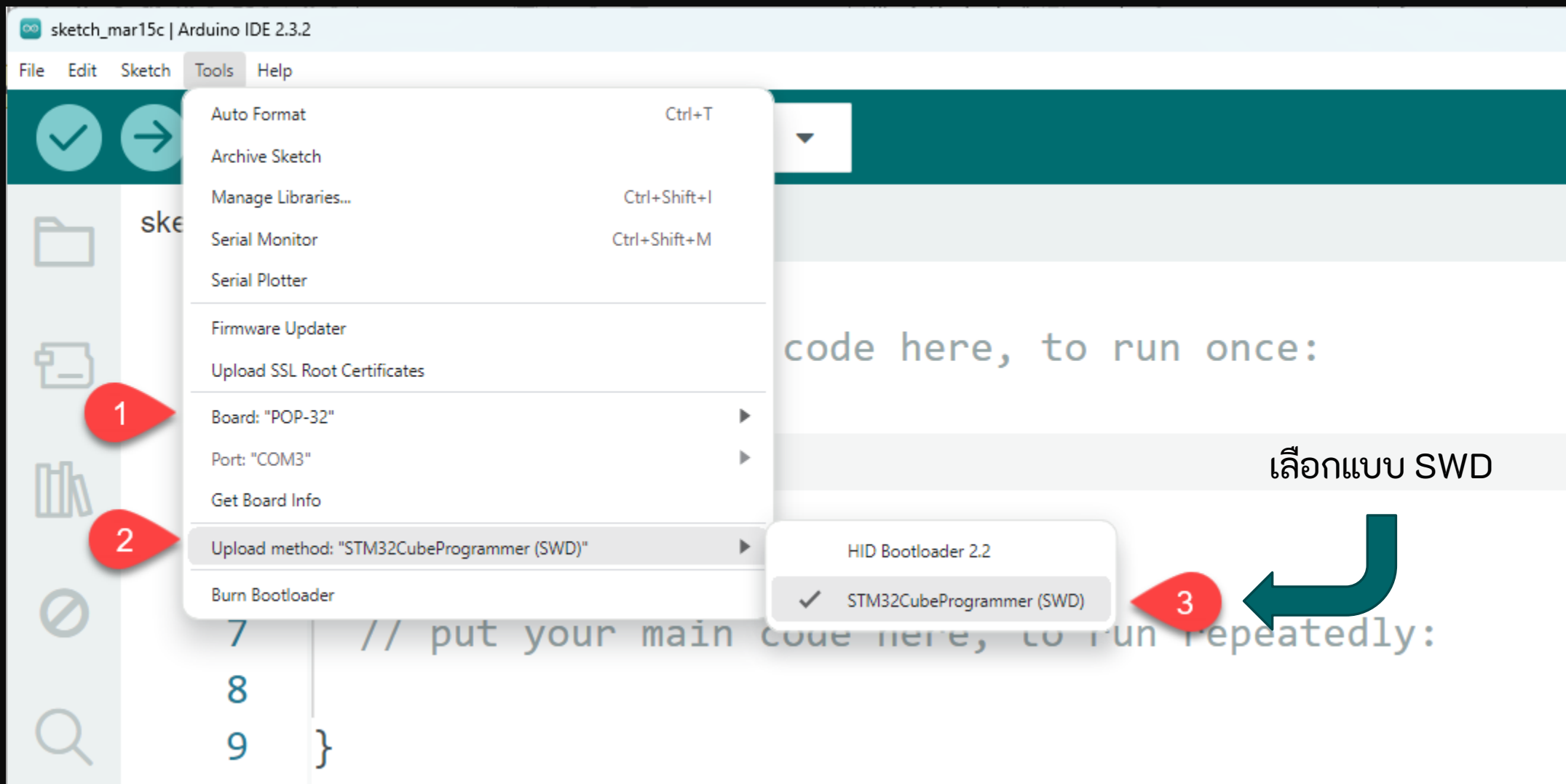
2

ต่อสาย USB เข้ากับ POP-32 ด้านบนสุด

3

เปิดสวิตช์จ่ายไฟ POP-32

# เลือกบอร์ดเป็น POP-32





# รูปแบบการทำงานของโปรแกรม Arduino

**#include <POP32.h>**

ผนวกรวมชุดคำสั่งสำหรับ POP-32

**void setup() {**

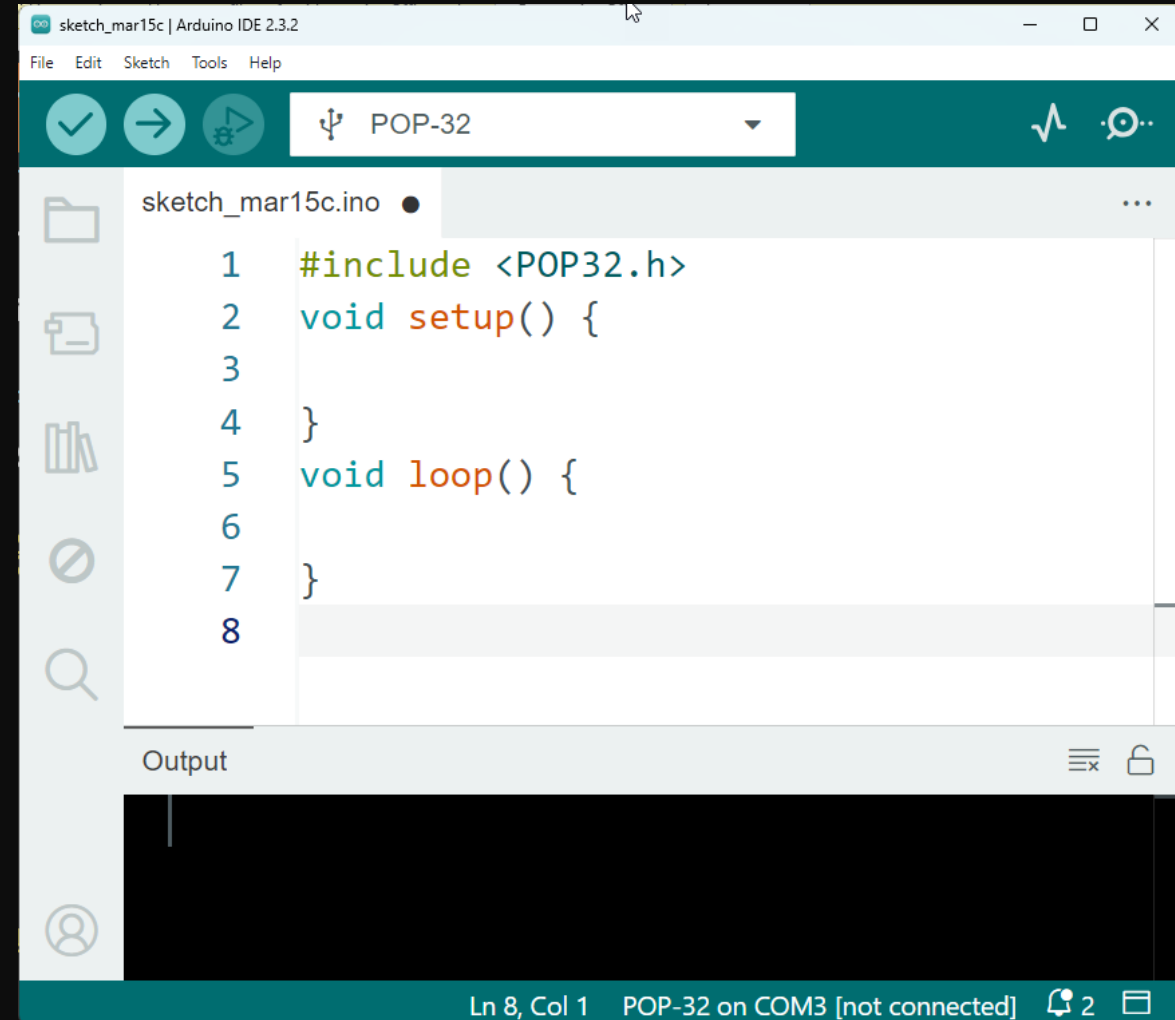
สำหรับกำหนดค่า เกิดขึ้นครั้งเดียว

**}**

**void loop() {**

โปรแกรมหลักทำงานต่อเนื่อง

**}**



# ตัวอย่างลำดับการประมวลผลของฟังก์ชัน setup และ loop

```
#include <POP32.h>
```

```
void setup()
```

```
{
```

```
    command1;
```

```
    command2;
```

```
    command3;
```

```
}
```

```
void loop()
```

```
{
```

```
    command4;
```

```
    command5;
```

```
    command6;
```

```
}
```

การประมวลผลโปรแกรมจะทำงานดังนี้

```
command1;
```

```
command2;
```

```
command3;
```

```
command4;
```

```
command5;
```

```
command6;
```

```
command4;
```

```
command5;
```

```
command6;
```

```
command4;
```

```
command5;
```

```
command6;
```

```
...
```

```
...
```

```
...
```

รอบเดียวสำหรับ setup

รอบที่ 1 สำหรับ loop

รอบที่ 2 สำหรับ loop

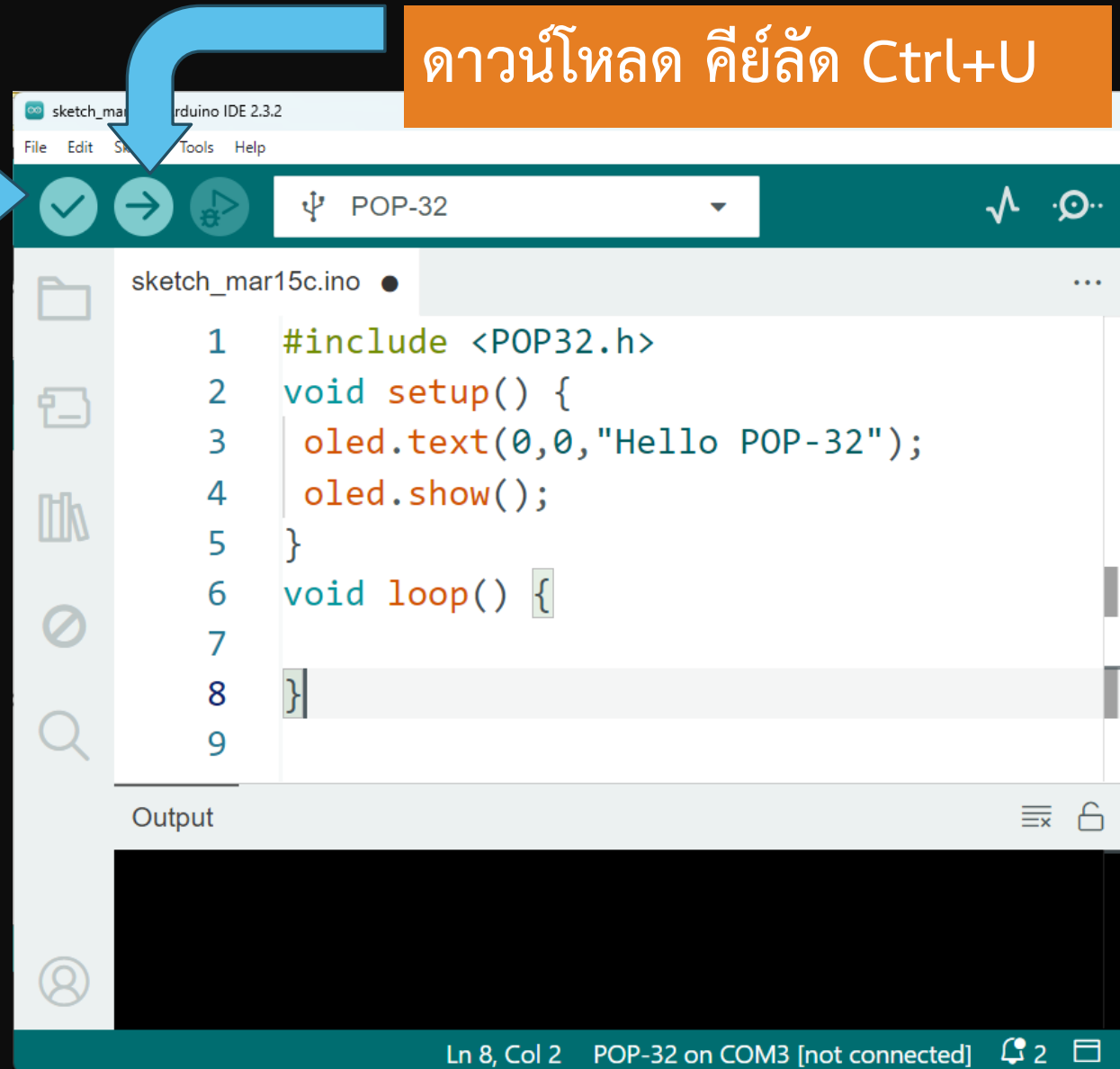
รอบที่ 3 สำหรับ loop

รอบที่ n สำหรับ loop

# เขียนโปรแกรมผ่าน Arduino

คอมไพล์ คีย์ลัด Ctrl+R

ดาวน์โหลด คีย์ลัด Ctrl+U



# แจ้งผลการคอมไพล์และขนาดหน่วยความจำที่ใช้

อัปโหลดโค้ดไปยัง POP-32  
คีย์ลัด Ctrl+U

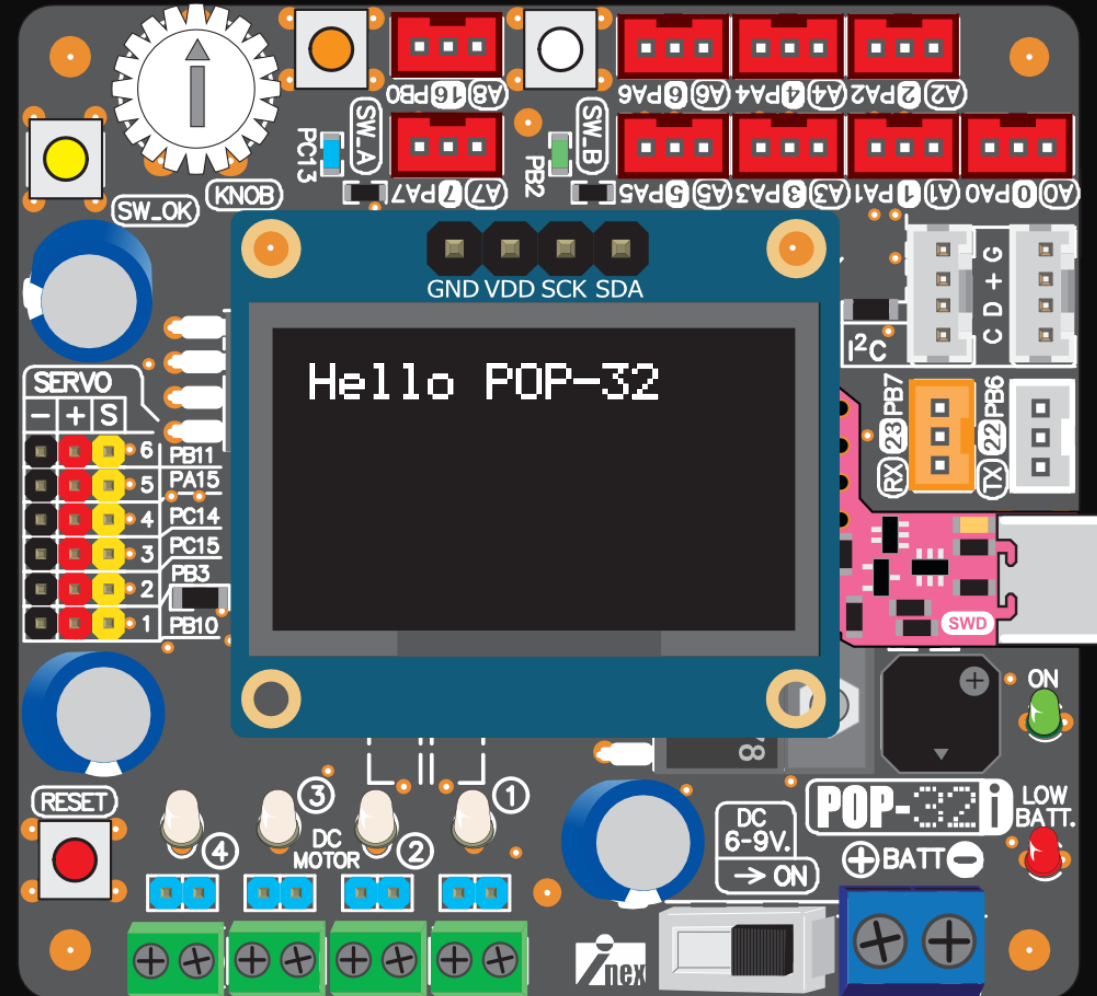


แจ้งผลว่าสมบูรณ์

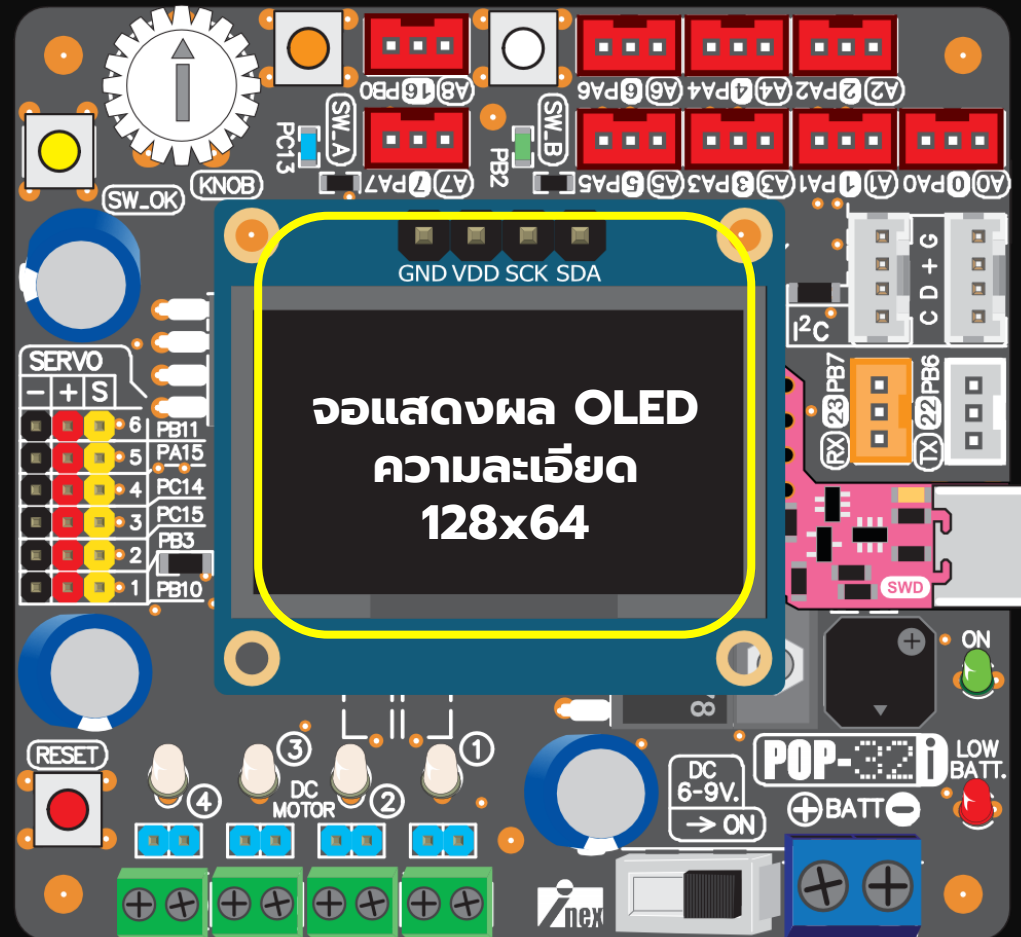
```
Output
```

```
RUNNING Program ...  
| Address:      : 0x8000000  
Application is running, Please Hold on...  
Start operation achieved successfully
```

Ln 6, Col 14 POP-32 on COM3 [not connected] 4



# กลุ่มคำสั่งแสดงผลที่หน้าจอ OLED



## ชุดคำสั่งสำหรับควบคุมการแสดงผล

คลาส OLED\_I2C\_SSD1309 เป็นคลาสที่รวบรวมเมธอดต่างๆ สำหรับควบคุมการแสดงผลกราฟิกแบบ OLED ความละเอียด 128x64 พิกเซล อย่างไรก็ดีเพื่อความสะดวกในการควบคุมการแสดงผลกราฟิกภายในคลาส OLED\_I2C\_SSD1309 ได้ทำการประกาศสร้างออบเจกต์ที่ทำการสืบทอดมาจากคลาส OLED\_I2C\_SSD1309 โดยตั้งชื่อเป็น **oled** เป็นที่เรียบร้อยแล้ว



## คำสั่งสำหรับควบคุมการแสดงผลข้อความ

**oled.text** แสดงข้อความที่จอแสดงผล OLED ได้ 21 ตัว 8 บรรทัด (textSize=1)

รูปแบบ

```
oled.text(x,y,*p,...);
```

พารามิเตอร์

**x** คือตำแหน่งบรรทัดมีค่าตั้งแต่ 0-7

**y** คือตำแหน่งตัวอักษรมีค่าตั้งแต่ 0-20

**\*p** คือข้อความที่ต้องการนำมาแสดง

ค่าพิเศษ

**%d** แสดงตัวเลขจำนวนเต็มในช่วง -2,147,483,648 ถึง 2,147,483,647

**%h** แสดงตัวเลขฐานสิบหก

**%b** แสดงตัวเลขฐานสอง

**%f** แสดงผลตัวเลขจำนวนจริง (แสดงทศนิยม 3 หลัก)

## คำสั่งสำหรับควบคุมการแสดงผลข้อความ

`oled.show`

ทำหน้าที่อัปเดตการแสดงผล

รูปแบบ

`oled.show()`

พารามิเตอร์      ไม่มี

การคืนค่า          ไม่มี

## รูปแบบ คำสั่ง delay

**delay** หน่วงเวลาในหน่วยมิลลิวินาที  
รูปแบบ

**delay**(time)

พารามิเตอร์

time คือ เวลาที่ต้องการหน่วงในหน่วยมิลลิวินาที

ตัวอย่างเช่น

**delay**(500); // หน่วงเวลา 500 มิลลิวินาที(0.5 วินาที)

**delay**(2000); // หน่วงเวลา 2000 มิลลิวินาที(2 วินาที)

## ตัวแปร (ที่ใช้งานบ่อย)

|       |                                                |                 |
|-------|------------------------------------------------|-----------------|
| byte  | 0-255                                          | (unsigned char) |
| bool  | 0-1                                            | true false      |
| int   | -2147483648 ถึง 2147483647                     |                 |
| char  | -128 ถึง 127                                   |                 |
| float | $-3.4 \times 10^{38}$ ถึง $3.4 \times 10^{38}$ |                 |

หาข้อมูลเพิ่มเติมจาก [reference](#)

# ชุดคำสั่งอื่นๆ สำหรับใช้งานอ็อบเจกต์ oled

## แสดงตัวอักษร

text  
textSize  
mode  
textColor  
textBackgroundColor  
clear  
fillScreen

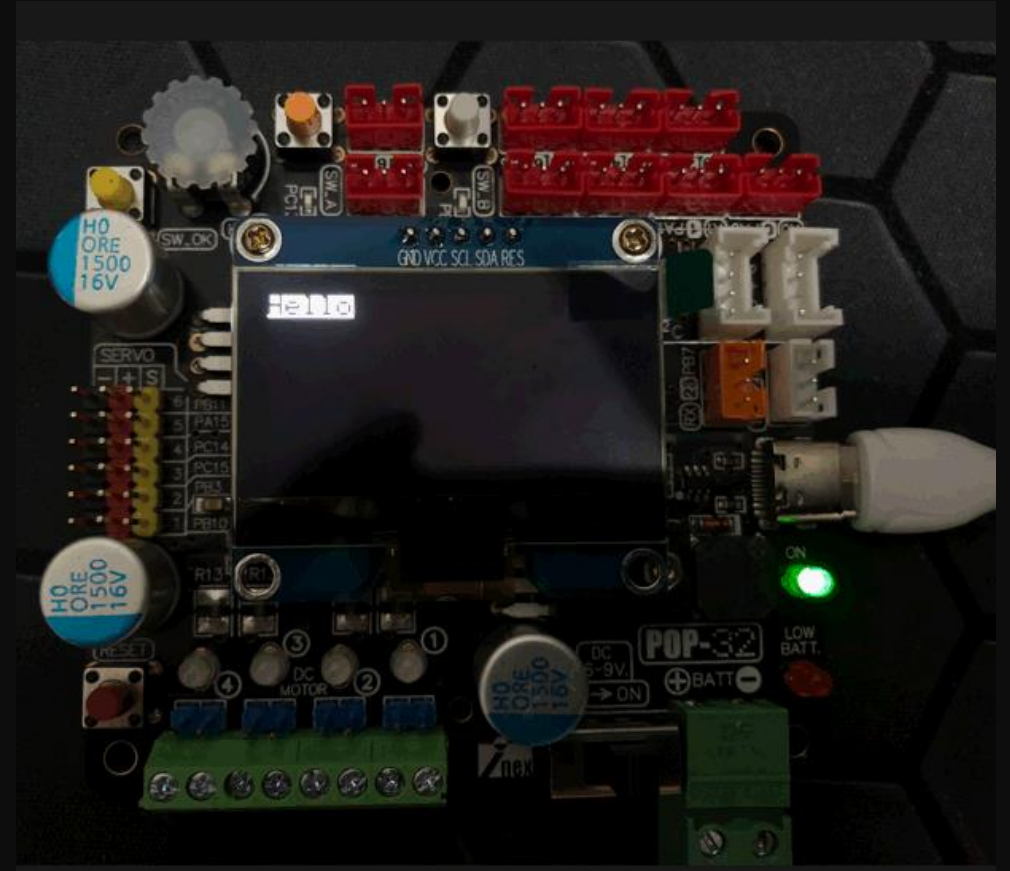
## แสดงกราฟิก

drawPixel  
drawRect  
fillRect  
drawLine  
drawCircle  
fillCircle

# ตัวอย่างกำหนดสีตัวอักษร

```
#include <POP32.h>
void setup(){
  oled.setTextColor(BLACK,WHITE);
  oled.text(0,0,"Hello world");
  oled.show();
}
void loop(){}

```

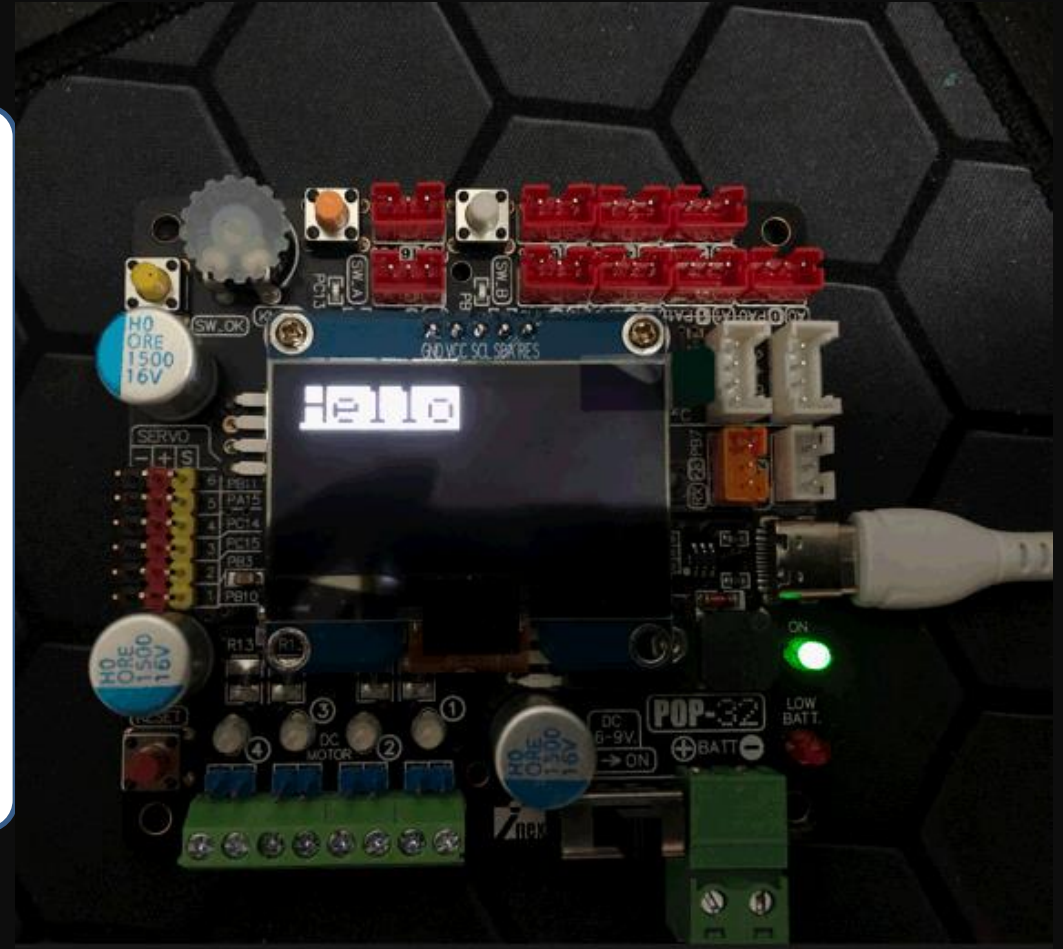


ตัวอักษรสีดำพื้นหลังตัวอักษรเป็นสีขาวเท่านั้น

# ตัวอย่างกำหนดขนาดตัวอักษร

```
#include <POP32.h>
void setup(){
  oled.setTextSize(2);
  oled.setTextColor(BLACK,WHITE);
  oled.text(0,0,"Hello world");
  oled.show();
}
void loop(){}

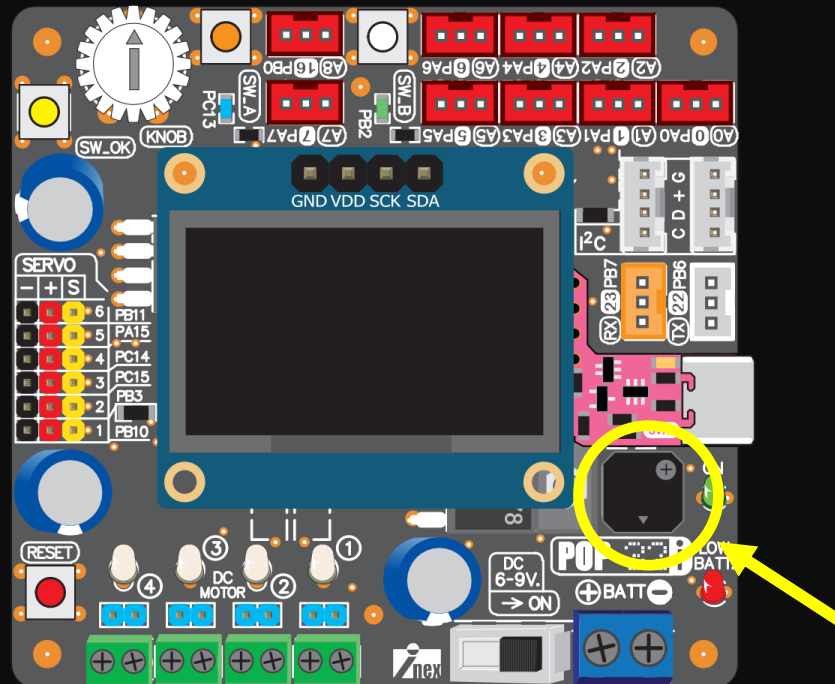
```



ตัวอักษรสีดำพื้นหลังตัวอักษรเป็นสีขาวมีขนาดเป็น 2 เท่า



# กลุ่มคำสั่งสร้างสัญญาณเสียงออกมาโพงเป็ยโซ



# การสร้างเสียงออกลำโพง

**beep()**

**sound(freq,time)**

**freq** คือ ความถี่เสียง

**time** คือ เวลา 1/100 วินาที

มีค่าความถี่ย่าน 300 Hz ถึง 3000 Hz

# ตัวอย่างโปรแกรมสร้างเสียงออกลำโพง

## ตัวอย่างการใช้งานคำสั่ง beep

```
#include <POP32.h>
void setup(){}
void loop(){
    beep();
    delay(500);
}
```

สร้างเสียง 500 Hz นาน 0.1 วินาที ต่อเนื่อง

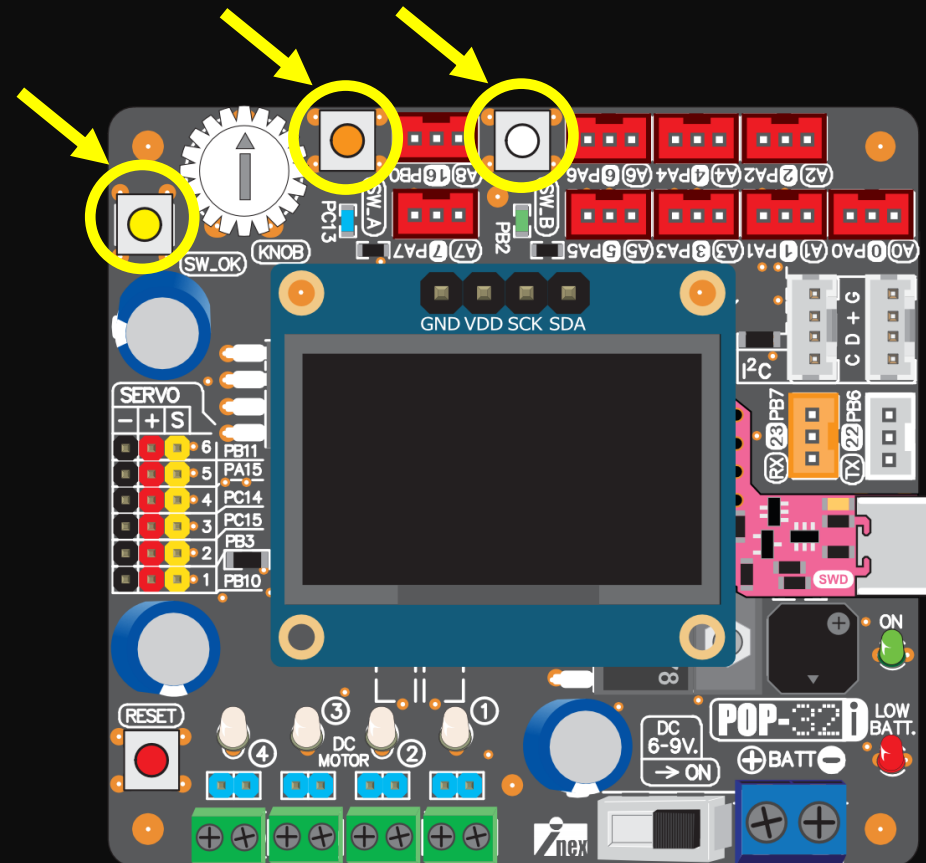
## ตัวอย่างการใช้งานคำสั่ง sound

```
#include <POP32.h>
void setup(){}
void loop(){
    sound(1000,100);
    delay(500);
}
```

สร้างเสียง 1 kHz นาน 0.1 วินาที ต่อเนื่อง

# กลุ่มคำสั่งอ่านค่าสถานะสวิตช์

A,B และ OK



# คำสั่งอ่านค่าสถานะสวิตช์

ทำหน้าที่อ่านค่าสถานะการกดสวิตช์ A, B และ OK

## รูปแบบ

SW\_A()

SW\_B()

SW\_OK()

## การคืนค่า

- คืนค่าเป็น true(1) ในกรณีที่สวิตช์ถูกกด
- คืนค่าเป็น false(0) ในกรณีที่สวิตช์ไม่ถูกกด

# คำสั่งรอกการกดสวิตช์

ทำหน้าที่รอกจนกระทั่งสวิตช์ที่กำหนด

(LED ประจำตัวสวิตช์ จะกระพริบต่อเนื่องเพื่อแจ้งผู้ใช้งานในระหว่างรอกการกดสวิตช์)

## รูปแบบ

waitSW\_A()

waitSW\_B()

waitSW\_OK()

waitAnykey()

## การคืนค่า

ไม่มีการคืนค่า

# คำสั่งตรวจสอบเงื่อนไข if-else

```
if (เงื่อนไข) {  
    คำสั่ง1 ทำเมื่อเงื่อนไขเป็นจริง;  
    ...  
}  
else {  
    คำสั่ง2 ทำเมื่อเงื่อนไขเป็นเท็จ;  
    ...  
}
```

ตัวแปร Boolean

จริง : True : 1

เท็จ : False : 0



# ตัวอย่างตรวจจบการกดสวิตช์ A

```
#include <POP32.h>
void setup(){}
void loop()
{
  if(SW_A())
  {
    beep();
    delay(200);
  }else{
    sound(2000,200);
    delay(200);
  }
}
```

หลังจากกดสวิตช์ A หรือ ไม่กดสวิตช์เสียงจะถูกกำเนิดความถี่ที่ต่างกัน

# ตัวอย่างตรวจจบการกดสวิตช์ A และ B

```
#include <POP32.h>
void setup(){
}
void loop()
{
    if(SW_A())
    {
        beep();
        delay(200);
    }
    if(SW_B())
    {
        sound(2000,200);
        delay(200);
    }
}
```

หลังจากกดสวิตช์ A และ B เสียงจะถูกกำเนิดความถี่ที่ต่างกัน

## ตัวอย่างการใช้ && (AND)

```
#include <POP32.h>
void setup(){
}
void loop()
{
    if(SW_A() && SW_B())
    {
        beep();
        delay(200);
    }else{
        sound(2000, 200);
        delay(200);
    }
}
```

กดสองปุ่มพร้อมกัน

ไม่กดสวิตช์

## ตัวอย่างการใช้ || (OR)

```
#include <POP32.h>
void setup(){
}
void loop()
{
    if(SW_A() || SW_B())
    {
        beep();
        delay(200);
    }else{
        sound(2000, 200);
        delay(200);
    }
}
```

กดปุ่มใดปุ่มหนึ่ง

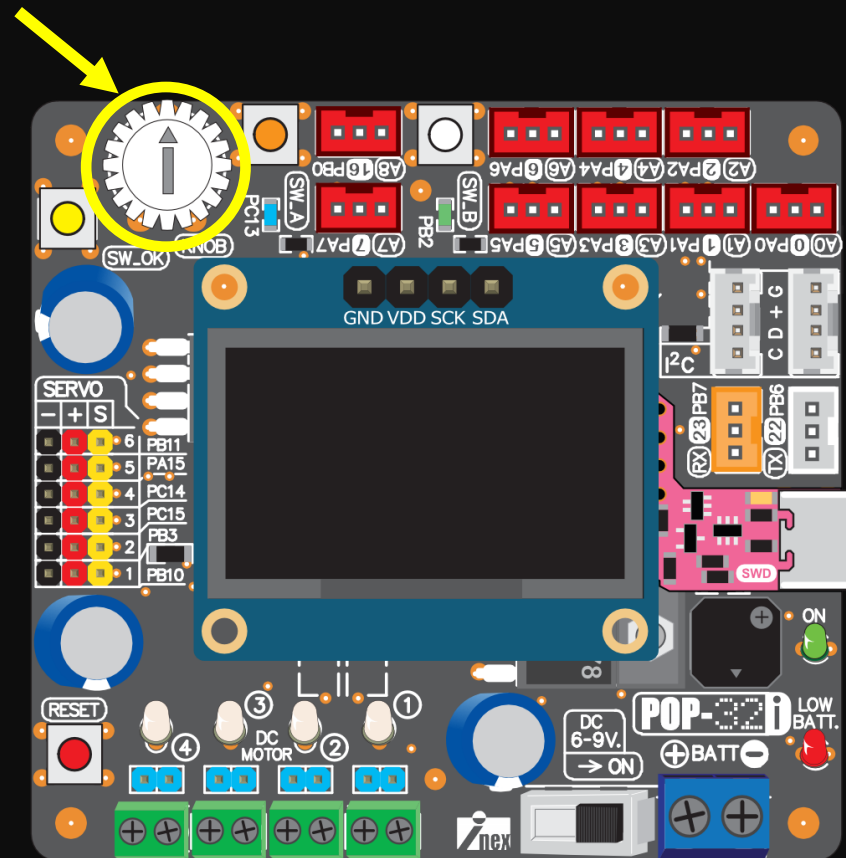
ไม่กดสวิตช์

# ตัวอย่างรอกจนกระทั่งมีการกดสวิตช์ OK

```
#include <POP32.h>
void setup(){
    oled.text(0,0,"Press SW_OK");
    oled.show();
    waitSW_OK();
}
void loop(){
    beep();
    delay(1000);
}
```

หลังจากกดสวิตช์ OK เสียงจะถูกกำเนิดทุกๆ 1 วินาที

# กลุ่มคำสั่งอ่านค่า KNOB



## คำสั่ง knob()

ทำหน้าที่อ่านค่าอะนาลอกจาก KNOB  
ในช่วง 0 ถึง 4000

รูปแบบ

knob()

การคืนค่า

คืนค่าตั้งแต่ 0 ถึง 4000 จากการปรับค่า

## คำสั่ง knob(x)

ทำหน้าที่อ่านค่าอะนาลอกจาก KNOB  
ในช่วง 0 ถึง x ตามที่กำหนด

รูปแบบ

knob(x)

การคืนค่า

คืนค่าตั้งแต่ 0 ถึง x จากการปรับค่า



## คำสั่ง knob(x,y)

ทำหน้าที่อ่านค่าอะนาลอกจาก KNOB ในช่วง x ถึง y ตามที่กำหนด

รูปแบบ

knob(x,y)

การคืนค่า

คืนค่าตั้งแต่ x ถึง y จากการปรับค่า

## ตัวอย่างอ่านค่า knob

```
#include <POP32.h>
void setup() {
}
void loop() {
    oled.text(0,0,"knob=%d ",knob());
    oled.show();
}
```

ทดสอบปรับหมุนที่ knob แล้วลอง  
เปลี่ยนการอ่านค่าทั้ง 3 รูปแบบ

# รูปแบบการสร้างฟังก์ชัน

ชื่อฟังก์ชัน

ตัวแปรที่ส่งไปยังฟังก์ชัน

```
void Fb(int spd=100) {  
    ชุดคำสั่ง  
    speed = 100 * (spd/100) ;  
    ...  
}
```

ชุดคำสั่งในฟังก์ชัน

การใช้งานฟังก์ชัน **Fb(100);**

# รูปแบบการสร้างฟังก์ชัน

ชื่อฟังก์ชัน

ตัวแปรที่ส่งไปยังฟังก์ชัน

```
void wheel(int s1, int s2, int s3)
    motor(1, s1);
    motor(2, s2);
    motor(3, s3);
}
```

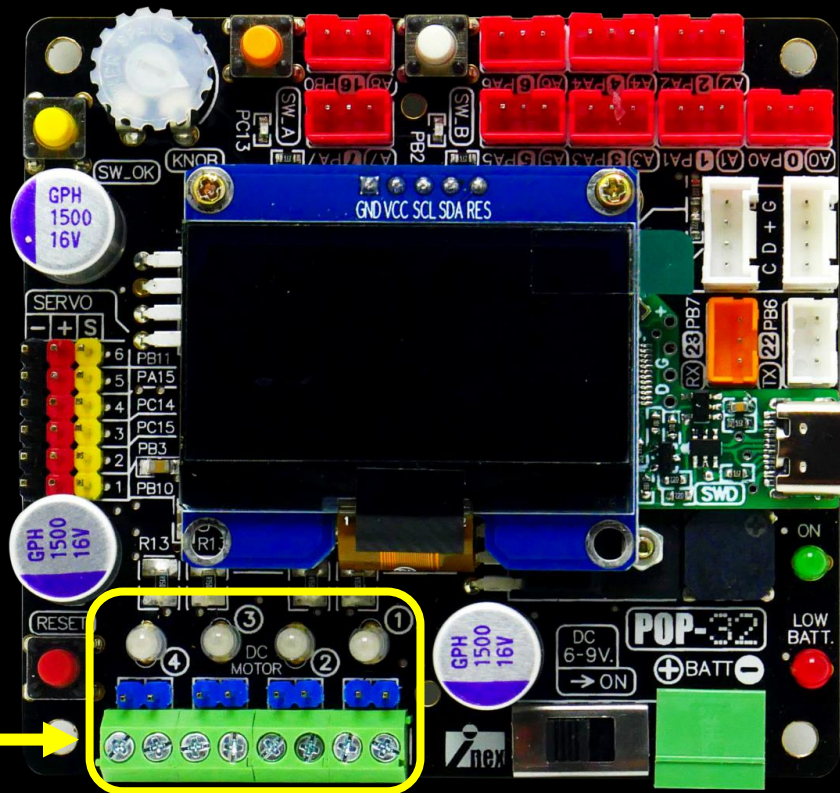
ชุดคำสั่งในฟังก์ชัน

การใช้งานฟังก์ชัน

wheel(100,100,100);

# คำสั่งขับเคลื่อนมอเตอร์ไฟตรง

จุดต่อมอเตอร์ 1-4



# คำสั่ง motor(ch,pow)

ทำหน้าที่ขับเคลื่อนมอเตอร์ช่องที่กำหนด

รูปแบบ

motor(ch,pow)

โดยที่

**ch** คือ ช่องขับ 1 ถึง 4

**pow** คือ ค่ากำลังขับในช่วง -100 ถึง 100

-กรณีเป็นค่าบวก ขับทิศทางไปข้างหน้า

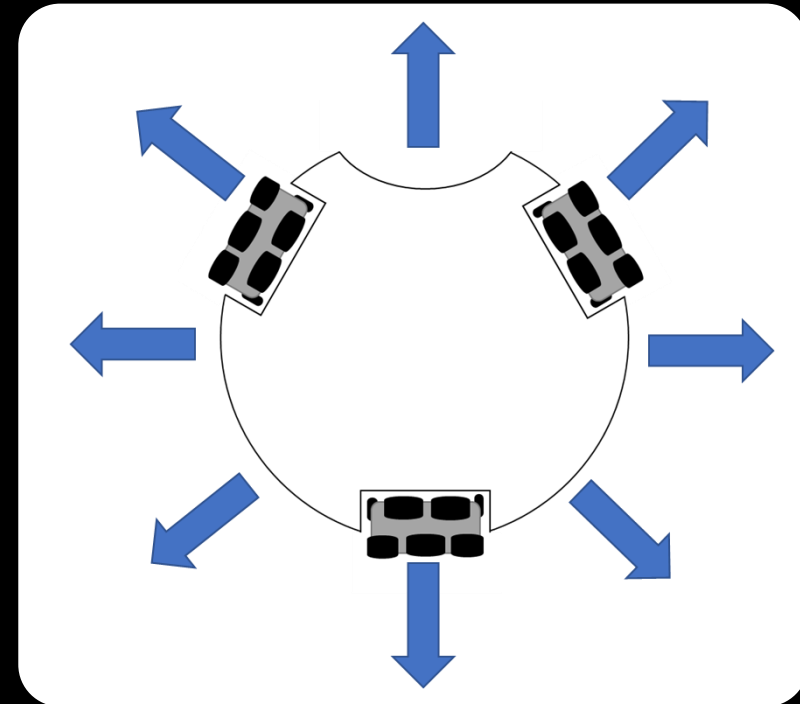
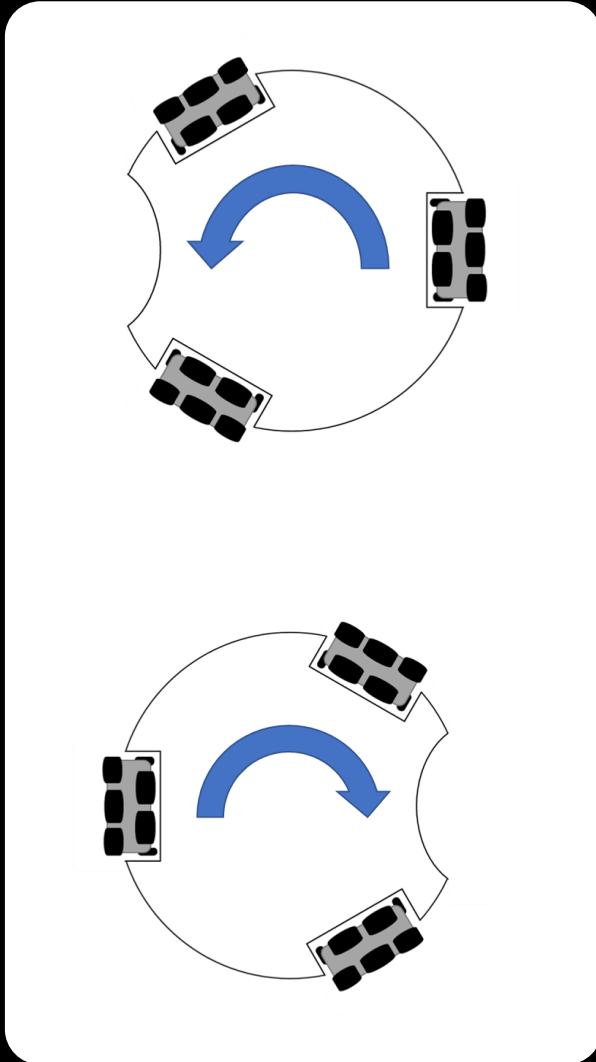
-กรณีเป็นค่าลบ ขับทิศทางถอยหลัง

การคืนค่า

ไม่มี

# การเคลื่อนที่แบบโฮโลโนมิก (Holonomic)

การเคลื่อนไหวแบบโฮโลโนมิกจะเกิดขึ้นได้หากระดับความอิสระที่ควบคุมได้เท่ากับทั้งหมด ยกตัวอย่างเช่น ล้อเลื่อนบนรถเข็นล้อปั้ง สามารถควบคุมการเคลื่อนไหวได้ในทุกองศาของความอิสระเท่าที่จะเป็นไปได้ ตรงกันข้ามกับล้อของจักรยาน



หลักการเคลื่อนที่หุ่นยนต์โดยการ  
ใช้ล้อ Omni แบบติดตั้ง 3 ล้อ



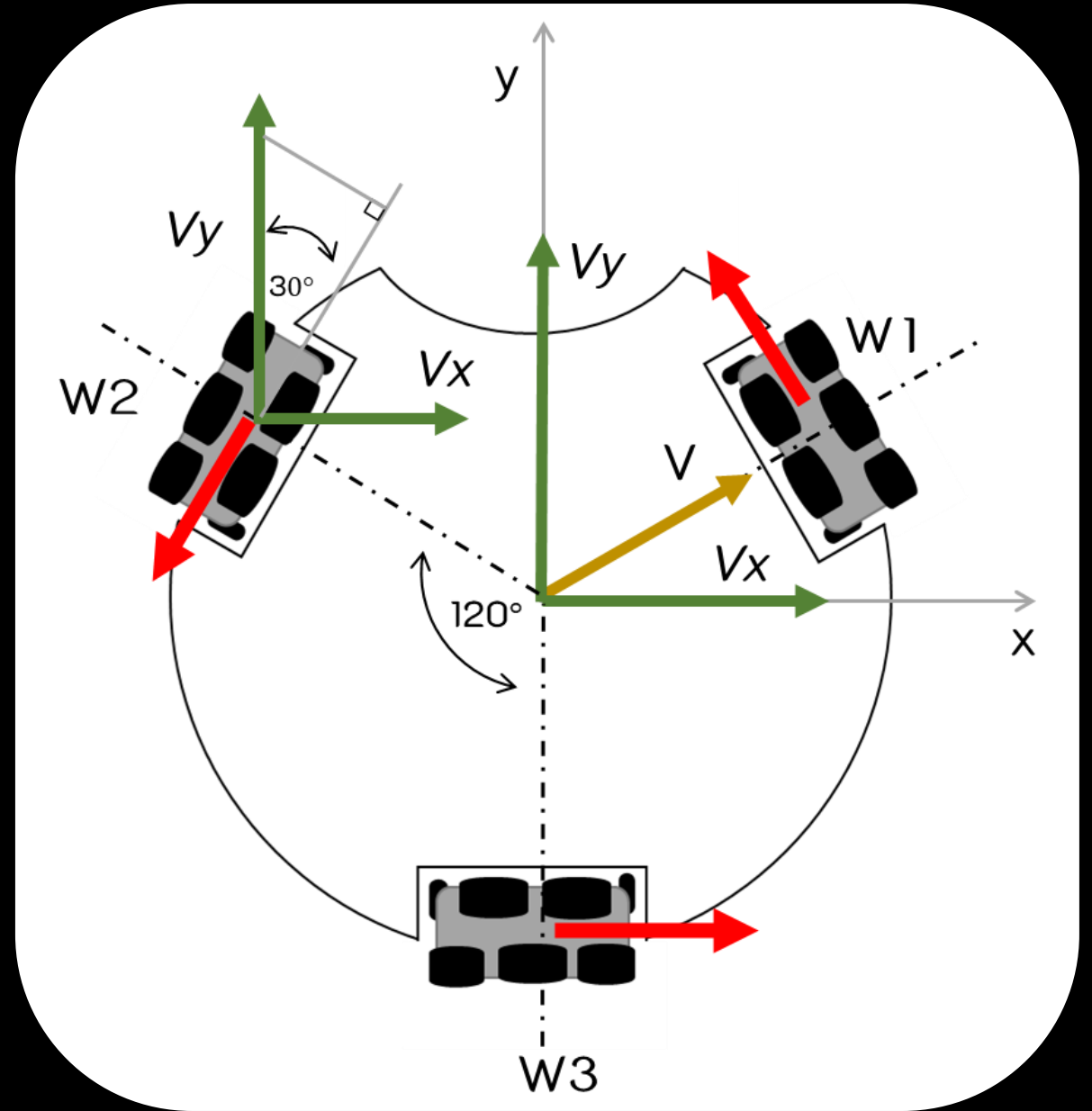
# ลักษณะล้อ Omni

ล้อ Omni เป็นล้อที่มีลักษณะพิเศษ โดยจะมีล้อที่มีขนาดเล็ก ๆ วางขวางซ้อนอยู่ในล้อหลักทำมุมตั้งฉากกับทิศทางแรงจุดของล้อ ทำให้สามารถเคลื่อนที่ในแนวตั้งและแนวนอนได้อย่างอิสระ มีความคล่องตัวสูง



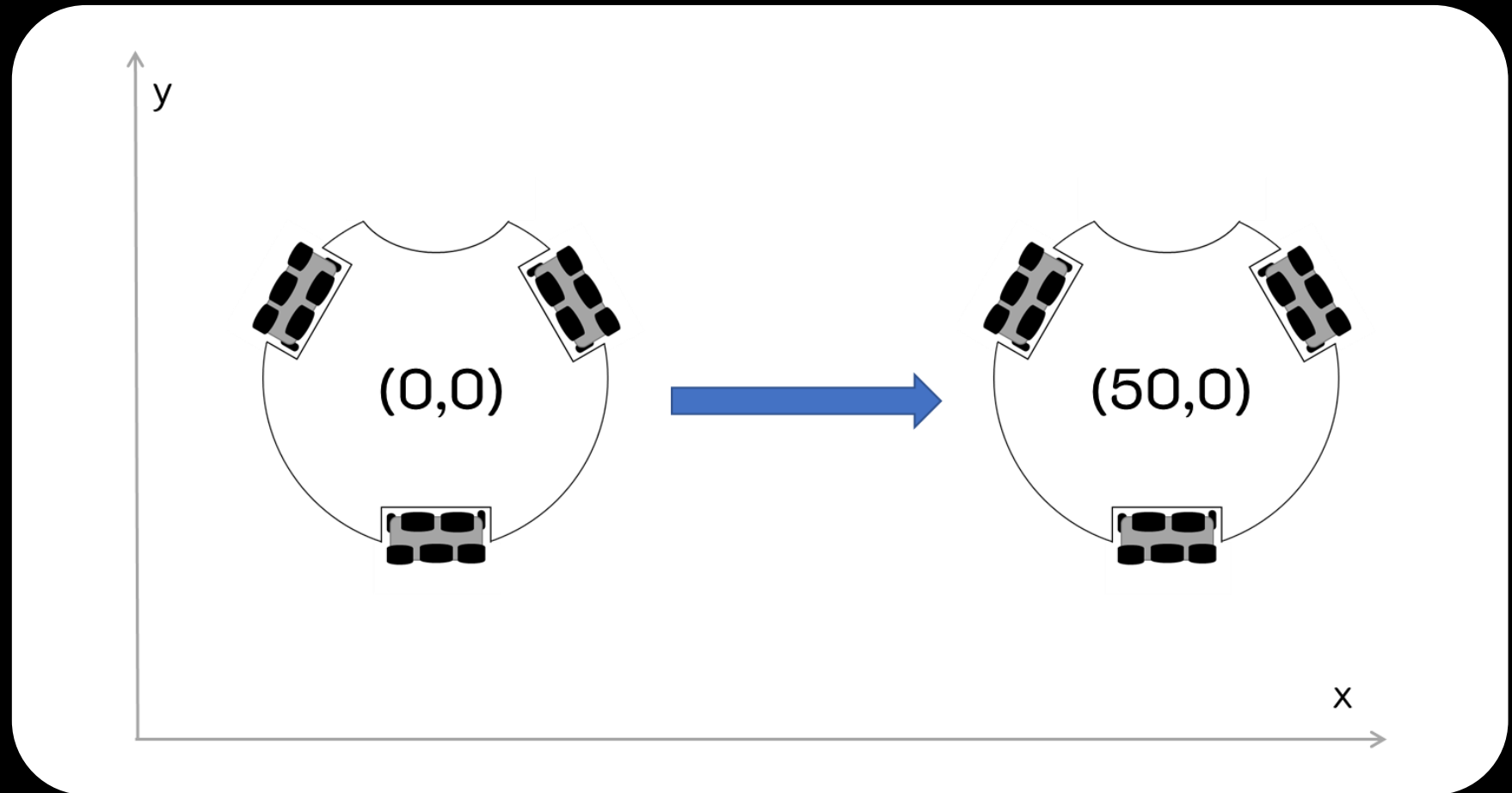
# การวางตำแหน่งล้อ Omni

แกนหมุนในแต่ละล้อ จะทำมุม  
 $120^\circ$  เท่าๆกัน

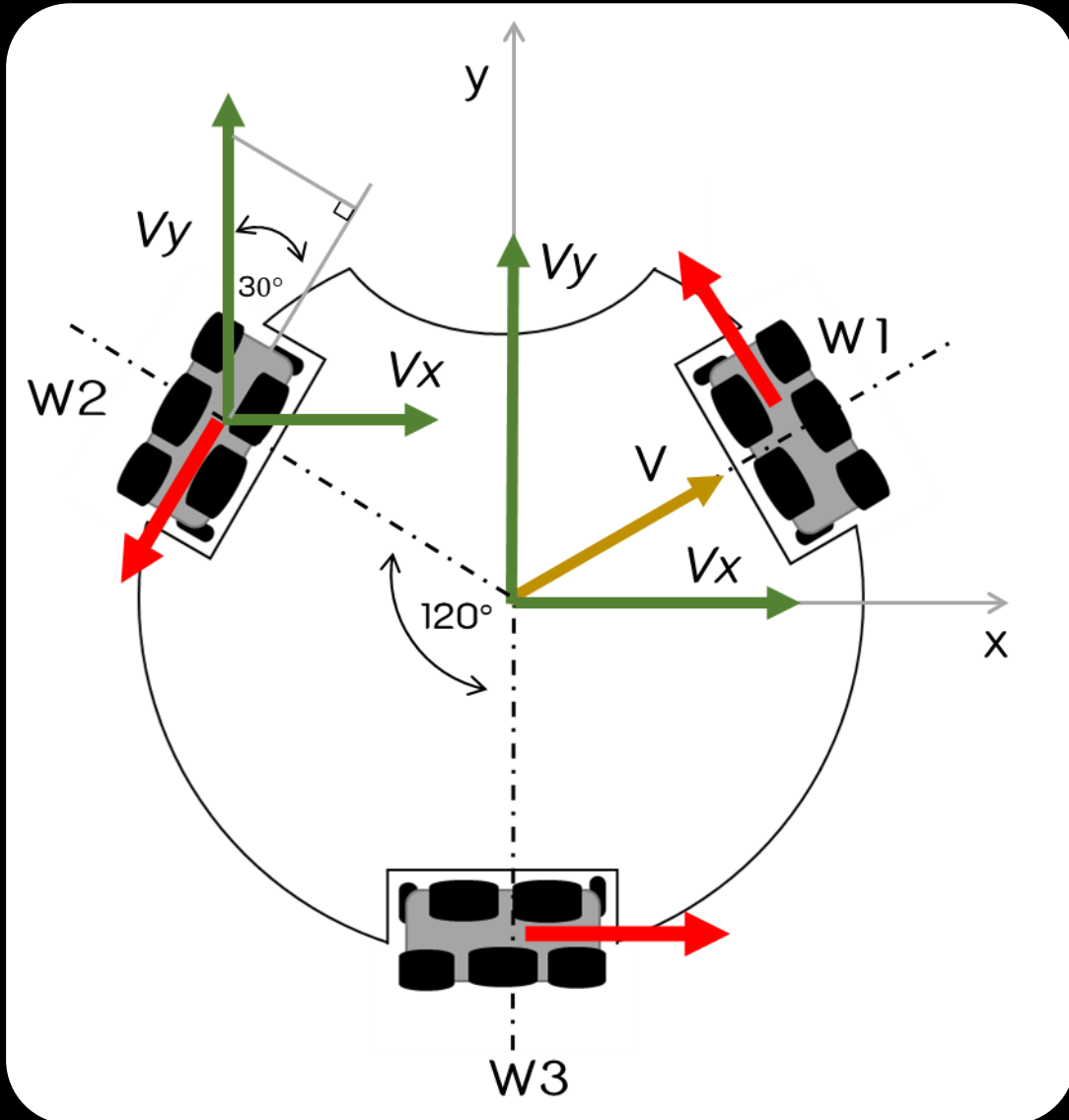


# สมการเคลื่อนที่แบบเชิงเส้น

การเคลื่อนที่แบบเชิงเส้นด้วยวิธีการนี้ คือหุ่นยนต์จะเคลื่อนที่ไปยังจุดปลายทางโดยไม่ต้องหมุนตัว เพียงแต่เป็นการเคลื่อนที่แบบเลื่อนสไลด์ไปยังจุดปลายทางหรือทิศทางได้ตามต้องการ



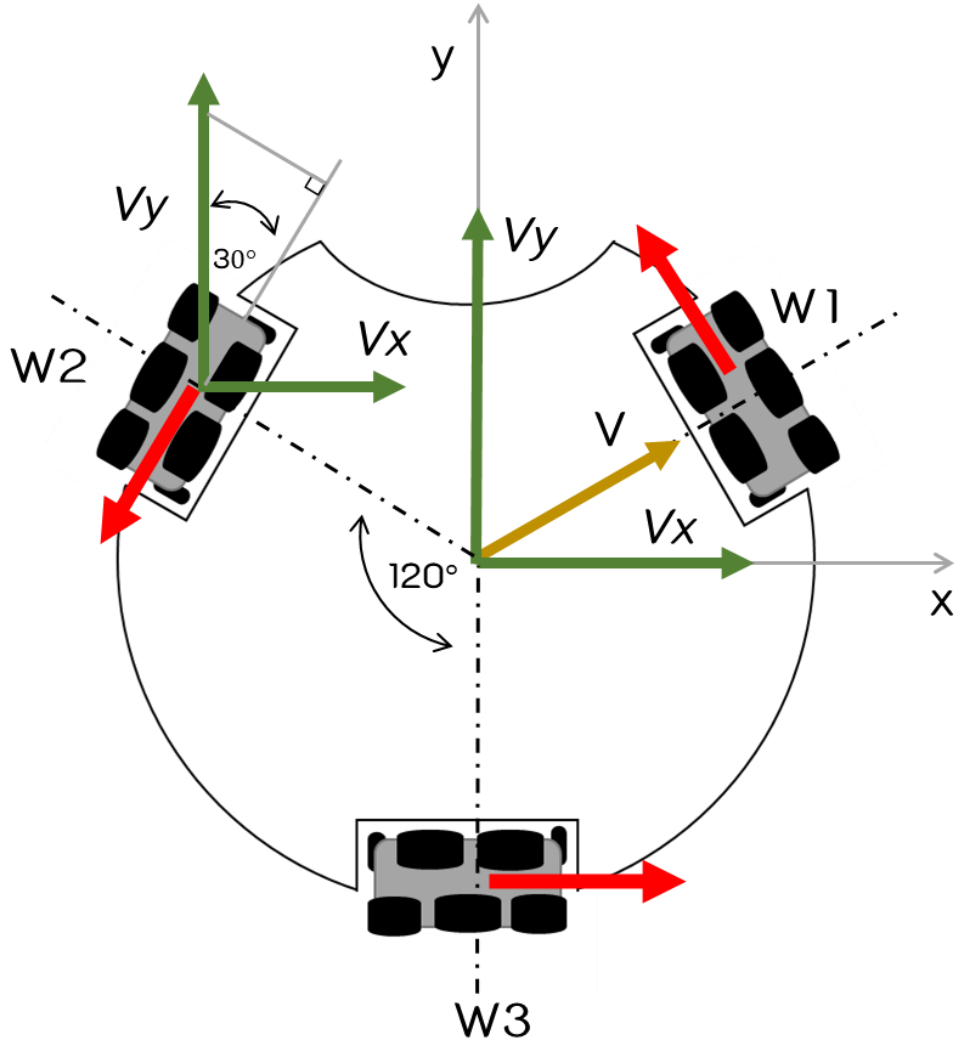
# สมการการเคลื่อนที่ ล้อที่ 1 (W1)



กำหนดทิศทางบวกของมอเตอร์โดยแทนลูกศรสีแดง  
แดงสังเกตได้ว่าชี้ไปทางแกน x ฝั่งลบและชี้ไปทางแกน  
y ฝั่งบวก โดยล้อทำมุมกับแนวแกน y  $30^\circ$  องศา จึง  
ได้รูปแบบสมการดังนี้

$$V1 = +V_y \cdot \cos(30) - V_x \cdot \sin(30)$$

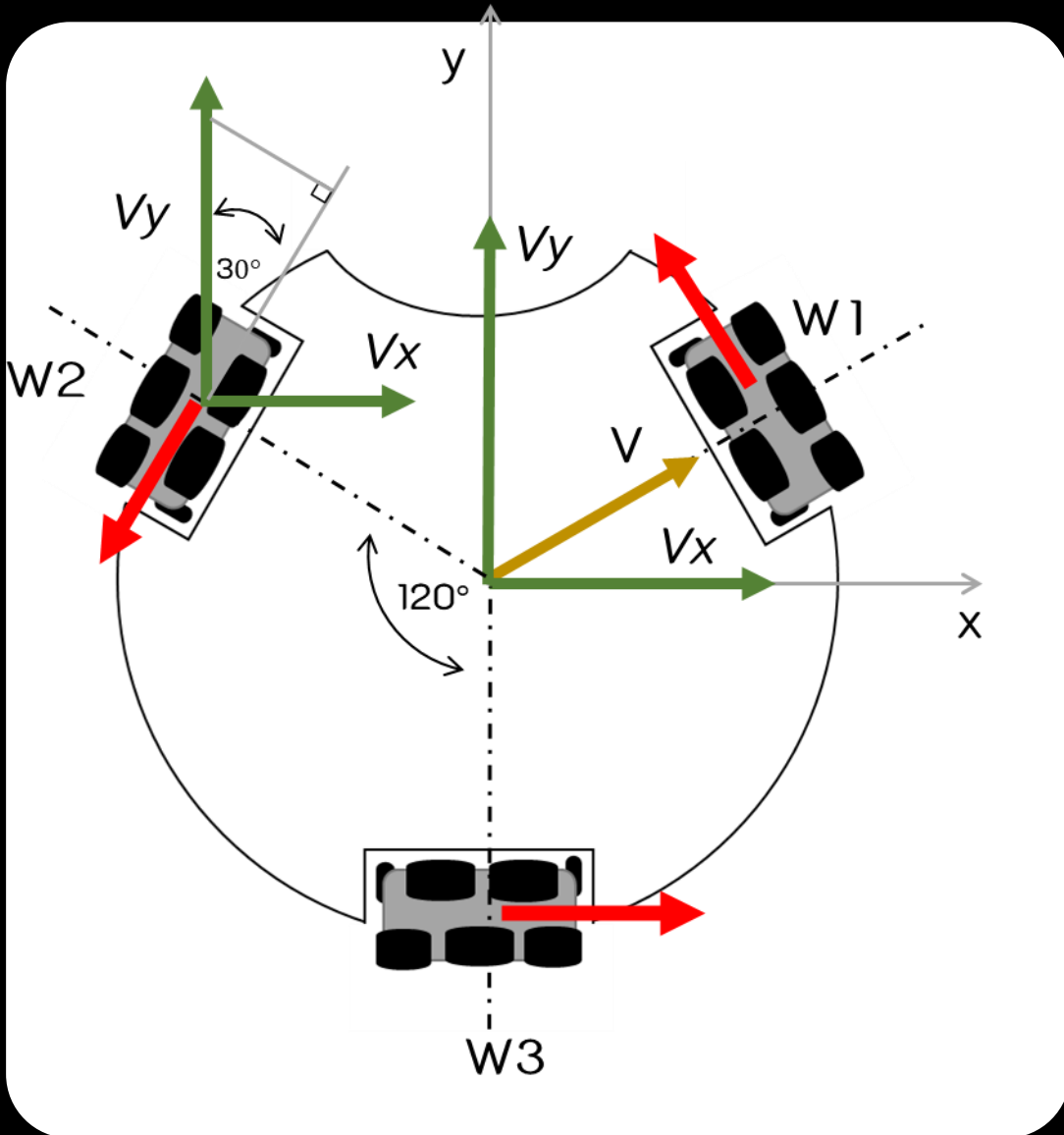
# สมการการเคลื่อนที่ ล้อที่ 2 (W2)



กำหนดทิศทางบวกของมอเตอร์โดยแทนลูกศรสีแดงสังเกตได้ว่า ชี้ไปทางแกน x ฝั่งลบและชี้ไปทางแกน y ฝั่งลบ โดยล้อทำมุมกับแนวแกน y 30 องศา จึงได้รูปแบบสมการดังนี้

$$V2 = -Vy * \cos(30) - Vx * \sin(30)$$

# สมการการเคลื่อนที่ ล้อที่ 3 (W3)



กำหนดทิศทางบวกของมอเตอร์หมุนโดยแทนลูกศรสีแดงสังเกตได้ว่าชี้ไปทางแกน x ฝั่งบวกและไม่ได้มีแรงใดๆกระทำในแนวแกน y โดยล้อทำมุมกับแนวแกน y 90 องศา จึงได้รูปแบบสมการดังนี้

$$V3 = Vy \cdot \cos(90) + Vx \cdot \sin(90)$$

$$V3 = Vx$$

# สรุปสมการเคลื่อนที่แบบเชิงเส้น

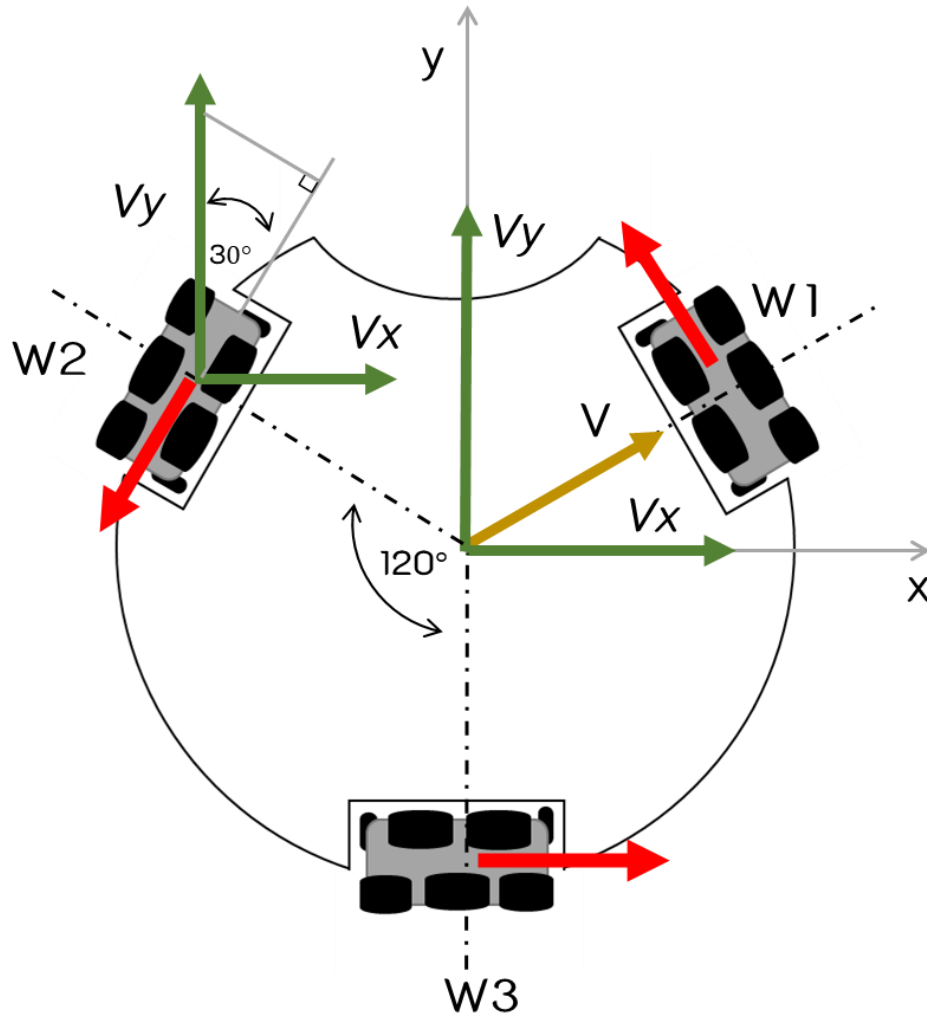


การกำหนดทิศทางหมุนของล้อ

จากรูป ลูกศรสีแดงกำหนดทิศทางบวกของมอเตอร์

ทิศทางบวก(+) หมุนตามเข็มนาฬิกา

ทิศทางลบ(-) หมุนทวนเข็มนาฬิกา



แสดงการหาทิศทางความเร็วของแต่ละล้อ

ล้อที่ 1 (W1) :

$$V1 = +Vy * \cos(30) - Vx * \sin(30)$$

ล้อที่ 2 (W2) :

$$V2 = -Vy * \cos(30) - Vx * \sin(30)$$

ล้อที่ 3 (W3) :

$$V3 = Vx$$

# ตัวอย่าง



ต้องการเคลื่อนที่ไปที่จุด  $V_x=50$  และ  $V_y=0$

วิธีทำ

ทิศทางล้อที่ 1

$$V_1 = -50 \cdot \sin(30) \Rightarrow -25$$

ทิศทางล้อที่ 2

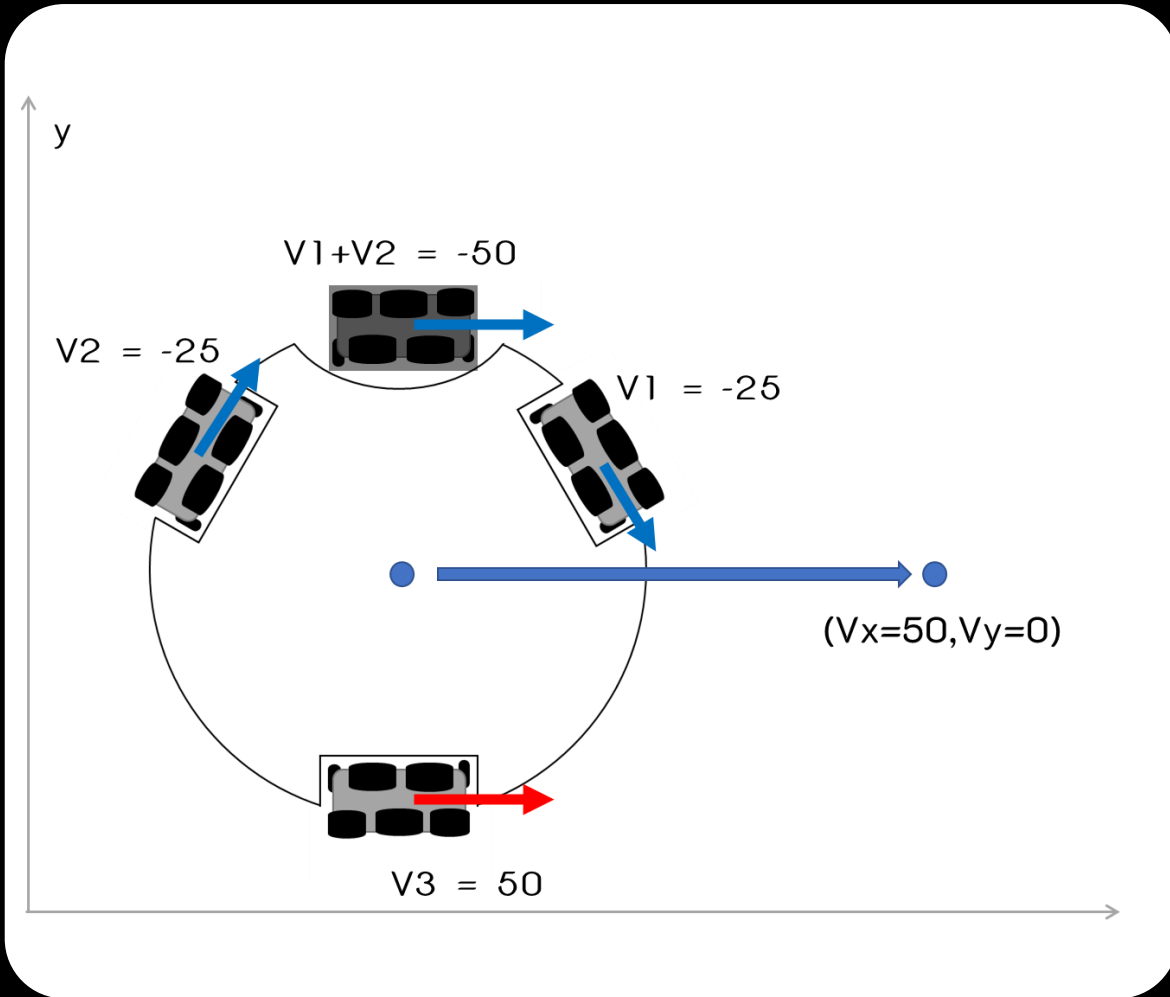
$$V_2 = -50 \cdot \sin(30) \Rightarrow -25$$

ทิศทางล้อที่ 3

$$V_3 = 50 \Rightarrow 50$$

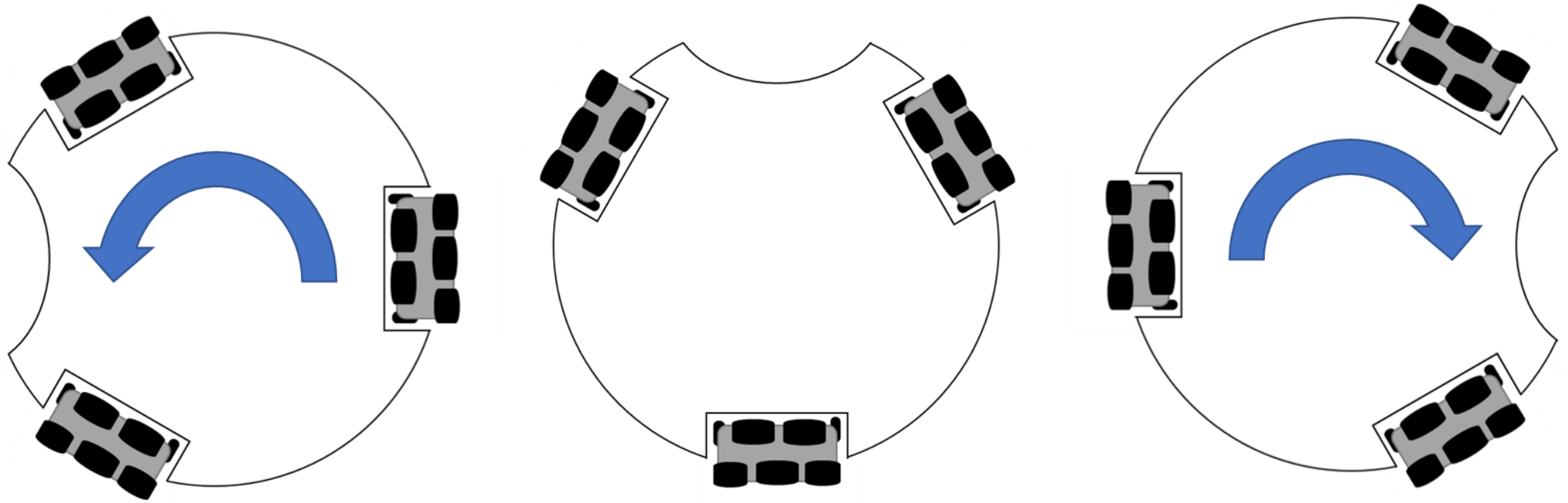


# อธิบายเพิ่มเติม

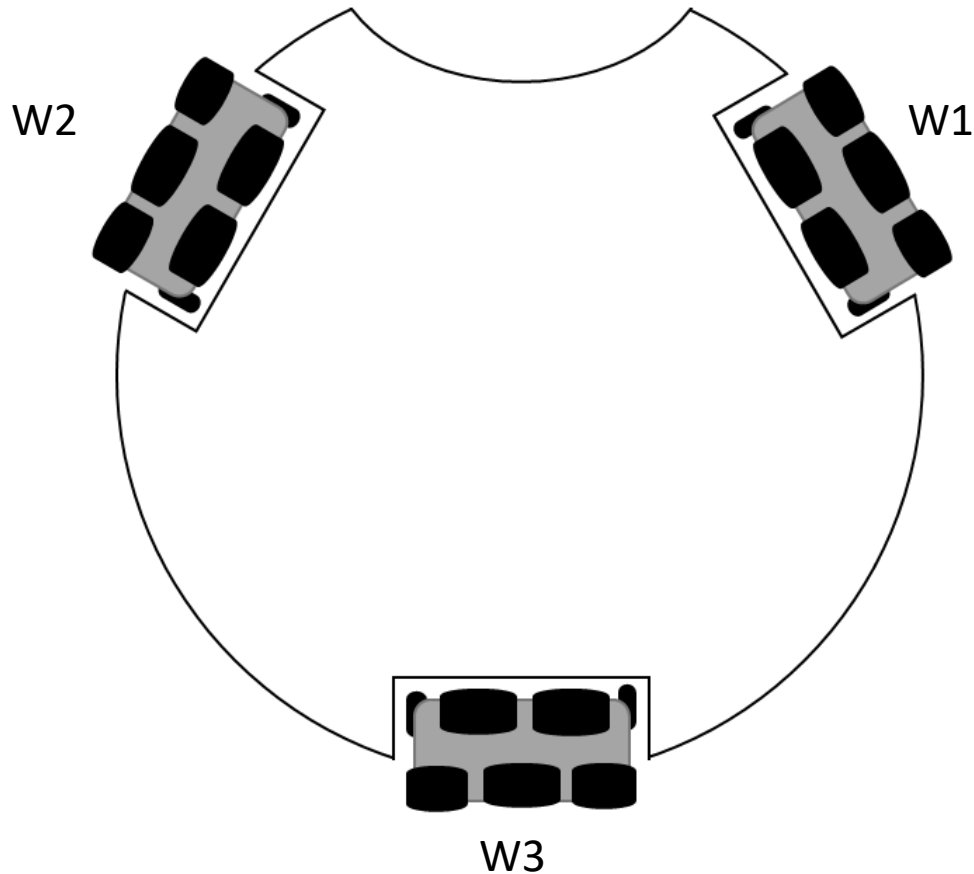


จากผลลัพธ์จะเห็นว่าเมื่อนำ  $V1$  และ  $V2$  มารวมกันจะมีค่าเท่ากับ  $-50$  โดยจะแสดงเป็นล้อเสมือนที่ถูกติดตั้งไว้อีกฝั่งของล้อที่ 3 ที่มีแรงกระทำค่าเท่ากับ  $50$  เมื่อมอเตอร์เริ่มทำงานหุ่นยนต์จะเคลื่อนที่ไปทางด้านขวาทันที

# สมการเคลื่อนที่แบบความเร็วเชิงมุม



# สมการความเร็วเชิงมุม



ความเร็วเชิงมุม ( $\omega$ : โอเมกา) จะนำมาใช้ในการหมุนตัวของหุ่นยนต์ โดยทั้ง 3 ล้อจะถูกกำหนดความเร็วและทิศทางการหมุนเท่าๆกัน

## สมการกำหนดความเร็วเชิงมุม

ล้อที่ 1

$$v_1 = \omega$$

ล้อที่ 2

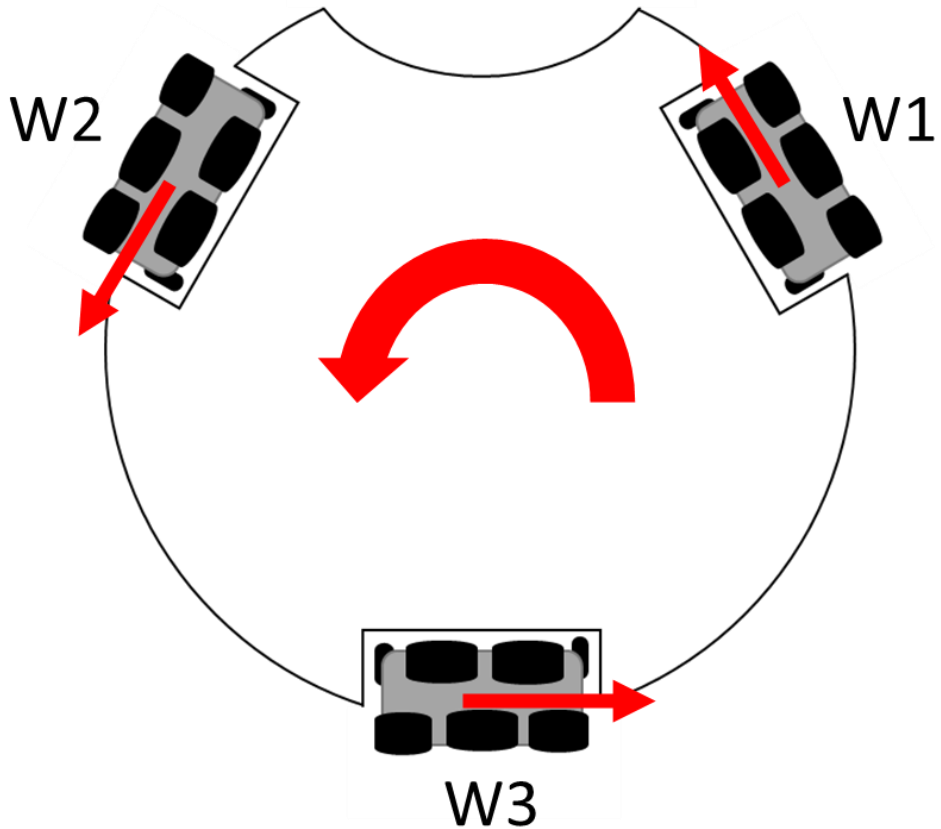
$$v_2 = \omega$$

ล้อที่ 3

$$v_3 = \omega$$

# ตัวอย่าง

ต้องการหมุนตัวหุ่นยนต์ด้วยความเร็ว 50 ในทิศทางทวนเข็มนาฬิกา



สมการกำหนดความเร็วเชิงมุม

ล้อที่ 1

$$V1 = 50$$

ล้อที่ 2

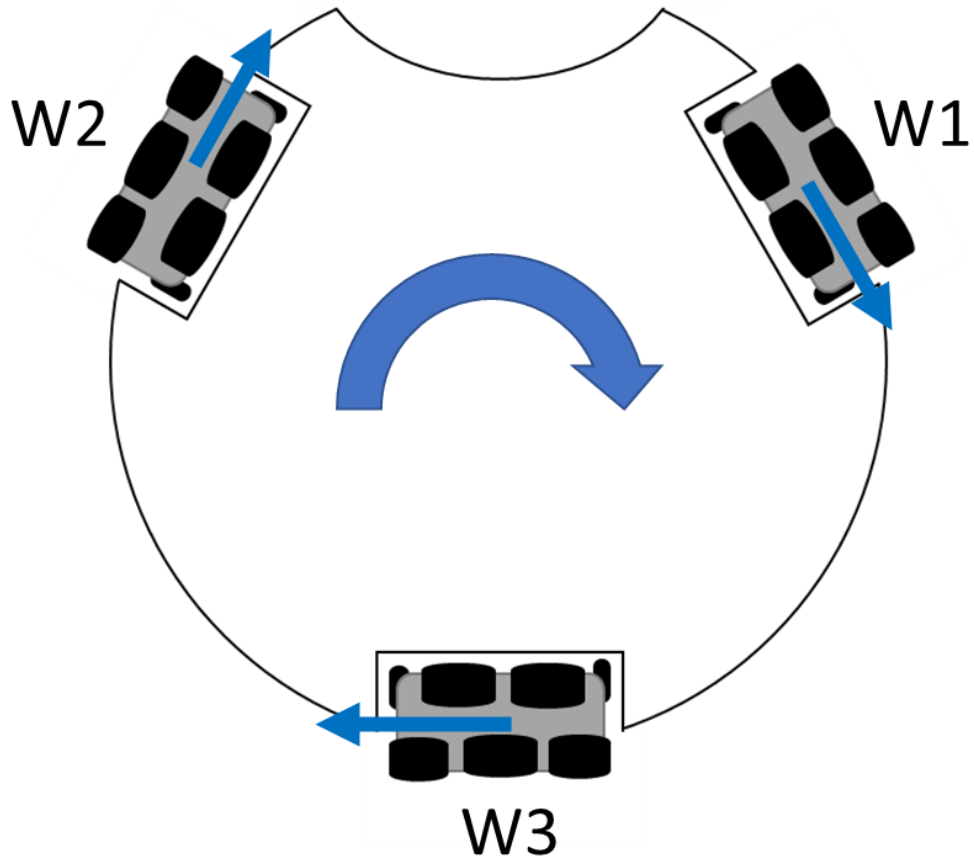
$$V2 = 50$$

ล้อที่ 3

$$V3 = 50$$

# ตัวอย่าง

ต้องการหมุนตัวหุ่นยนต์ด้วยความเร็ว 50 ในทิศทางตามเข็มนาฬิกา



สมการกำหนดความเร็วเชิงมุม

ล้อที่ 1

$$V1 = -50$$

ล้อที่ 2

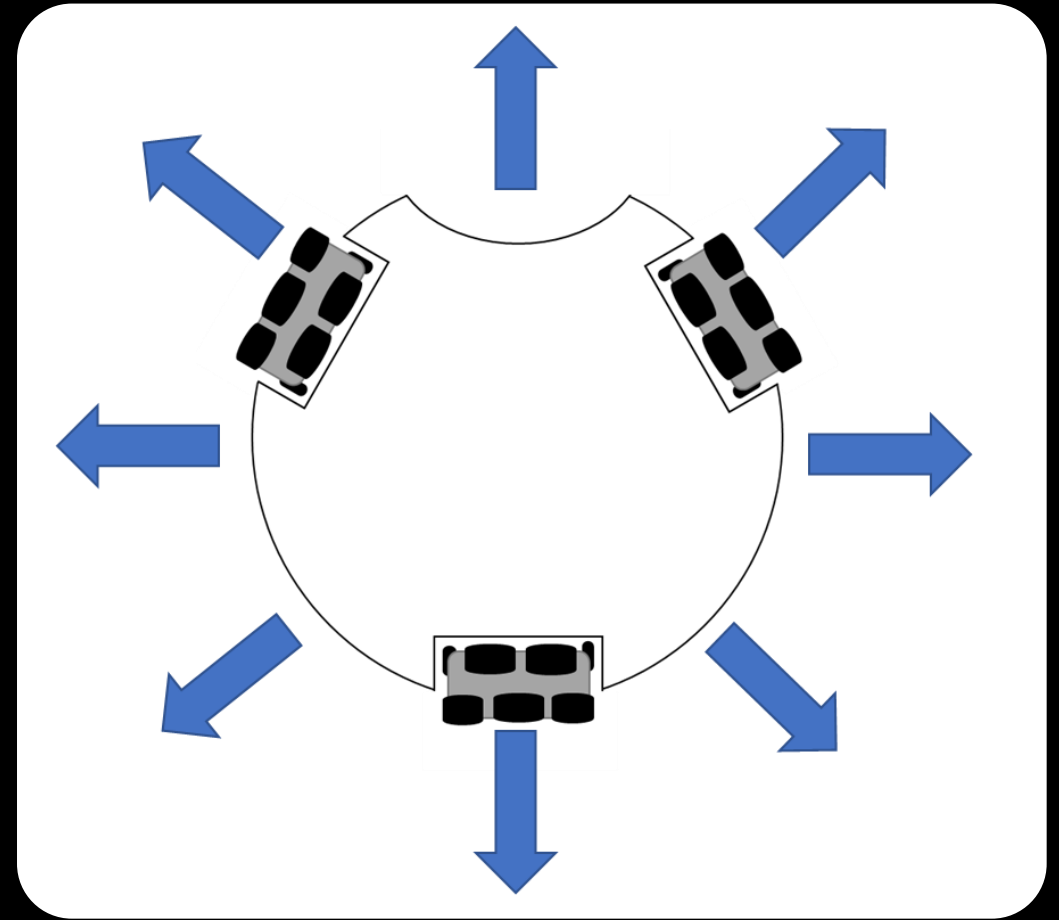
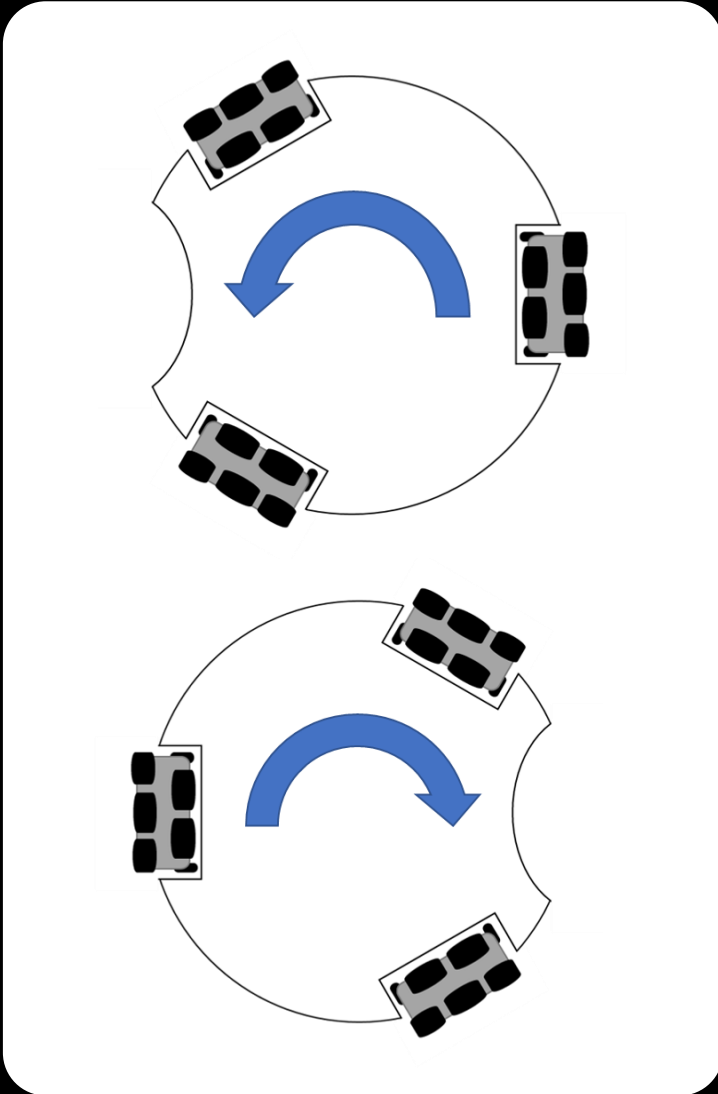
$$V2 = -50$$

ล้อที่ 3

$$V3 = -50$$

# สรุปการเคลื่อนที่ทั้ง 10 รูปแบบ

ตัวอย่างจะใช้ค่าแรงที่มาคำนวณเท่ากับ 50  
และหากการเคลื่อนที่ใดในแนวทแยงจะใช้ค่า  $V_x$   
และ  $V_y$  เท่ากัน โดยต่างเพียงแค่ทิศทาง



# ขับเคลื่อนหุ่นยนต์ข้างหน้า

ต้องการเคลื่อนที่ไปที่จุด  $V_x=0$  และ  $V_y=50$

## วิธีทำ

ทิศทางล้อที่ 1

$$V_1 = 50 * \cos(30) \Rightarrow 43.30$$

ทิศทางล้อที่ 2

$$V_2 = -50 * \cos(30) \Rightarrow -43.30$$

ทิศทางล้อที่ 3

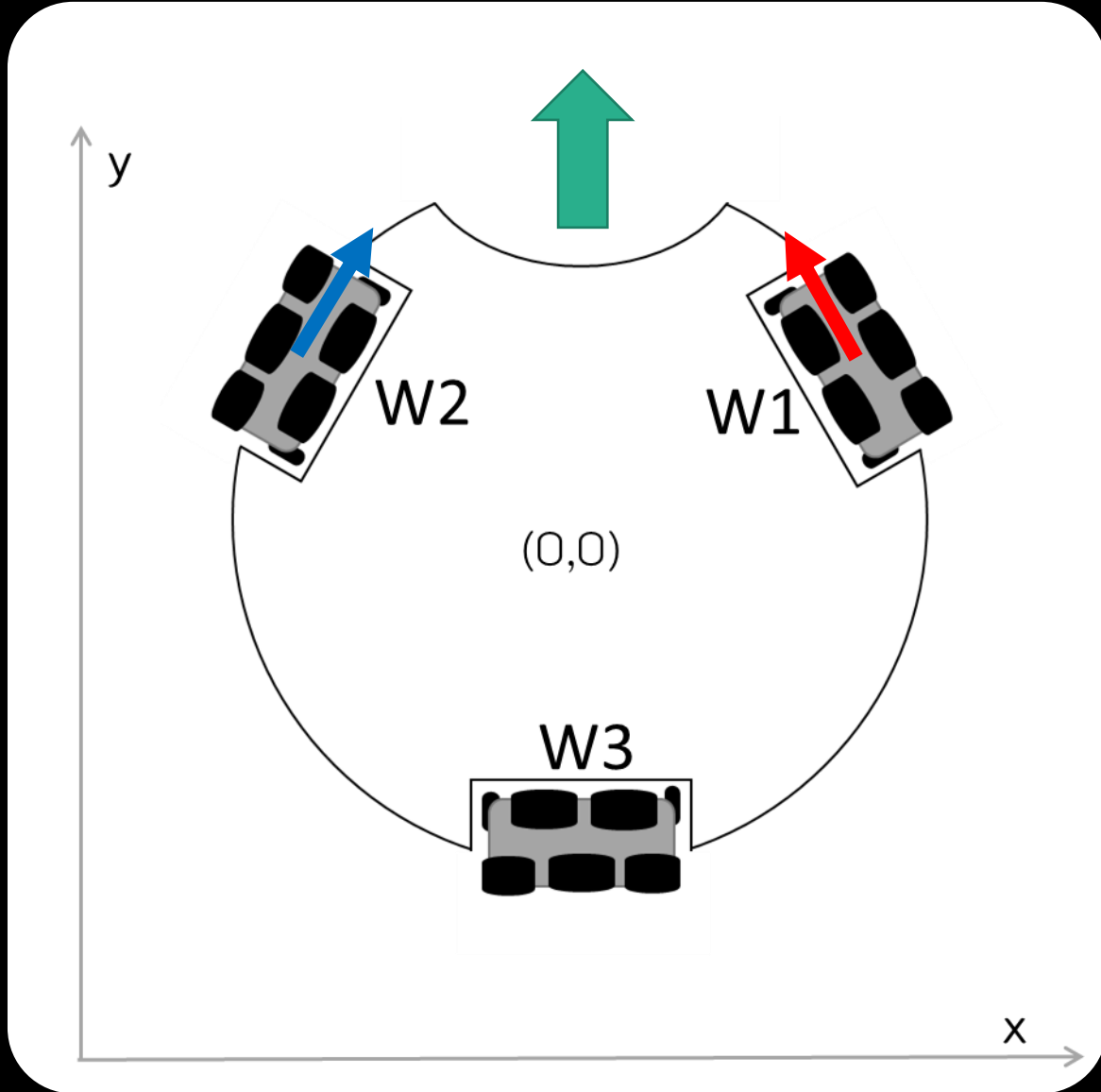
$$V_3 = 0$$

## ตัวอย่างโปรแกรม

```
motor(1, 43);
```

```
motor(2, -43);
```

```
motor(3, 0);
```



# สร้างโปรแกรม

```
#include <POP32.h>

void setup() {
}

void loop() {
    waitSW_A_bmp();
    wheel(43,-43,0);delay(3000);
    ao();
}

void wheel(int s1, int s2, int s3){
    motor(1, s1);
    motor(2, s2);
    motor(3, s3);
}
```

ขับเคลื่อนหุ่นยนต์ข้างหน้า



# ขับเคลื่อนหุ่นยนต์ถอยหลัง

ต้องการเคลื่อนที่ไปที่จุด  $V_x=0$  และ  $V_y=-50$

## วิธีทำ

### ทิศทางล้อที่ 1

$$V_1 = -50 * \cos(30) \Rightarrow -43.30$$

### ทิศทางล้อที่ 2

$$V_2 = 50 * \cos(30) \Rightarrow 43.30$$

### ทิศทางล้อที่ 3

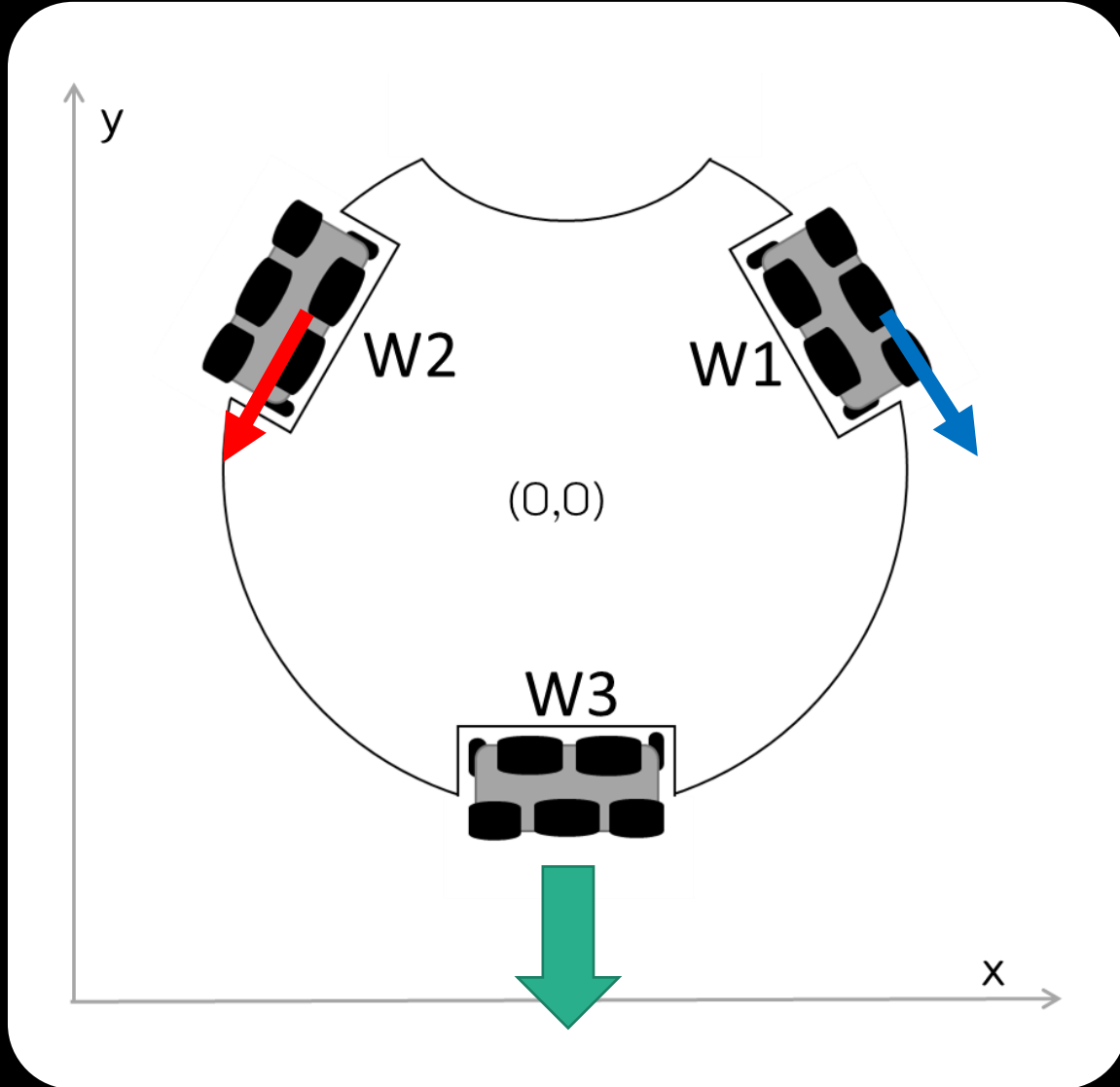
$$V_3 = 0$$

## ตัวอย่างโปรแกรม

```
motor(1, -43.30);
```

```
motor(2, 43.30);
```

```
motor(3, 0);
```



# สร้างโปรแกรม

```
#include <POP32.h>

void setup() {
}

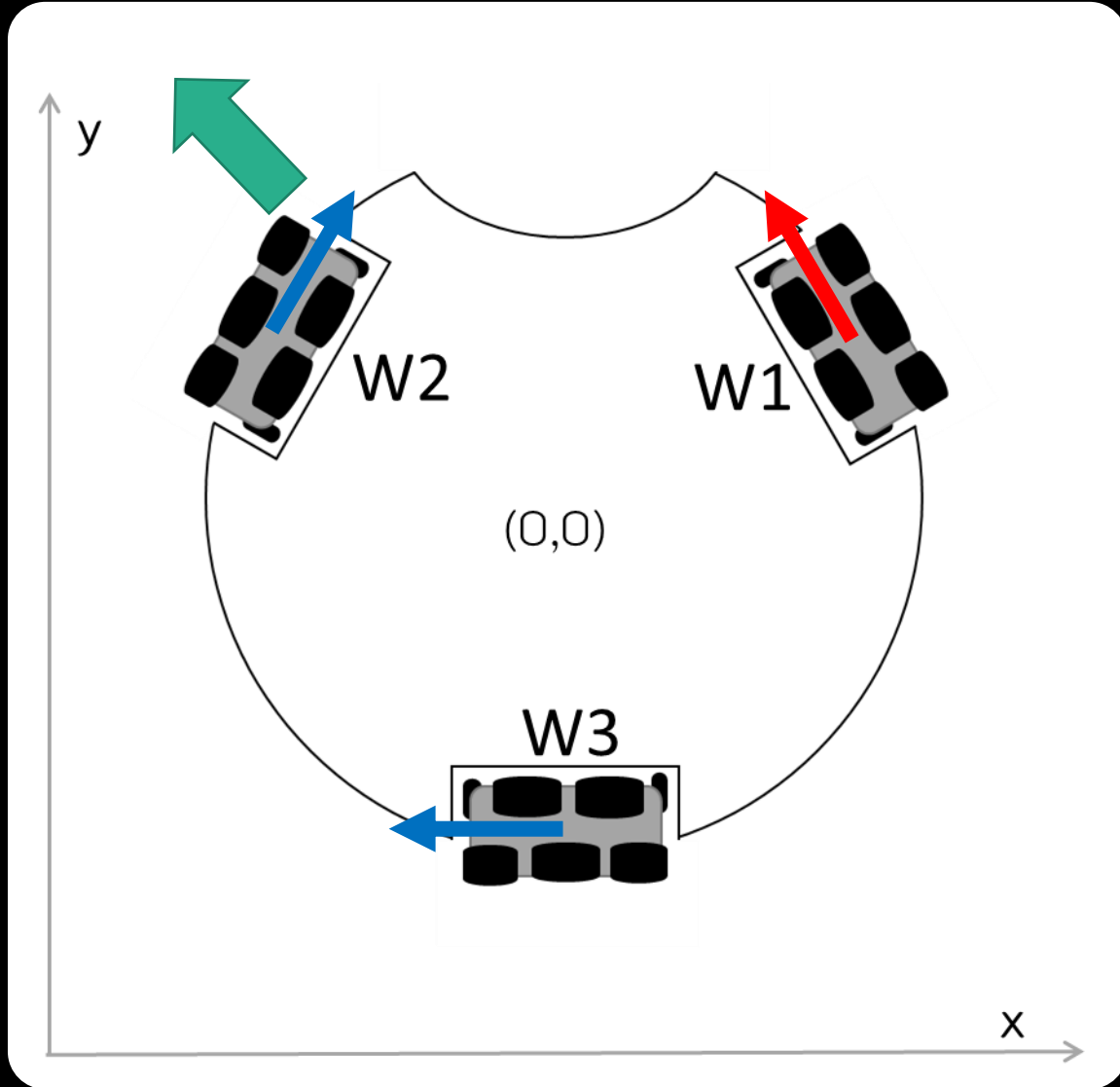
void loop() {
    waitSW_A_bmp();
    wheel(-43,43,0);delay(3000);
    ao();
}

void wheel(int s1, int s2, int s3){
    motor(1, s1);
    motor(2, s2);
    motor(3, s3);
}
```

ขับเคลื่อนหุ่นยนต์ถอยหลัง

# ขับเคลื่อนหุ่นยนต์

ทะแยงซ้ายไปข้างหน้า



ต้องการเคลื่อนที่ไปที่จุด  $V_x = -50$  และ  $V_y = 50$

## วิธีทำ

ทิศทางล้อที่ 1

$$V_1 = 50 * \cos(30) - (-50) \sin(30) \Rightarrow 68.30$$

ทิศทางล้อที่ 2

$$V_2 = -(50) * \cos(30) - (-50) \sin(30) \Rightarrow -18.30$$

ทิศทางล้อที่ 3

$$V_3 = (-50)$$

## ตัวอย่างโปรแกรม

```
motor(1, 68.30);  
motor(2, -18.30);  
motor(3, -50);
```

# สร้างโปรแกรม

```
#include <POP32.h>

void setup() {
}

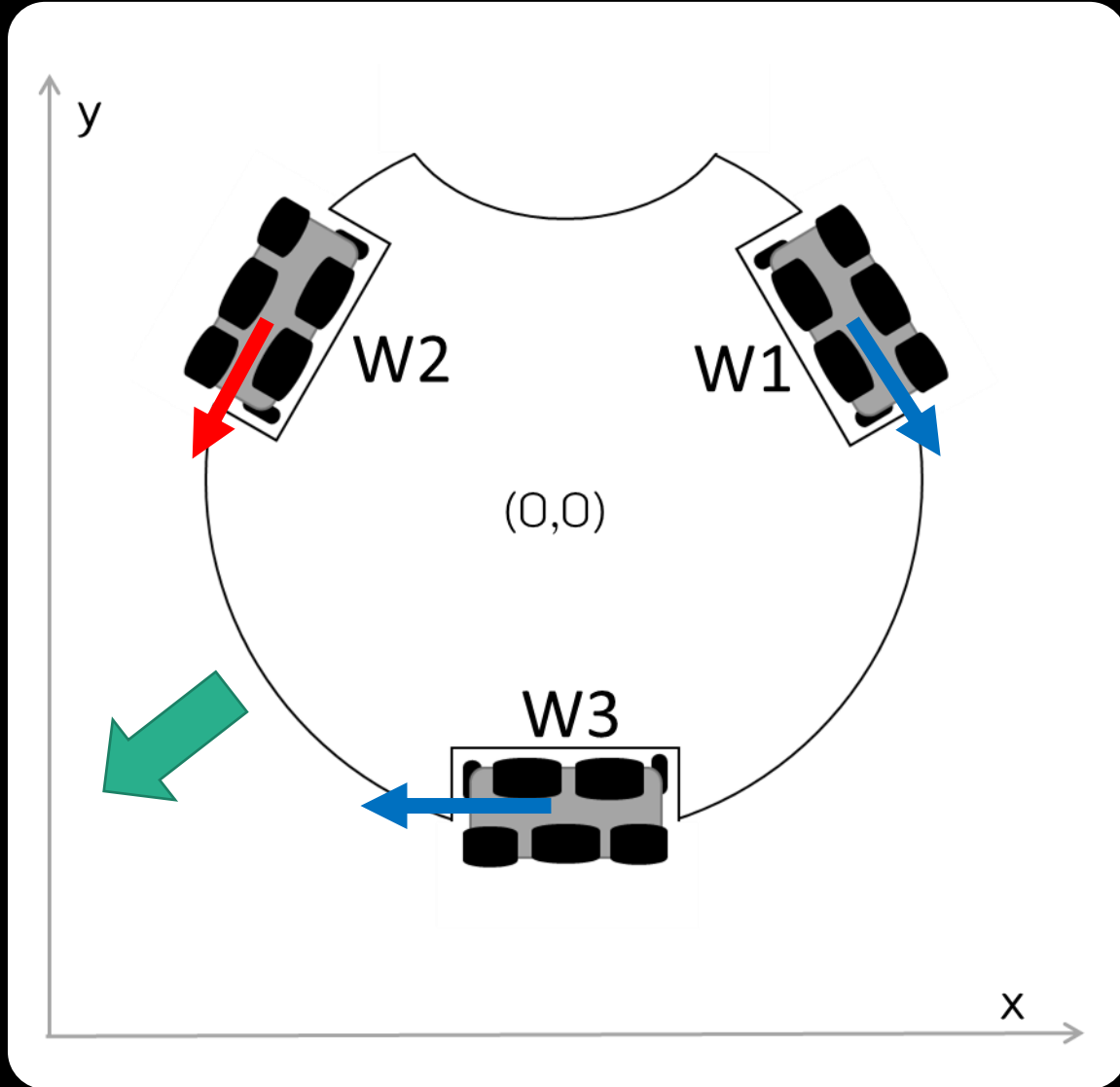
void loop() {
    waitSW_A_bmp();
    wheel(68,-18,-50); delay(3000);
    ao();
}

void wheel(int s1, int s2, int s3){
    motor(1, s1);
    motor(2, s2);
    motor(3, s3);
}
```

ขับเคลื่อนหุ่นยนต์  
ทะแยงซ้ายไปข้างหน้า

# ขับเคลื่อนหุ่นยนต์

## ถอยหลังทะแยงซ้าย



ต้องการเคลื่อนที่ไปที่จุด  $V_x = -50$  และ  $V_y = -50$

### วิธีทำ

ทิศทางล้อที่ 1

$$V1 = (-50) * \cos(30) - (-50) * \sin(30) \Rightarrow -18.30$$

ทิศทางล้อที่ 2

$$V2 = -(-50) * \cos(30) - (-50) * \sin(30) \Rightarrow 68.30$$

ทิศทางล้อที่ 3

$$V3 = (-50)$$

### ตัวอย่างโปรแกรม

```
motor(1, -18.30);
```

```
motor(2, 68.30);
```

```
motor(3, -50);
```

# สร้างโปรแกรม

```
#include <POP32.h>

void setup() {
}

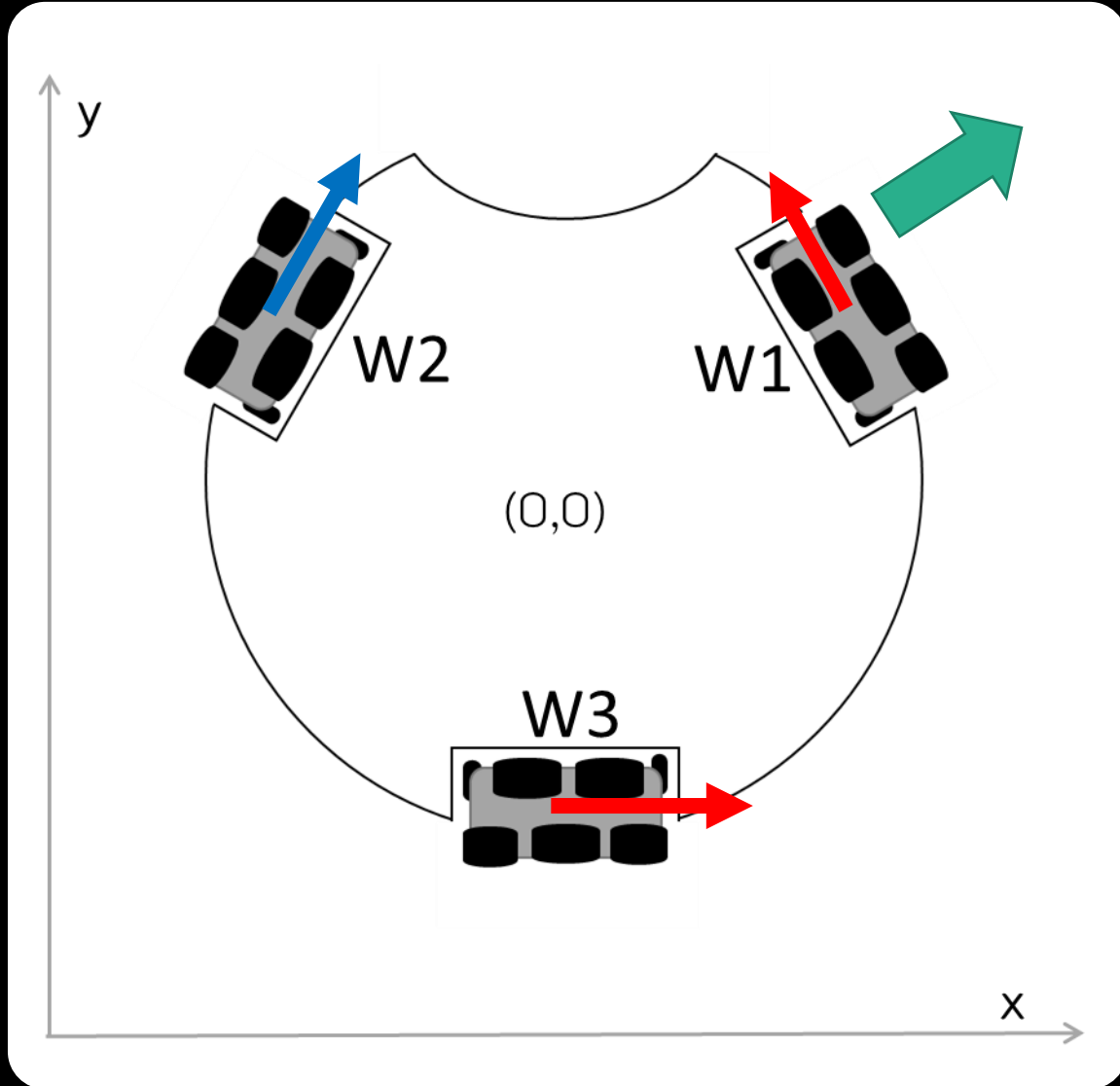
void loop() {
    waitSW_A_bmp();
    wheel(-18,68,-50); delay(3000);
    ao();
}

void wheel(int s1, int s2, int s3){
    motor(1, s1);
    motor(2, s2);
    motor(3, s3);
}
```

ขับเคลื่อนหุ่นยนต์  
ถอยหลังทะแยงซ้าย

# ขับเคลื่อนหุ่นยนต์

ทะแยงขวาไปข้างหน้า



ต้องการเคลื่อนที่ไปที่จุด  $V_x=50$  และ  $V_y=50$

วิธีทำ

ทิศทางล้อที่ 1

$$V1 = (50) * \cos(30) - (50) * \sin(30) \Rightarrow 18.30$$

ทิศทางล้อที่ 2

$$V2 = -(50) * \cos(30) - (50) * \sin(30) \Rightarrow -68.30$$

ทิศทางล้อที่ 3

$$V3 = 50$$

ตัวอย่างโปรแกรม

```
motor(1, 18.30);  
motor(2, -68.30);  
motor(3, 50);
```

# สร้างโปรแกรม

```
#include <POP32.h>

void setup() {
}

void loop() {
    waitSW_A_bmp();
    wheel(18,-68,50); delay(3000);
    ao();
}

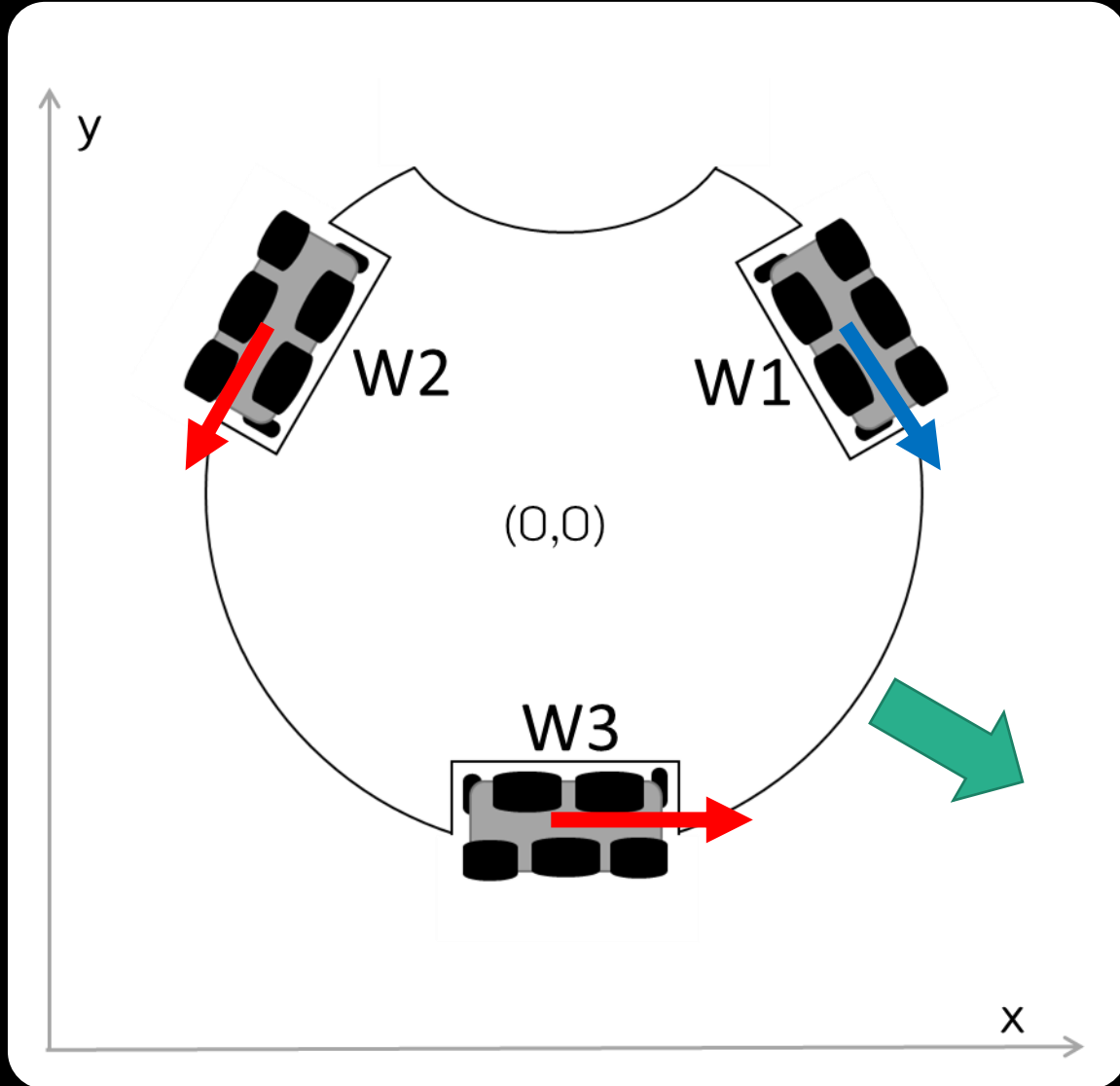
void wheel(int s1, int s2, int s3){
    motor(1, s1);
    motor(2, s2);
    motor(3, s3);
}
```

ขับเคลื่อนหุ่นยนต์  
ทะแยงขวาไปข้างหน้า



# ขับเคลื่อนหุ่นยนต์

## ถอยหลังทะแยงขวา



ต้องการเคลื่อนที่ไปที่จุด  $V_x=50$  และ  $V_y=-50$

### วิธีทำ

#### ทิศทางล้อที่ 1

$$V1 = (-50) * \cos(30) - (50) * \sin(30) \Rightarrow -68.30$$

#### ทิศทางล้อที่ 2

$$V2 = -(-50) * \cos(30) - (50) * \sin(30) \Rightarrow 18.30$$

#### ทิศทางล้อที่ 3

$$V3 = 50$$

### ตัวอย่างโปรแกรม

```
motor(1, -68.30);
```

```
motor(2, 18.30);
```

```
motor(3, 50);
```

# สร้างโปรแกรม

```
#include <POP32.h>

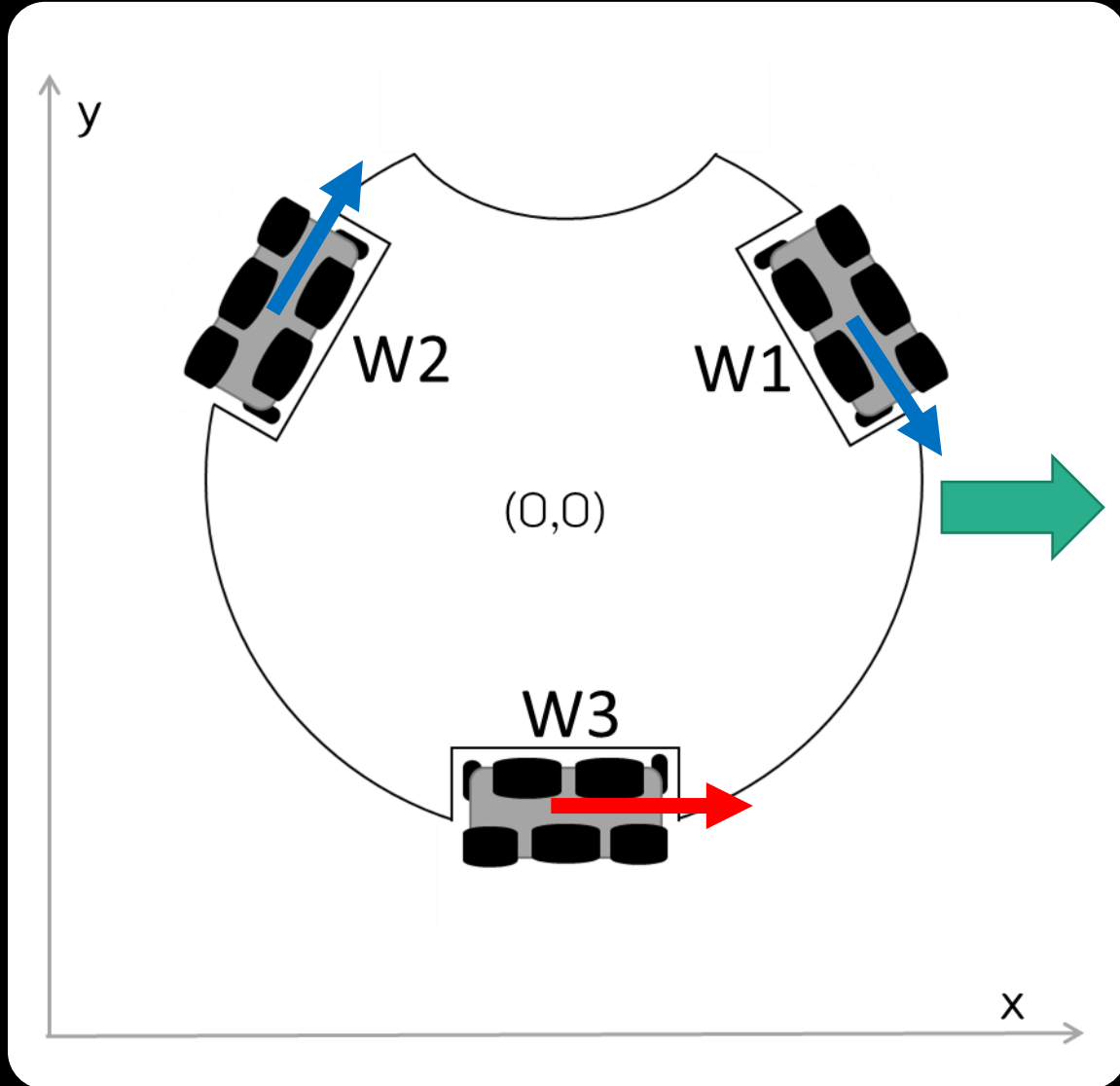
void setup() {
}

void loop() {
    waitSW_A_bmp();
    wheel(-68,18,50); delay(3000);
    ao();
}

void wheel(int s1, int s2, int s3){
    motor(1, s1);
    motor(2, s2);
    motor(3, s3);
}
```

ขับเคลื่อนหุ่นยนต์  
ถอยหลังทะแยงขวา

# ขับเคลื่อนหุ่นยนต์ ไปทางด้านขวา



ต้องการเคลื่อนที่ไปที่จุด  $V_x=50$  และ  $V_y=0$

## วิธีทำ

ทิศทางล้อที่ 1

$$V1 = -(50) * \sin(30) \Rightarrow -25$$

ทิศทางล้อที่ 2

$$V2 = -(50) * \sin(30) \Rightarrow -25$$

ทิศทางล้อที่ 3

$$V3 = 50$$

ตัวอย่างโปรแกรม

```
motor(1, -25);
```

```
motor(2, -25);
```

```
motor(3, 50);
```

# สร้างโปรแกรม

```
#include <POP32.h>

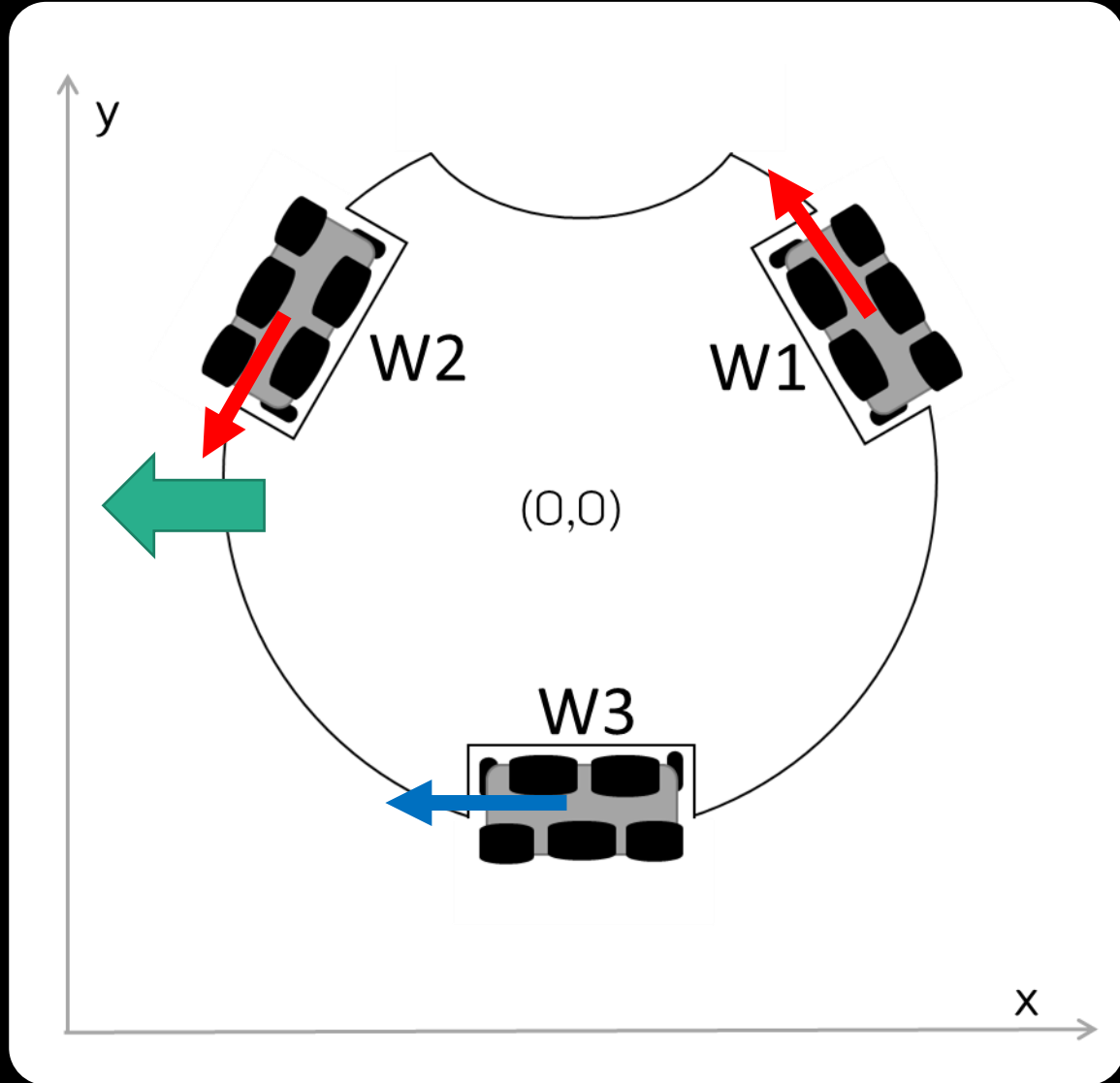
void setup() {
}

void loop() {
    waitSW_A_bmp();
    wheel(-25,-25,50); delay(3000);
    ao();
}

void wheel(int s1, int s2, int s3){
    motor(1, s1);
    motor(2, s2);
    motor(3, s3);
}
```

ขับเคลื่อนหุ่นยนต์  
ไปทางด้านขวา

# ขับเคลื่อนหุ่นยนต์ ไปทางด้านซ้าย



ต้องการเคลื่อนที่ไปที่จุด  $V_x = -50$  และ  $V_y = 0$

## วิธีทำ

ทิศทางล้อที่ 1

$$V_1 = -(-50)\sin(30) \Rightarrow 25$$

ทิศทางล้อที่ 2

$$V_2 = -(-50)\sin(30) \Rightarrow 25$$

ทิศทางล้อที่ 3

$$V_3 = -50$$

## ตัวอย่างโปรแกรม

```
motor(1, 25);  
motor(2, 25);  
motor(3, -50);
```

# สร้างโปรแกรม

```
#include <POP32.h>

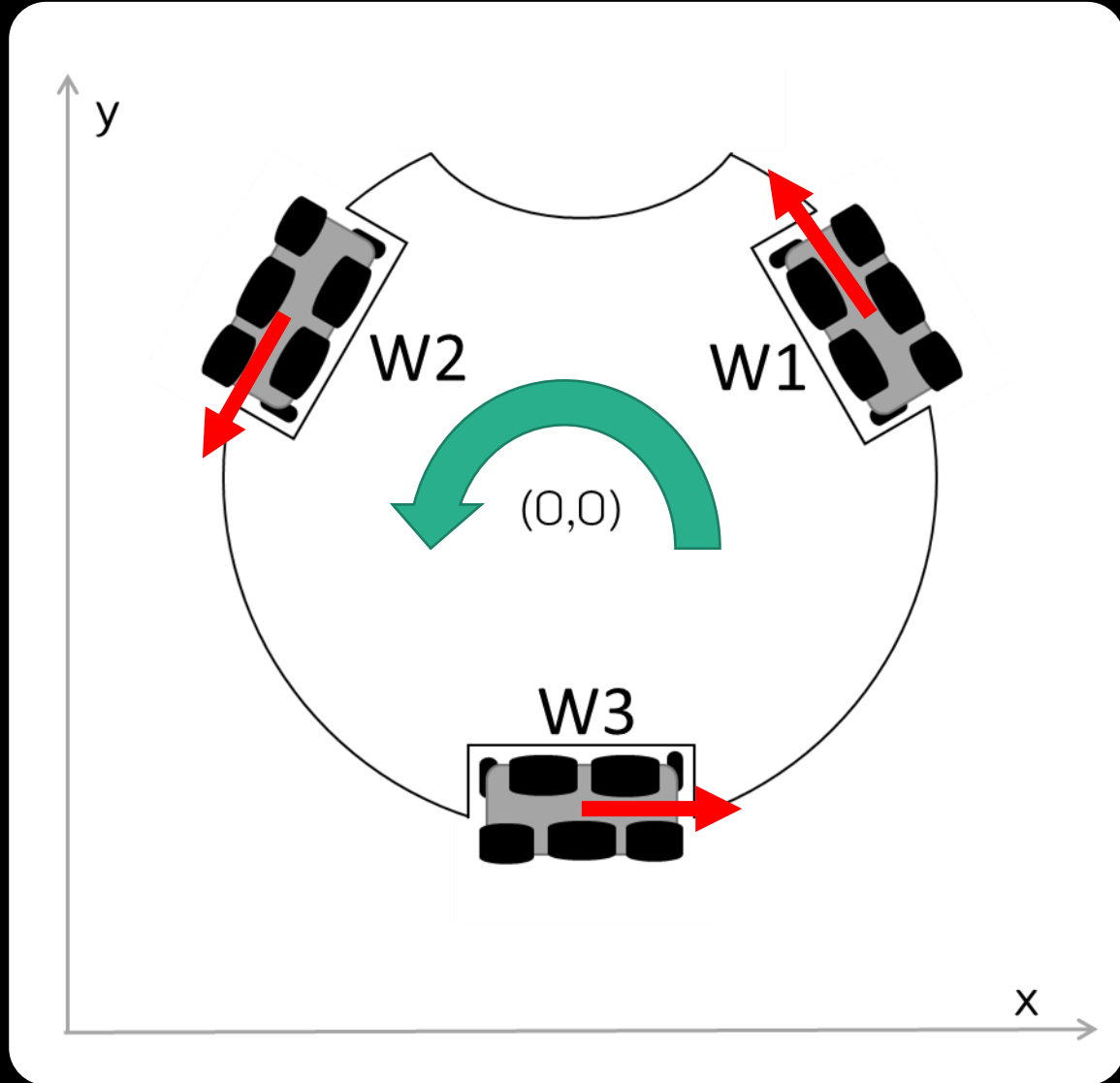
void setup() {
}

void loop() {
    waitSW_A_bmp();
    wheel(25,25,-50); delay(3000);
    ao();
}

void wheel(int s1, int s2, int s3){
    motor(1, s1);
    motor(2, s2);
    motor(3, s3);
}
```

ขับเคลื่อนหุ่นยนต์  
ไปทางด้านซ้าย

# ขับเคลื่อนหุ่นยนต์ หมุนซ้าย



ต้องการเคลื่อนที่หมุนซ้ายด้วยความเร็วเท่ากับ 50

## วิธีทำ

ทิศทางล้อที่ 1

$$V1 = 50$$

ทิศทางล้อที่ 2

$$V2 = 50$$

ทิศทางล้อที่ 3

$$V3 = 50$$

## ตัวอย่างโปรแกรม

```
motor(1, 50);
```

```
motor(2, 50);
```

```
motor(3, 50);
```

# สร้างโปรแกรม

```
#include <POP32.h>

void setup() {
}

void loop() {
    waitSW_A_bmp();
    wheel(50,50,50); delay(3000);
    ao();
}

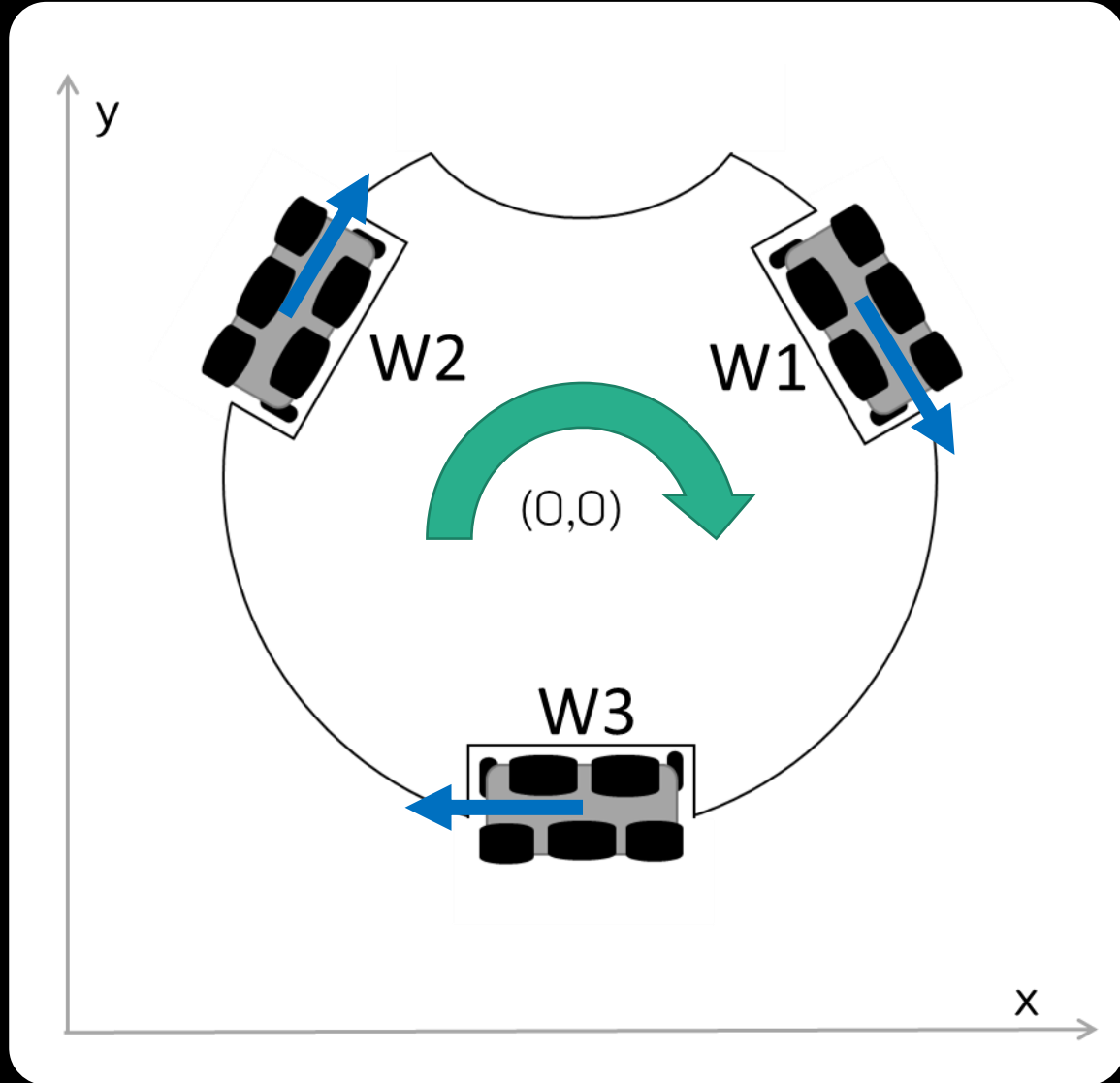
void wheel(int s1, int s2, int s3){
    motor(1, s1);
    motor(2, s2);
    motor(3, s3);
}
```

ขับเคลื่อนหุ่นยนต์  
หมุนซ้าย



# ขับเคลื่อนหุ่นยนต์

## หมุนขวา



ต้องการเคลื่อนที่หมุนขวาด้วยความเร็วเท่ากับ 50

### วิธีทำ

ทิศทางล้อที่ 1

$$V1 = -50$$

ทิศทางล้อที่ 2

$$V2 = -50$$

ทิศทางล้อที่ 3

$$V3 = -50$$

### ตัวอย่างโปรแกรม

```
motor(1, -50);
```

```
motor(2, -50);
```

```
motor(3, -50);
```

# สร้างโปรแกรม

```
#include <POP32.h>

void setup() {
}

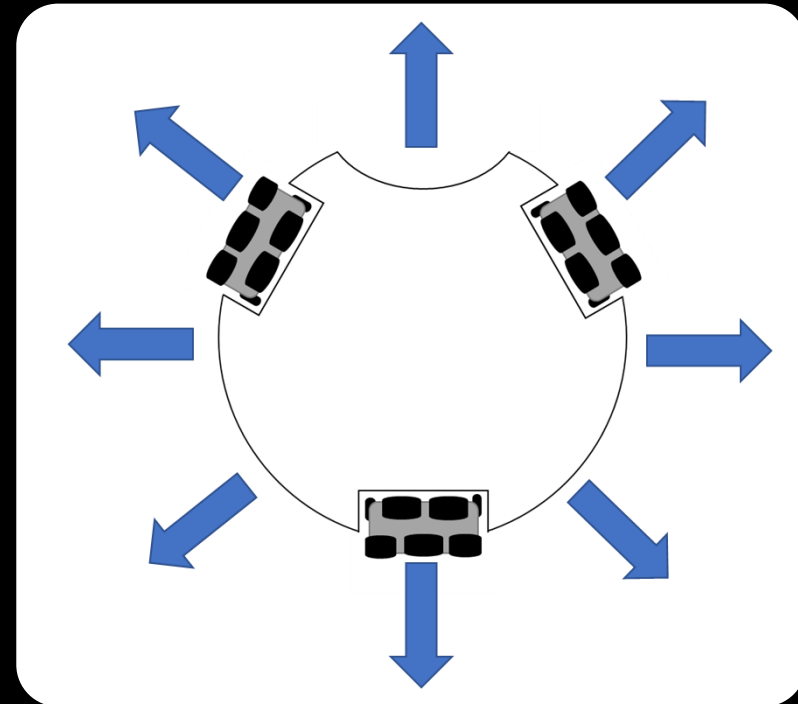
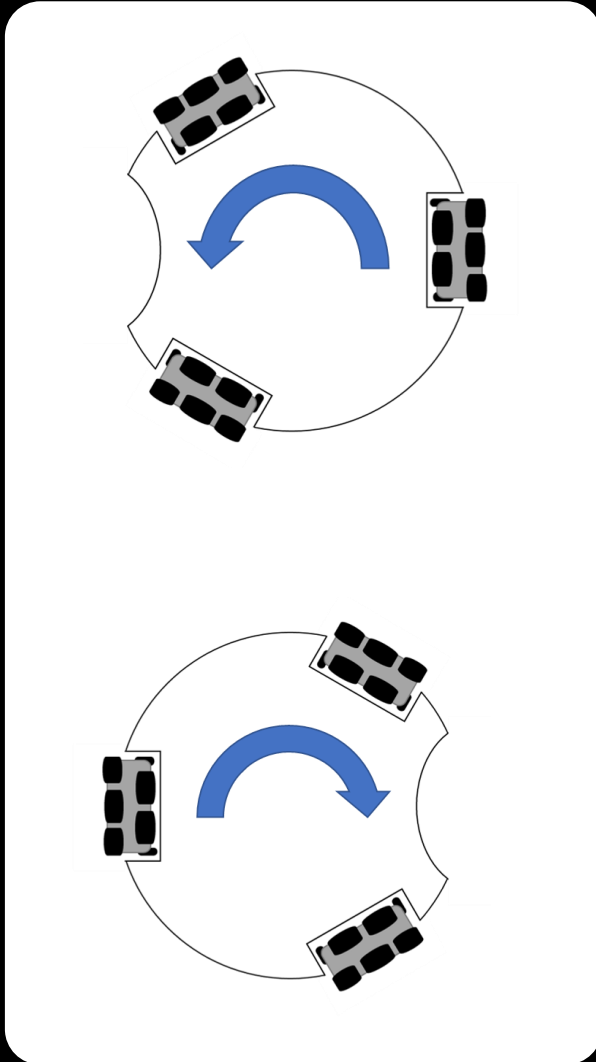
void loop() {
    waitSW_A_bmp();
    wheel(-50,-50,-50); delay(3000);
    ao();
}

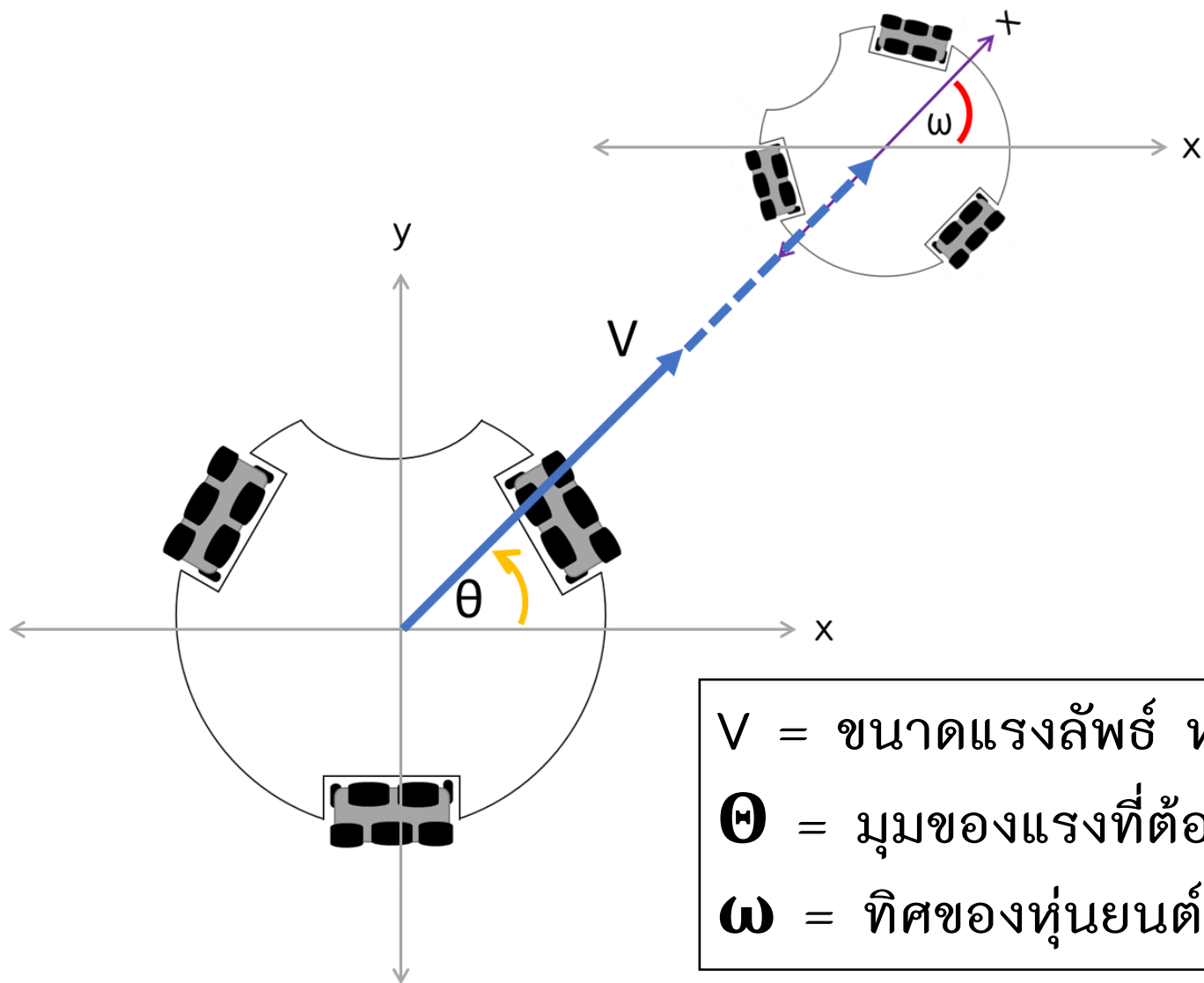
void wheel(int s1, int s2, int s3){
    motor(1, s1);
    motor(2, s2);
    motor(3, s3);
}
```

ขับเคลื่อนหุ่นยนต์  
หมุนขวา

# การเคลื่อนที่แบบโฮโลโนมิก (Holonomic)

การเคลื่อนไหวแบบโฮโลโนมิกจะเกิดขึ้นได้หากระดับความอิสระที่ควบคุมได้เท่ากับทั้งหมด ยกตัวอย่างเช่น ล้อเลื่อนบนรถเข็นล้อปั้ง สามารถควบคุมการเคลื่อนไหวได้ในทุกองศาของความอิสระเท่าที่จะเป็นไปได้ ตรงกันข้ามกับล้อของจักรยาน





## ลักษณะการเคลื่อนที่

$V$  = ขนาดแรงลัพธ์ หรือ ความเร็ว

$\theta$  = มุมของแรงที่ต้องการเคลื่อนที่ไป

$\omega$  = ทิศของหุ่นยนต์ปลายทางที่ต้องการเมื่อเทียบกับต้นทาง

# คำสั่ง

1 degrees = 0.0174532925 radians

**cos**(radians)

**sin**(radians)

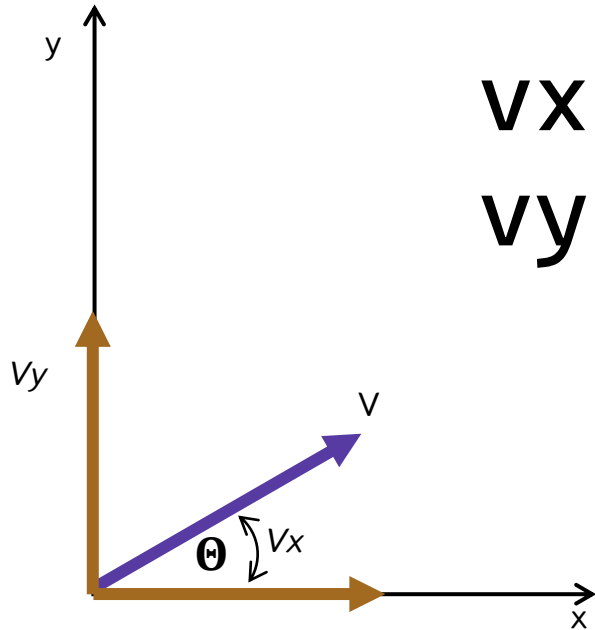
# ตัวอย่าง

```
double x = cos(1.566)
```

```
double x = cos(90 * 0.0174)
```

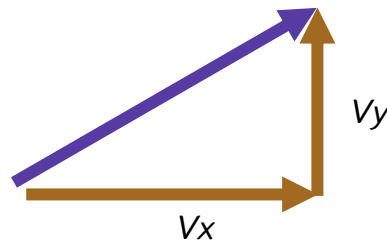
x จะมีค่าเท่ากับ 1

# การแตกแรง



$$V_x = V * \cos(\theta)$$

$$V_y = V * \sin(\theta)$$



# การสร้างโปรแกรม

การแตกแรง

```
thetaRad = theta * degToRad;  
vx = spd * cos(thetaRad);  
vy = spd * sin(thetaRad);
```

คำนวณแรงในแต่ละล้อ

```
spd1 = vy * cos30 - vx * sin30 + omega;  
spd2 = - vy * cos30 - vx * sin30 + omega;  
spd3 = vx + omega;
```

กำหนดแรงในแต่ละล้อ

```
wheel(spd1, spd2, spd3);
```

# สร้างโปรแกรมการแบบโฮโลโนมิก

```
#define degToRad 0.0174f
#define sin30 sin(30.f * degToRad)
#define cos30 cos(30.f * degToRad)
float thetaRad, vx, vy, spd1, spd2, spd3;
void holonomic(float spd, float theta, float omega){
    thetaRad = theta * degToRad;
    vx = spd * cos(thetaRad);
    vy = spd * sin(thetaRad);
    spd1 =  vy * cos30 - vx * sin30 + omega;
    spd2 = - vy * cos30 - vx * sin30 + omega;
    spd3 =  vx + omega;
    wheel(spd1, spd2, spd3);
}
```

เรียกใช้งานฟังก์ชัน  
holonomic(20,0,0)



# สร้างโปรแกรม

```
#define degToRad 0.0174f
#define sin30 sin(30.f * degToRad)
#define cos30 cos(30.f * degToRad)
float thetaRad, vx, vy, spd1, spd2, spd3;
void wheel(int s1, int s2, int s3){
    motor(1, s1);
    motor(2, s2);
    motor(3, s3);
}
void holonomic(float spd, float theta, float omega) {
    thetaRad = theta * degToRad;
    vx = spd * cos(thetaRad);
    vy = spd * sin(thetaRad);
    spd1 = vy * cos30 - vx * sin30 + omega;
    spd2 = -vy * cos30 - vx * sin30 + omega;
    spd3 = vx + omega;
    wheel(spd1, spd2, spd3);
}
```

```
#include <POP32.h>
void setup() {
}
void loop() {
    waitSW_A_bmp();
    holonomic(30, 0, 0); delay(3000);
    ao();
}
```

ขับเคลื่อนหุ่นยนต์  
หมุนขวา



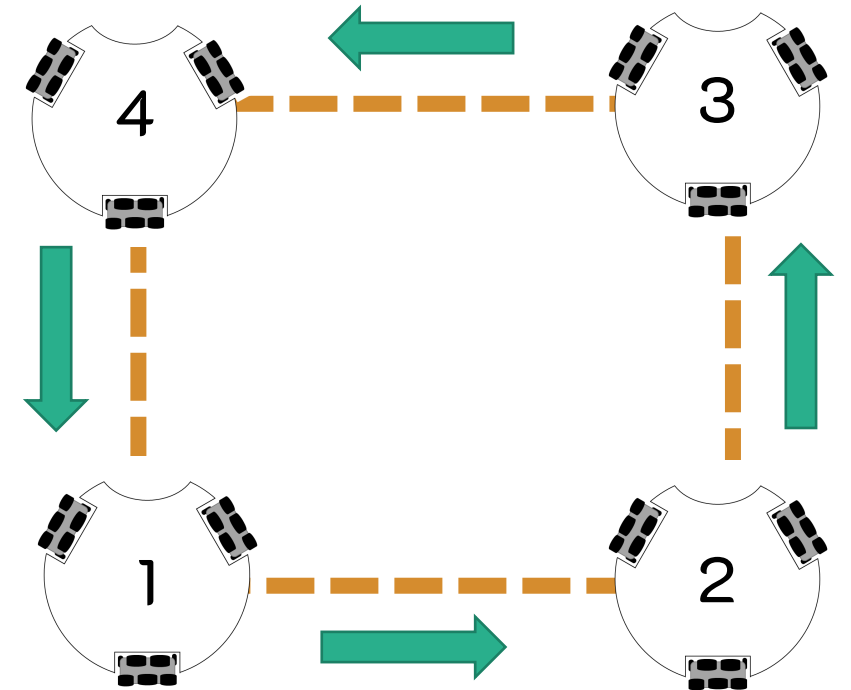
# สร้างโปรแกรม

```
#include <POP32.h>

void setup() {
}

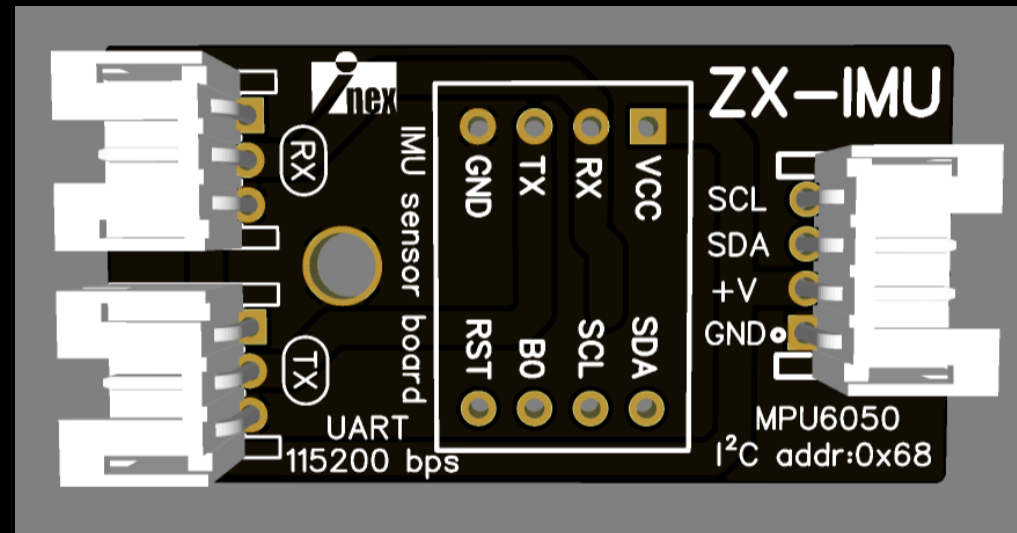
void loop() {
    waitSW_A_bmp();
    holonomic(30, 0, 0); delay(3000);
    holonomic(30, 90, 0); delay(3000);
    holonomic(30, 180, 0); delay(3000);
    holonomic(30, 270, 0); delay(3000);
    ao();
}
```

ขับเคลื่อนหุ่นยนต์เป็นรูปสี่เหลี่ยม

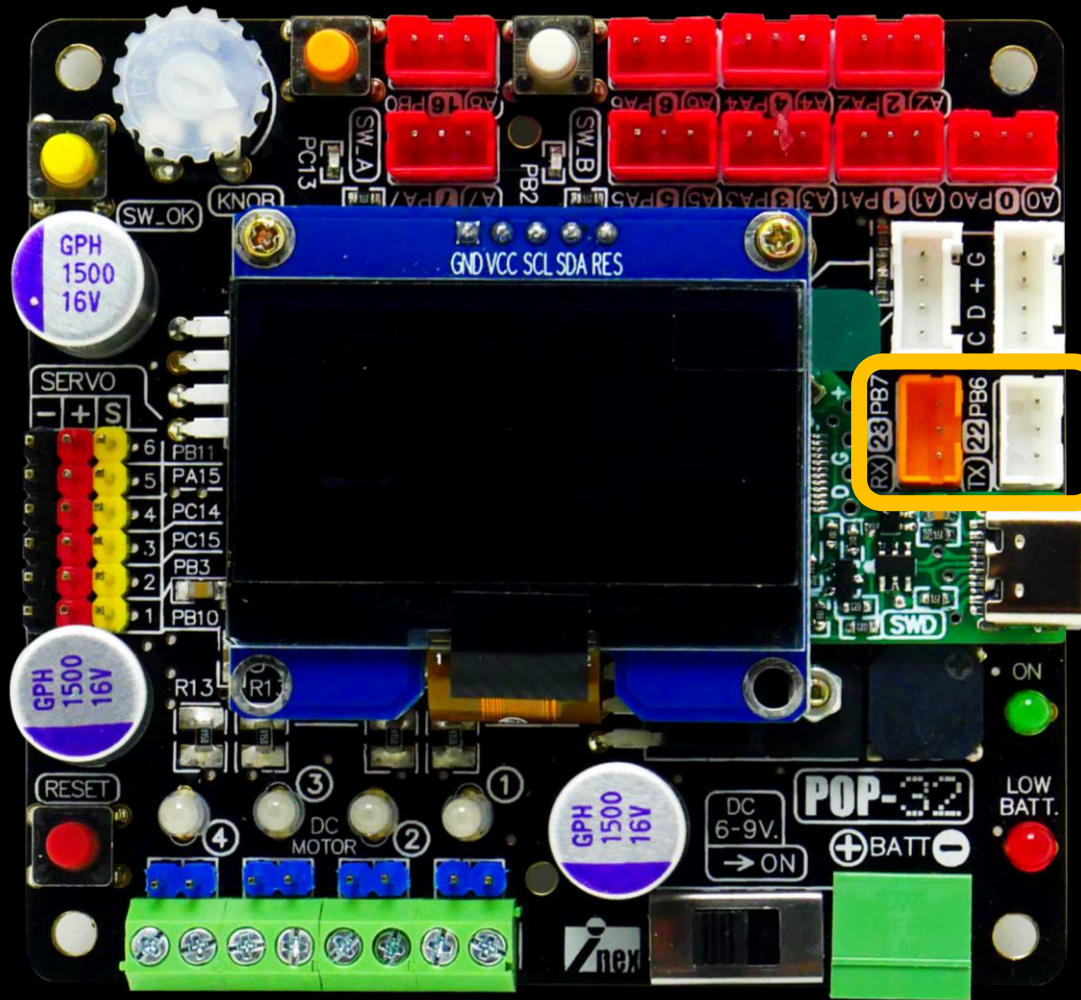


# ZX-IMU

สามารถต่อใช้งานได้ทั้ง I2C(SDA,SCL) และ Serial(Rx,Tx) โมดูลประกอบไปด้วย MPU6050 เป็นเซนเซอร์ Gyro และ Accelerometer โดยมีไมโครคอนโทรลเลอร์ ใช้เป็นตัวกลางเพื่ออ่านค่าจาก MPU6050 และส่งค่าออก Serial(Rx,Tx)



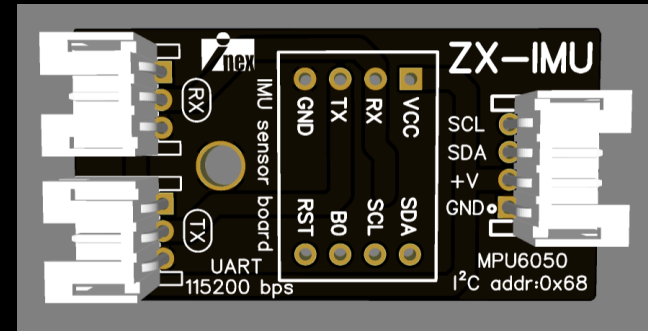
# การต่อใช้งาน

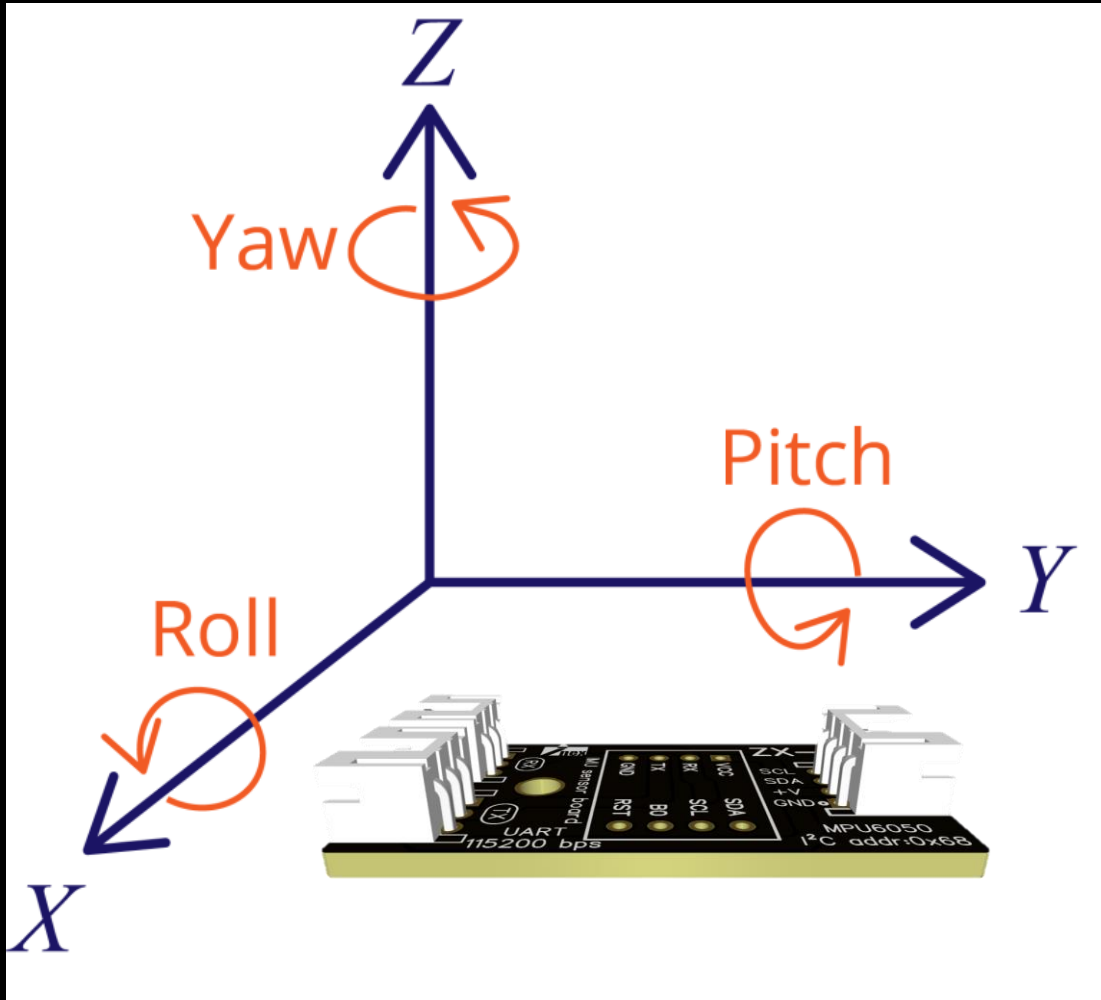


ใช้แบบ UART

TX → RX

RX ← TX





# Yaw

นำมาใช้เป็นเข็มทิศอ้างอิงให้หุ่นยนต์

# คำสั่ง

**Serial.begin(baud rate)**

กำหนดอัตราการสื่อสารข้อมูลเช่น 115200 bs

**Serial.write(data)**

ส่งข้อมูลออกไป ได้เพียงแค่ว่าละ 1 byte

**Serial.available()**

จำนวนไบต์ที่สามารถอ่านได้

**Serial.read()**

อ่านข้อมูลอนุกรมที่เข้ามา

# ฟังก์ชันรีเซ็ตมุม Yaw

ขณะรีเซ็ตหุ่นยนต์ต้องอยู่นิ่งๆ อย่างน้อย  
3 วินาที ก่อนใช้งาน

เริ่มต้นใช้งาน Serial ช่องหมายเลข 1

ปรับปรงค่ามุม pitch และ roll ให้เป็น 0

```
void zeroYaw() {  
  Serial1.begin(115200);delay(100);  
  Serial1.write(0XA5);Serial1.write(0X54);delay(100);  
  Serial1.write(0XA5);Serial1.write(0X55);delay(100);  
  Serial1.write(0XA5);Serial1.write(0X52);delay(100);  
}
```

เลือกโหมดอัตโนมัติ

ปรับปรงค่ามุม Yaw ให้เป็น 0

# ฟังก์ชันอ่านค่า Yaw

```
float pvYaw;  
uint8_t rxCnt = 0, rxBuf[8];  
bool getIMU() {  
    while (Serial1.available()) {  
        rxBuf[rxCnt] = Serial1.read();  
        if (rxCnt == 0 && rxBuf[0] != 0xAA) return false;  
        rxCnt++;  
        if (rxCnt == 8) {  
            rxCnt = 0;  
            if (rxBuf[0] == 0xAA && rxBuf[7] == 0x55) {  
                pvYaw = (int16_t)(rxBuf[1] << 8 | rxBuf[2]) / 100.f;  
                return true;  
            }  
        }  
    }  
    return false;  
}
```

เมื่อมีข้อมูลส่งเข้ามา

อ่านค่าเก็บไว้ในตัวแปร

ตรวจสอบความถูกต้อง

ตรวจสอบความถูกต้อง

อ่านและคำนวณค่ามุม Yaw



# สร้างโปรแกรมอ่านค่าYaw

code -> [Test\\_ZX-IMU.ino](#)

```
#include <POP32.h>

void setup() {
  zeroYaw();
}

void loop() {
  if (SW_A()) {
    zeroYaw();
  }
  getIMU();
  oled.text(0, 0, "Yaw=%f",pvYaw);
  oled.show();
}
```

กดปุ่ม A เพื่อรีเซ็ตเข็มทิศ จากนั้น  
หมุนหุ่นยนต์เพื่อทดสอบ

## คำสั่งตรวจสอบเงื่อนไข while

รูปแบบ

```
while(เงื่อนไข) {  
    ทำคำสั่งจนเงื่อนไขเป็นเท็จ;  
}
```

ออกจาก while แบบทันทีได้ด้วยคำสั่ง

break;

```
#include <POP32.h>  
int i=0;  
void setup() {}  
void loop() {  
    while(SW_A()){  
        beep();  
    }  
}
```

# คำสั่งวนทำซ้ำแบบมีเงื่อนไข **for**

รูปแบบ

```
for (ค่าเริ่มต้น; เงื่อนไข; เพิ่ม/ลดค่า)
{
    //statement(s);
}
```

ออกจาก for แบบทันทีได้ด้วยคำสั่ง

**break;**

```
#include <POP32.h>
void setup() {
    waitSW_OK_bmp();}
void loop() {
    for (int i = 0; i < 5; i++) {
        oled.text(0, 0, "%d ", i);
        oled.show();
        delay(100);
    }
}
```

# คำสั่ง

**millis()**

เป็นคำสั่งที่ใช้อ่านค่าเวลาที่กำลังเดินอยู่มีหน่วยเป็น มิลลิวินาที

# ตัวอย่าง

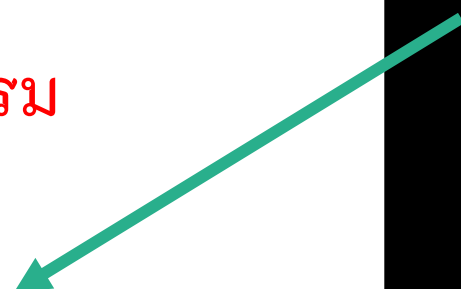
```
unsigned long loopTime = millis()
```

loopTime จะมีค่าเป็นเวลานับเพิ่มเข้าไปเรื่อยๆ

# สร้างโปรแกรมออกจาก while โดยใช้เวลาในการหยุด

```
loopTimer = millis();  
while (1) {  
  command1;  
  command2;  
  คำสั่งต่างๆที่เกี่ยวข้องในโปรแกรม  
  command3;  
  command4;  
  if (millis() - loopTimer >= 2000)break;  
}
```

จะวนอยู่ใน while นาน 2 วินาที



# สร้างโปรแกรมอ่านค่าYaw

code -> [Test\\_Auto\\_Zero.ino](#)

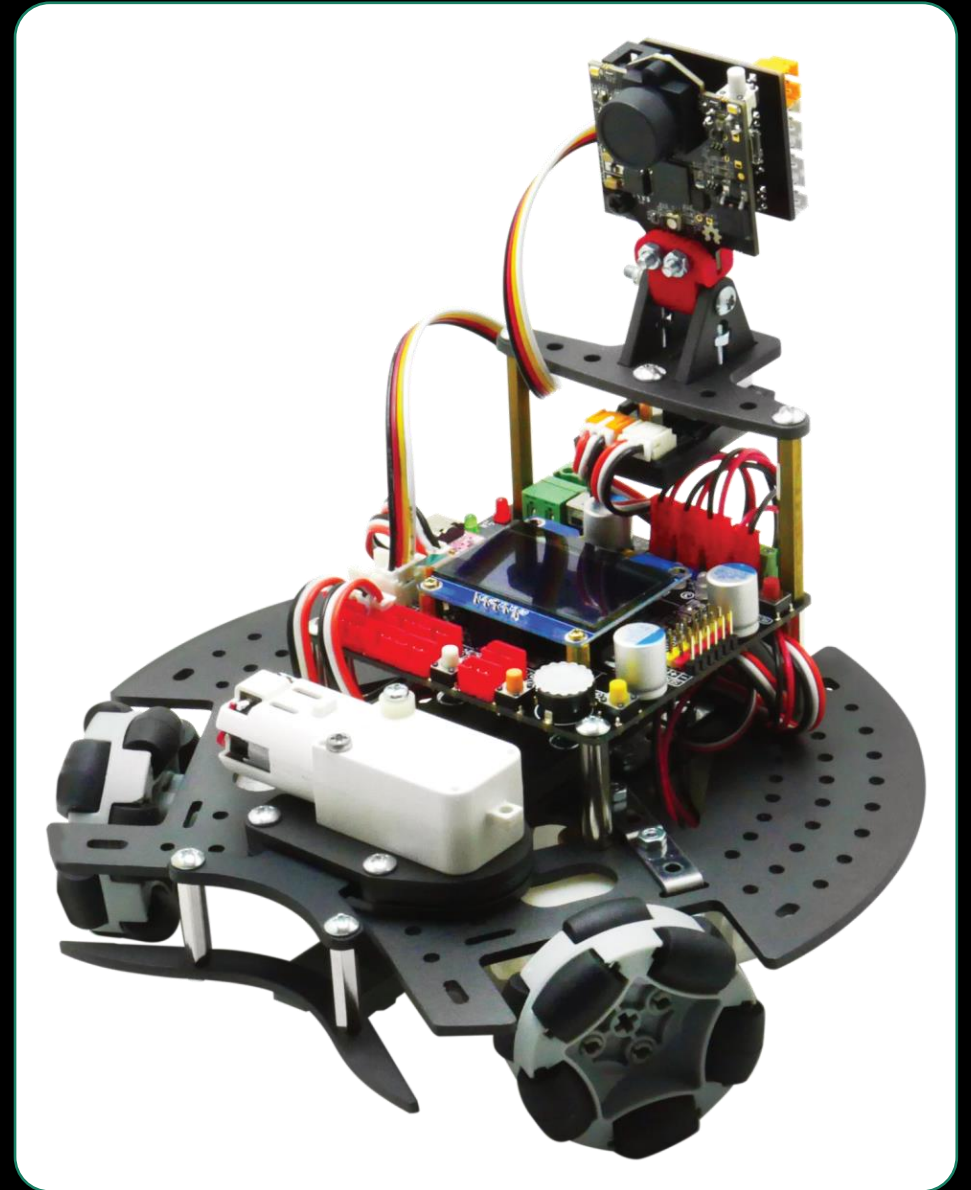
```
#include <POP32.h>
void setup() {
  Auto_zero();
  oled.text(4, 0, "SW_B => RUN");
  while (!SW_B()) {
    getIMU();
    oled.text(0, 0, "Yaw=%f    ", pvYaw);
    oled.show();
  }
void loop() {
}
```

```
void Auto_zero() {
  zeroYaw();
  getIMU();
  int timerOut = millis();
  oled.clear();
  oled.text(1, 2, "Setting zero");
  while (abs(pvYaw) > 0.02) {
    if (getIMU()) {
      oled.text(3, 6, "Yaw: %f    ", pvYaw);
      oled.show();
      beep();
      if (millis() - timerOut > 5000) {
        zeroYaw();
        timerOut = millis();
      }
    }
  }
  oled.clear();
  oled.show();
}
```

รอจนกว่าเสียงจะหยุด จากนั้นหมุนหุ่นยนต์เพื่อ  
ทดสอบ จากนั้นกดปุ่ม B เพื่อออกไปยัง void loop()

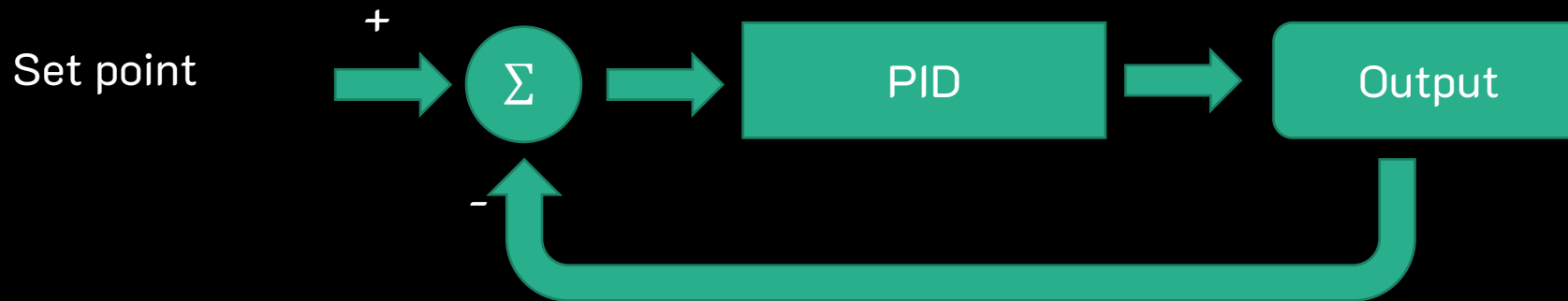
# การเคลื่อนที่เพื่อรักษาทิศของหุ่นยนต์ให้คงที่

การเคลื่อนที่เพื่อรักษาทิศทางของหุ่นยนต์ให้คงที่  
จะต้องอาศัยการควบคุมความเร็วมอเตอร์ โดยที่  
ความเร็วมอเตอร์จะแปรผันตามความต่างของทิศ  
อ้างอิงอย่างรวดเร็ว



# การควบคุมแบบ PID : Proportional Integral Derivative

เป็นการควบคุมแบบป้อนกลับ โดยใช้วิธีการนำค่าความผิดพลาดที่ได้จากผลต่างระหว่าง ค่าที่ต้องการ (Set point) กับค่าป้อนกลับมาคำนวณ เพื่อจะพยายามลดค่าความผิดพลาดให้เหลือน้อยที่สุด หรือให้ค่าความผิดพลาดเข้าใกล้ 0 ให้ได้มากที่สุด





# การควบคุมแบบ PID : Proportional Integral Derivative

**P : สัดส่วนความผิดพลาด**

$$\text{Error} = \text{Set\_point} - \text{feedback}$$

**I : สัดส่วนความผิดพลาดสะสม**

$$\text{SumError} = \text{SumError} + \text{Error}$$

**D : สัดส่วนความผิดพลาดปัจจุบันต่ออดีต**

$$\text{dError} = \text{Error} - \text{PrevError}$$

สมการ PID จะนำค่า  $k_p, k_i, k_d$  มาเพิ่มอัตราการขยายในแต่ละเทอม

$$\text{Output} = (\text{Error} * k_p) + (\text{SumError} * k_i) + (\text{dError} * k_d)$$

## หน้าที่ของแต่ละเทอม

**Kp** : ช่วยทำหน้าที่ควบคุมทิศทางหุ้่นยนต์ให้เข้าใกล้สู่จุดที่ค่าผิดพลาดเท่ากับ 0

**Ki** : ช่วยให้หุ้่นยนต์เข้าใกล้สู่จุดที่ค่าผิดพลาดเท่ากับ 0 มากขึ้น

**Kd** : ช่วยลดการสะบัด ให้หุ้่นยนต์เข้าสู่จุดที่ค่าผิดพลาดเท่ากับ 0 ได้เร็ว

การปรับจูนค่า  $K_p, K_i, K_d$  จะขึ้นอยู่กับความเร็วของมอเตอร์, การตอบสนองของค่า Error กล่าวคือความเร็วของตัวประมวลผลหรือการตอบสนองของเซนเซอร์ที่ใช้งานนั่นเอง ในโปรแกรมตัวอย่างจะเห็นว่าอาจปรับแค่ค่า  $k_p$  ก็เพียงพอที่จะทำให้หุ้่นยนต์ปรับทิศได้อย่างมีประสิทธิภาพแล้ว

# สร้างโปรแกรมเคลื่อนที่รักษาทิศของหุ่นยนต์

```
float pvYaw;  
#define head_Kp 0.1f  
#define head_Ki 0.0f  
#define head_Kd 0.0f  
float head_error, head_pError, head_w ,head_d, head_i;
```

ประกาศตัวแปรและกำหนดค่า

```
void setup(){  
  zeroYaw();  
  waitSW_A_bmp();  
}
```

เริ่มต้นใช้งานรีเซตค่า Yaw และ รอกดปุ่ม

# คำสั่ง

**constrain(value,min,max)**

เป็นคำสั่งที่ใช้กำหนดค่าให้อยู่ในช่วงที่เราต้องการ

# ตัวอย่าง

```
int y = constrain(300,-180,180)
```

y จะมีค่าเท่ากับ 180

**P : สัดส่วนความผิดพลาด**

$$\text{Error} = \text{Set\_point} - \text{feedback}$$

$$\text{head\_error} = \text{spYaw} - \text{pvYaw}$$

หาค่าความผิดพลาด

$$\text{head\_w} = (\text{head\_error} * \text{head\_Kp})$$

เพิ่มอัตราการขยาย

I : สัดส่วนความผิดพลาดสะสม

$\text{SumError} = \text{SumError} + \text{Error}$

$\text{head}_i = \text{head}_i + \text{head\_error}$

สะสมค่าความผิดพลาด

$\text{head}_i = \text{constrain}(\text{head}_i, -180, 180)$

จำกัดรอบค่าความผิดพลาด ไม่ให้สูงเกินค่าความเร็วสูงสุด

$\text{head}_w = (\text{head}_i * \text{head\_Ki})$

เพิ่มอัตราการขยาย

D : สัดส่วนความผิดพลาดปัจจุบันต่ออดีต

$$dError = Error - PrevError$$

ผลต่างค่าความผิดพลาดปัจจุบันต่ออดีต

$$\begin{aligned} head\_d &= head\_error - head\_pError \\ head\_pError &= head\_error \end{aligned}$$

$$head\_w = (head\_d * head\_Kd)$$

เพิ่มอัตราการขยาย

## รวมเทอม PID เข้าด้วยกัน

```
head_w = (head_error * head_Kp) + (head_i * head_Ki) + (head_d * head_Kd)
```

รวมทุกเทอมเข้าด้วยกัน

```
head_w = constrain(head_w ,-100,100);
```

จำกัดค่าไม่ให้เกินความเร็วมอเตอร์

คำสั่งปรับความเร็วมอเตอร์

```
holonomic(0, 0, head_w)
```



# สร้างฟังก์ชัน heading

```
#define head_Kp 0.1f
#define head_Ki 0.0f
#define head_Kd 0.0f
float head_error, head_pError, head_w ,head_d, head_i;
void heading(float spd, float theta, float spYaw){
    head_error = spYaw - pvYaw;
    head_i = head_i + head_error;
    head_i = constrain(head_i ,-180,180);
    head_d = head_error - head_pError;
    head_w = (head_error * head_Kp) + (head_i * head_Ki) + (head_d * head_Kd);
    head_w = constrain(head_w ,-100,100);
    holonomic(spd, theta, head_w);
    head_pError = head_error;
}
```

# สร้างโปรแกรมเคลื่อนที่รักษาทิศของหุ่นยนต์

code -> [Test\\_heading.ino](#)

```
void setup() {  
  Auto_zero();  
  waitSW_A_bmp();  
}  
void loop() {  
  if(SW_A()){  
    Auto_zero();  
  }  
  getIMU();  
  heading(0, 0, 0);  
}
```

กดปุ่ม A เพื่อเลือกทิศใหม่

heading(float spd, float theta, float spYaw)

Spd = ความเร็ว

theta = มุมที่ต้องการเคลื่อนที่ไป

spYaw = ทิศที่ต้องการรักษาไว้

# เคลื่อนที่ไปข้างหน้าโดยการรักษาทิศไว้ให้คงที่

```
void setup() {  
  Auto_zero();  
  waitSW_A_bmp();  
}  
void loop() {  
  if(SW_A()){  
    Auto_zero();  
  }  
  getIMU();  
  heading(20, 90, 0);  
}
```

heading(float spd, float theta, float spYaw)

Spd = 20

theta = 90

spYaw = 0

# ความแตกต่างแบบระหว่าง

ใช้ ZX-IMU

```
void setup() {  
  Auto_zero();  
  waitSW_A_bmp();  
}  
void loop() {  
  if(SW_A()){  
    Auto_zero();  
  }  
  getIMU();  
  heading(20, 90, 0);  
}
```

ปรับหมุนตัวให้ด้านหน้าเข้าที่ทิศอ้างอิงเสมอ

ไม่ใช้ ZX-IMU

```
void setup() {  
  Auto_zero();  
  waitSW_A_bmp();  
}  
void loop() {  
  if(SW_A()){  
    Auto_zero();  
  }  
  getIMU();  
  holonomic(20, 90, 0);  
}
```

ไม่ปรับหมุนตัวคืนมาที่ทิศเดิม

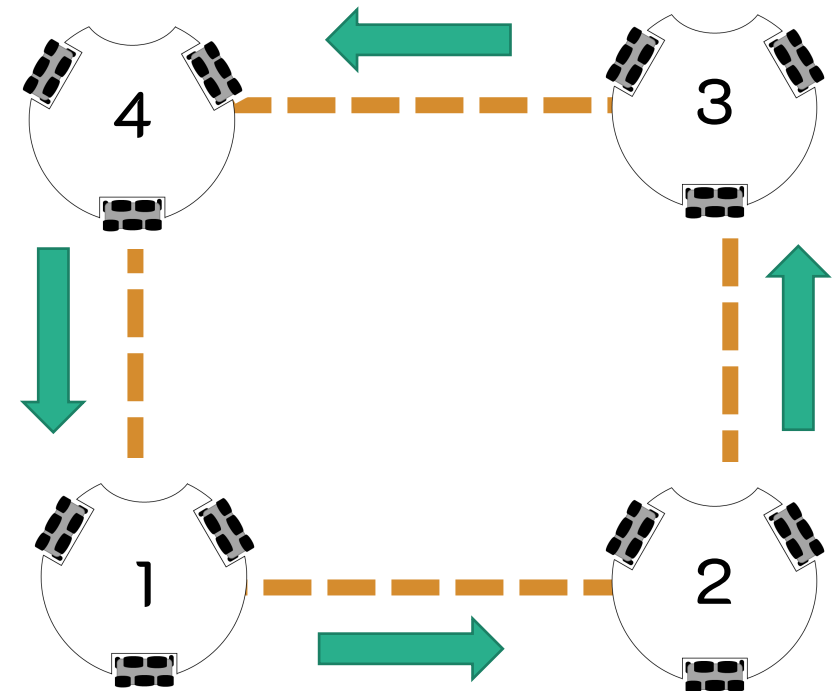
```

unsigned long loopTimer = millis();
void loop() {
  ao();
  waitSW_A_bmp();
  Auto_zero();
  loopTimer = millis();
  while(1){
    getIMU();heading(30, 0, 0);
    if (millis()-loopTimer >= 2000)break;
  }
  loopTimer = millis();
  while(1){
    getIMU();heading(30, 90, 0);
    if (millis()-loopTimer >= 2000)break;
  }
  loopTimer = millis();
  while(1){
    getIMU();heading(30, 180, 0);
    if (millis()-loopTimer >= 2000)break;
  }
  loopTimer = millis();
  while(1){
    getIMU();heading(30, 270, 0);
    if (millis()-loopTimer >= 2000)break;
  }
}

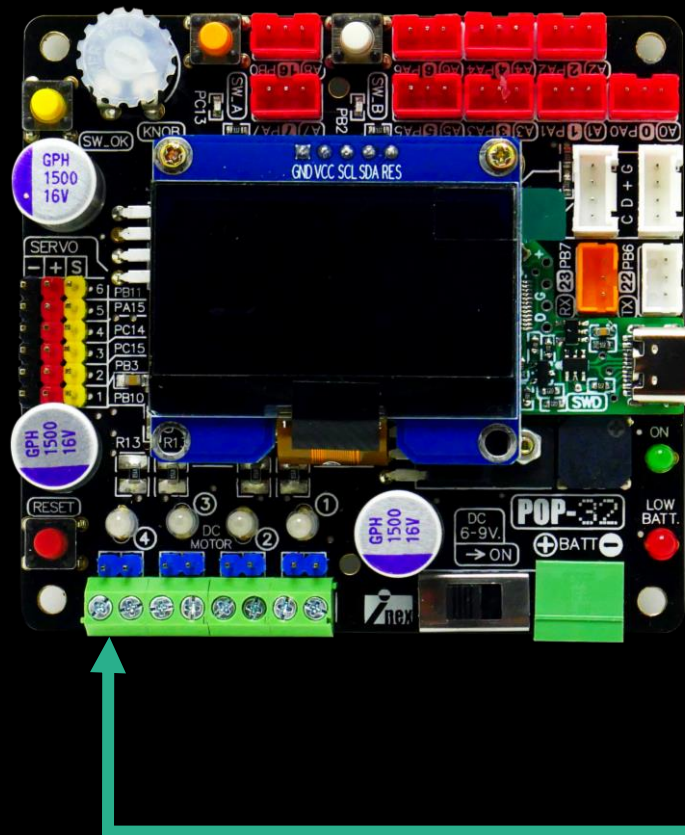
```

# ขับเคลื่อนหุ่นยนต์เป็นรูปสี่เหลี่ยม

code -> [Test\\_holonomic\\_heading.ino](#)

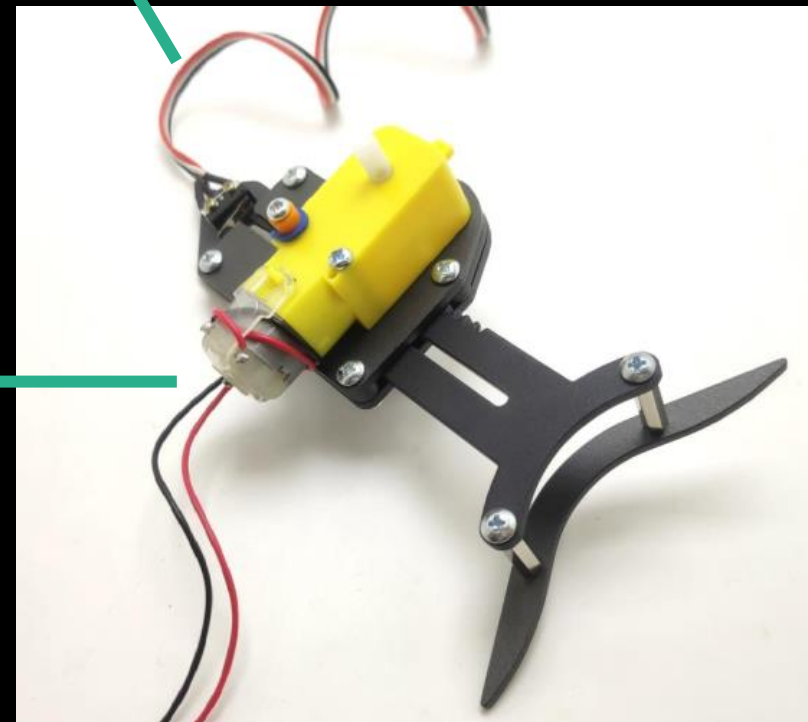


# ตัวถังลูกบอล



สวิตช์ A0

มอเตอร์ช่องที่ 4



# สร้างโปรแกรมยิงลูกบอล

code -> Test\_shoot\_V2.ino

```
void shoot() {  
    motor(4, reloadSpd);  
    delay(150);  
    motor(4, 0);  
    delay(50);  
}
```

```
#include <POP32.h>  
#define limPin PA0  
#define reloadSpd 60  
int timer = 0;  
void setup() {}  
void loop() {  
    if (SW_A()) { reload();}  
    if (SW_B()) { shoot(); }  
    if (SW_OK()) {shoot();reload();}  
}
```

```
void reload() {  
    motor(4, reloadSpd);  
    timer = 0;  
    for (int i = 0; i < 2000; i++) {  
        timer++;  
        if (in(limPin)) break;  
        delay(1);  
    }  
    if (timer == 2000) { // ถ้าก้านยิงติด  
        motor(4, -reloadSpd); // เลื่อนก้านยิงไปข้างหน้า  
        delay(500); // ก่อน 0.5 วินาที  
        motor(4, reloadSpd);  
        timer = 0;  
        for (int i = 0; i < 2000; i++) {  
            timer++;  
            if (in(limPin)) break;  
            delay(1);  
        }  
    }  
    motor(4, 0);  
}
```

จะวนทำซ้ำนาน 2 วินาที หรือจนกว่าสวิตช์  
จะถูกกด แล้วมอเตอร์หยุดทำงาน

# เริ่มต้นใช้งานกล่อง Pixy

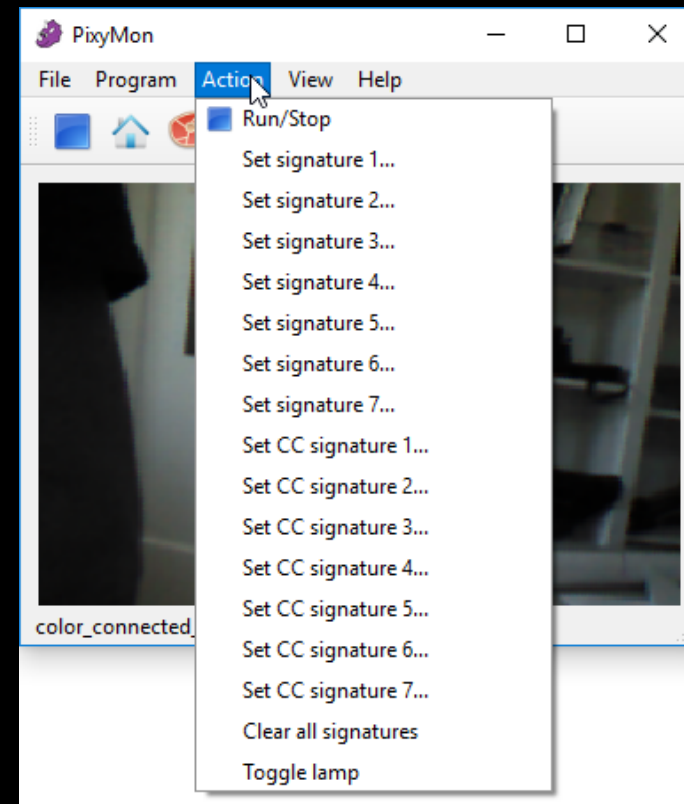
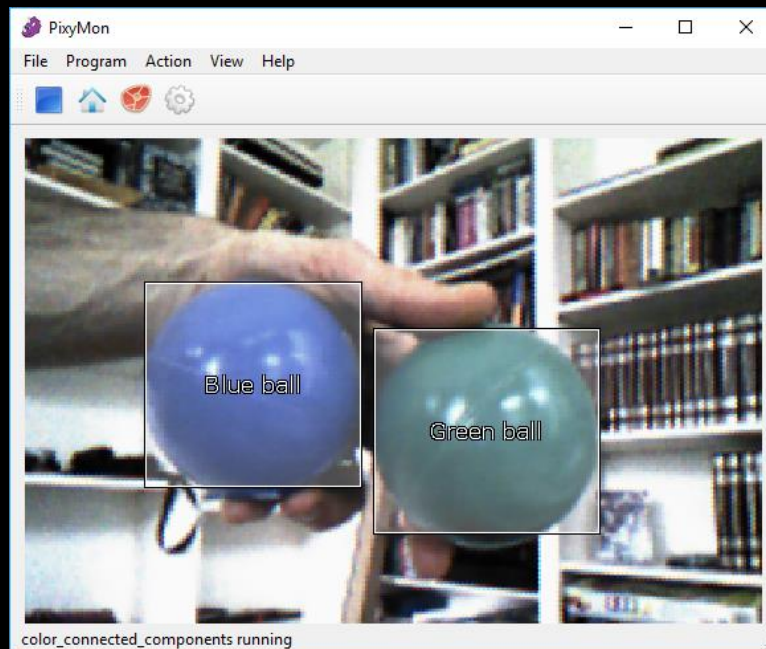




ใช้จำแนกสีได้ตามที่เรากำหนด



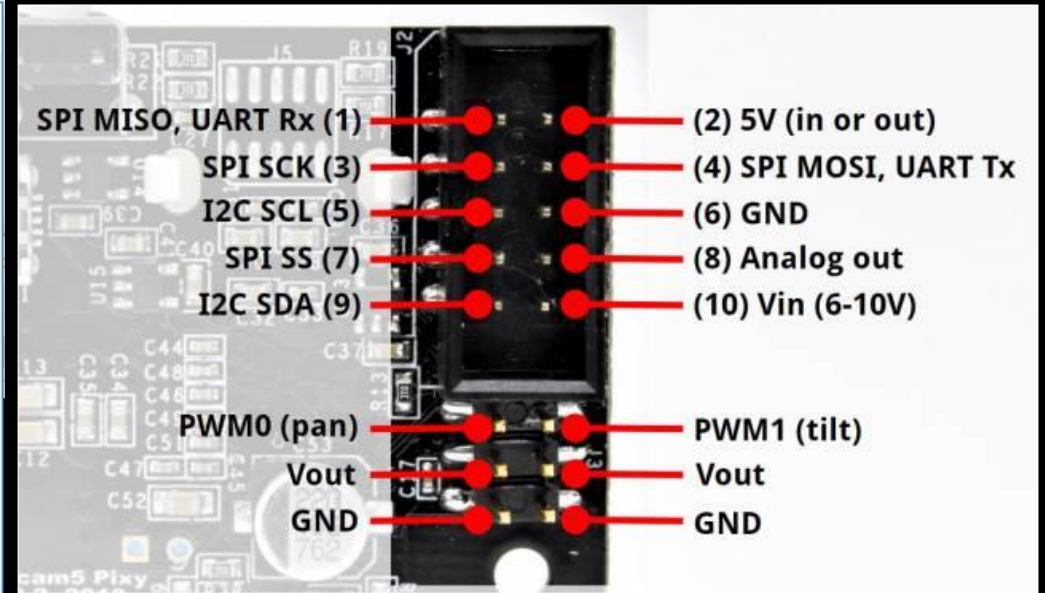
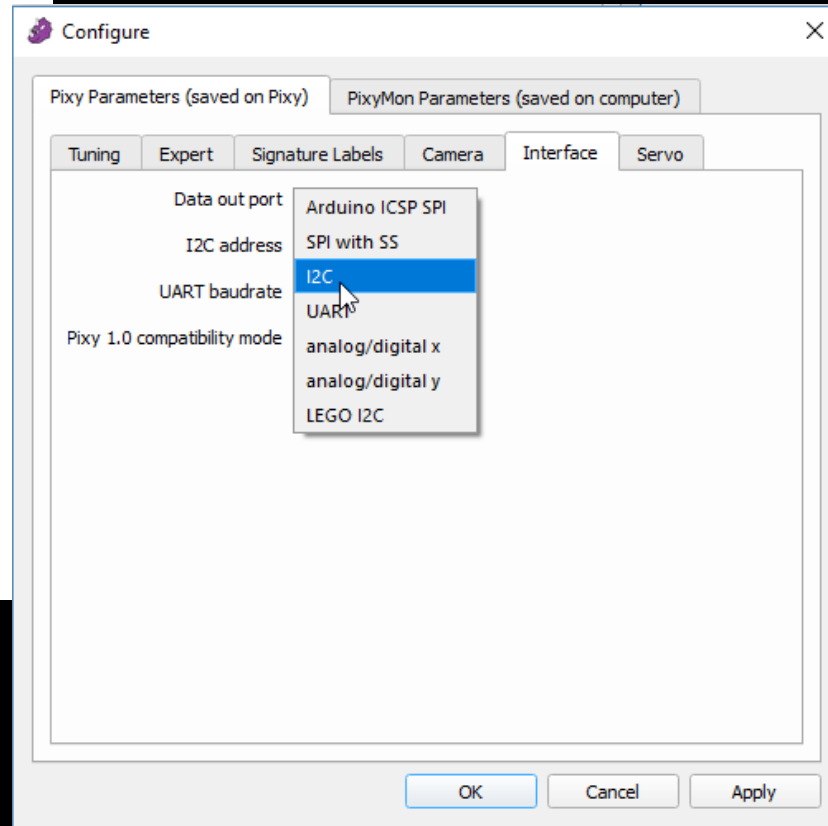
Pixy



เชื่อมต่อกับบอร์ดไมโครคอนโทรลเลอร์ด้วย SPI , IIC หรือ UATR



Pixy



# ติดตั้งโปรแกรม PixyMon v2

PixyMon V2 คือ โปรแกรมช่วยตั้งค่าต่างๆของกล้อง Pixy  
ดาวน์โหลดได้ที่ <https://pixycam.com/downloads-pixy2>

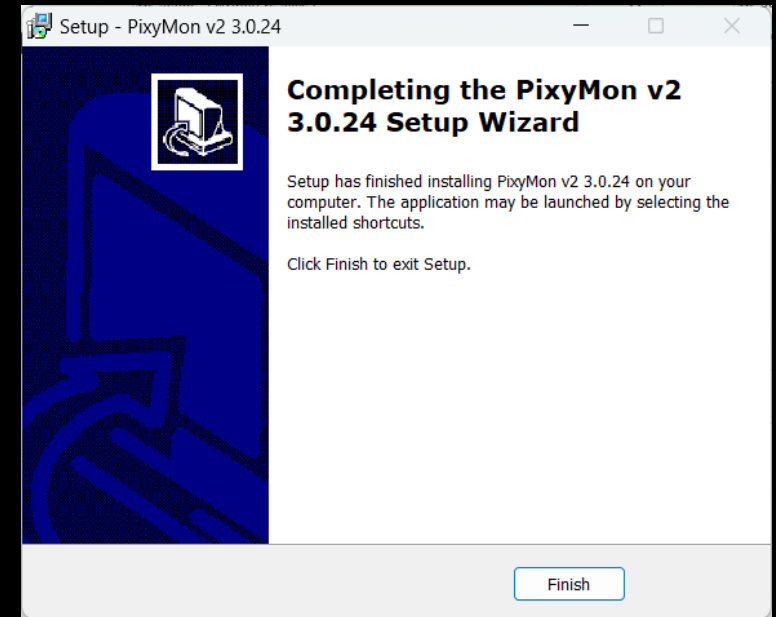
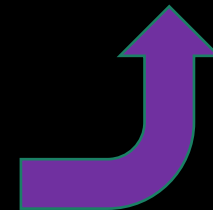
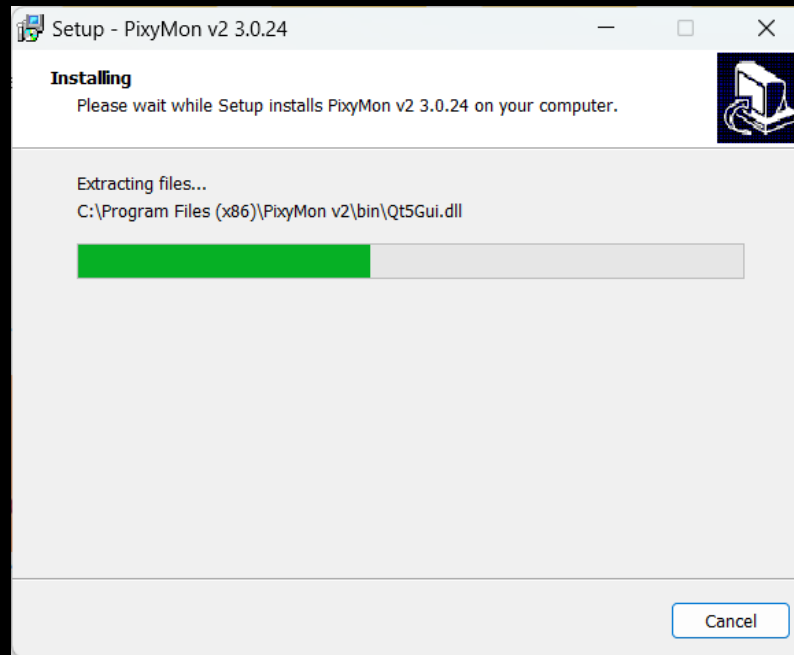
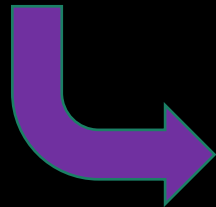
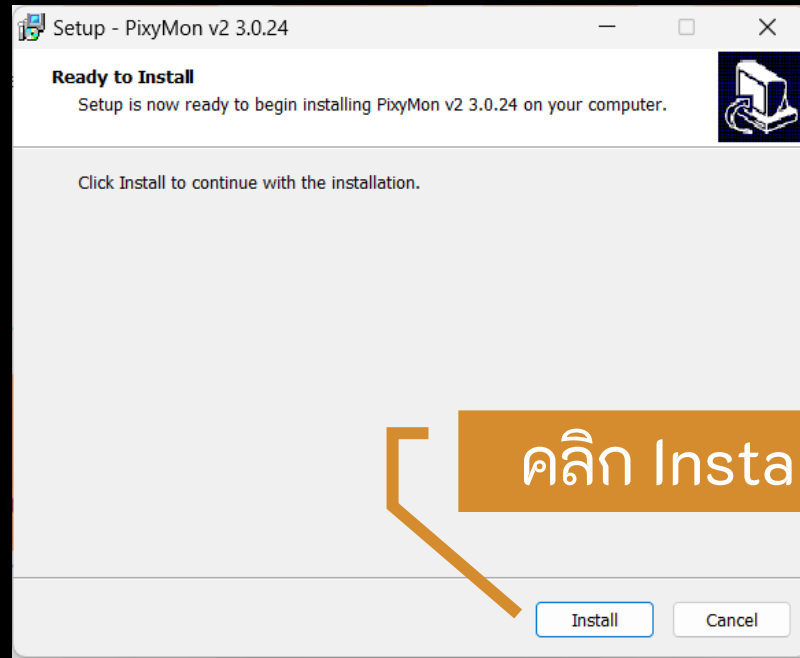
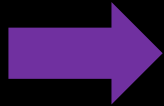
## PixyMon v2

PixyMon v2 is the configuration utility for Pixy2 that runs on Windows, MacOS and Linux.

- [Pixymon v2 Windows version 3.0.24 \(exe\)](#)
  - [installation docs for Windows Vista, 7, 8, 10](#)
  - [installation docs for XP](#)
- [PixyMon v2 Mac version 3.0.24 \(dmg, High Sierra\)](#)
  - [installation docs](#)
- Linux Pixymon v2 is available through [github](#)
  - [installation docs](#)

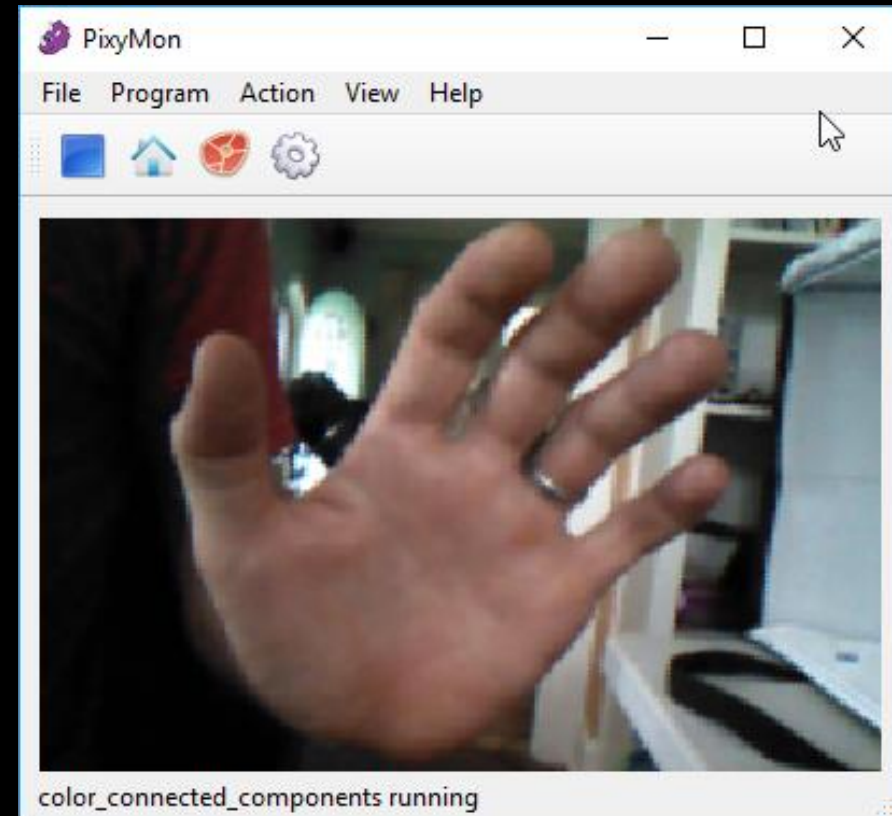
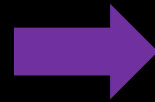
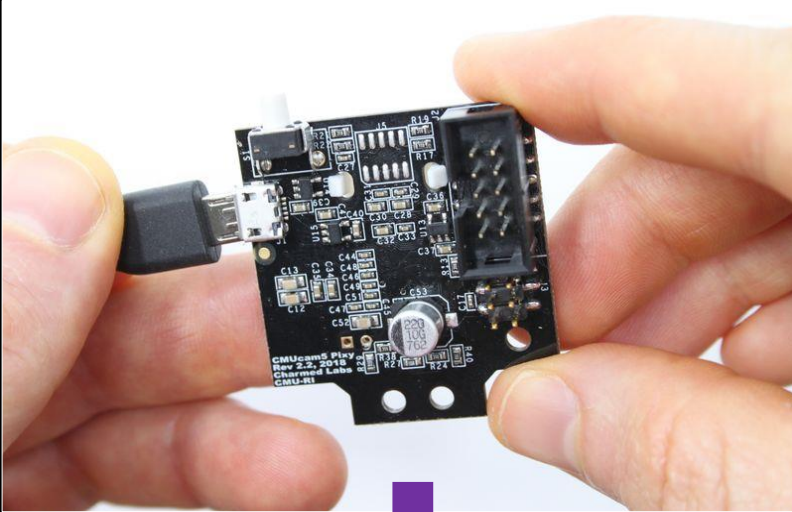
เลือกติดตั้งให้ตรงตามระบบปฏิบัติการของเครื่องคอมพิวเตอร์

ดับเบิลคลิกเพื่อติดตั้ง





# ทดสอบเชื่อมต่อ Pixy เข้ากับคอมพิวเตอร์



เปิดโปรแกรม PixyMon เพื่อทดสอบ โปรแกรมจะแสดงภาพจากกล้องขึ้นมาโดยอัตโนมัติ

# อัปเดตเฟิร์มแวร์

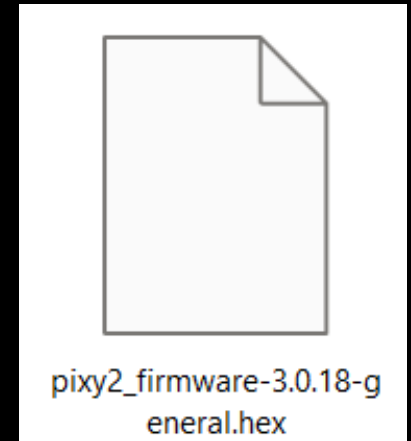
ดาวน์โหลดได้ที่

<https://pixycam.com/downloads-pixy2>

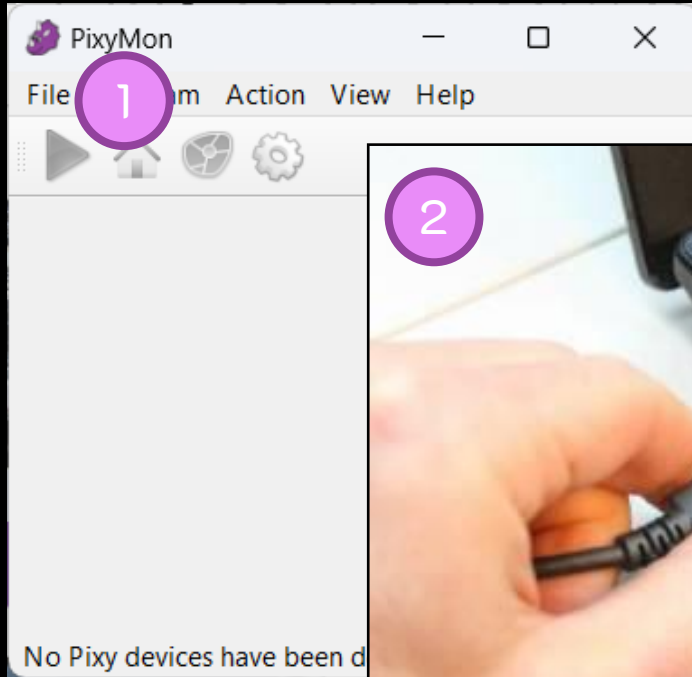
## Pixy2 firmware

Pixy2 firmware is code that runs on Pixy2 itself.

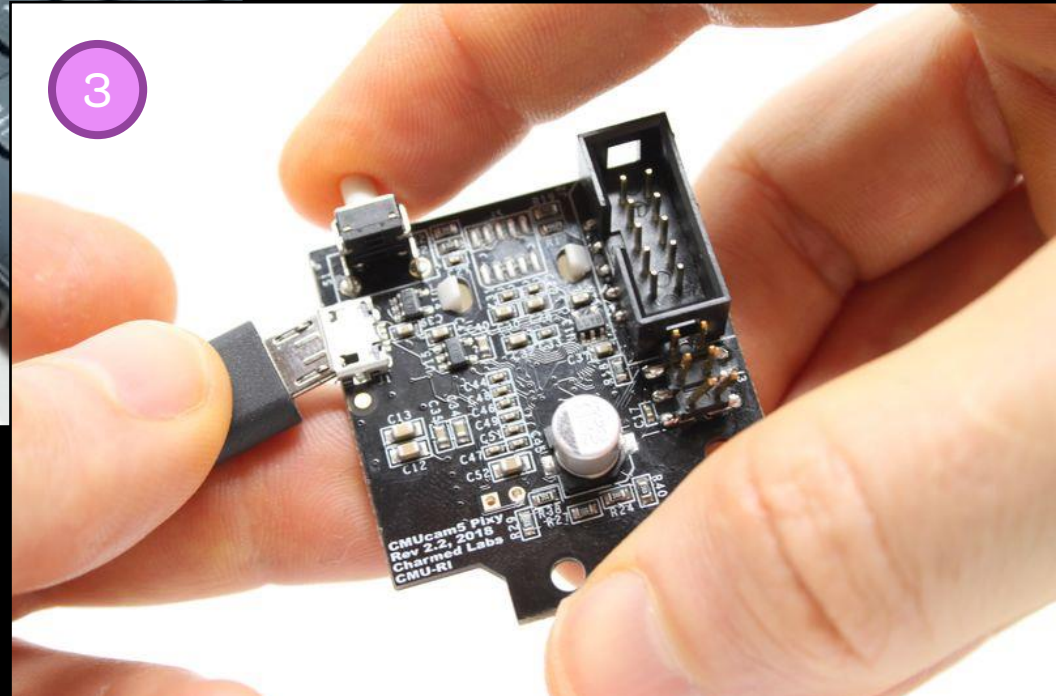
- Pixy2 general firmware version 3.0.18 (hex)
- Pixy2 LEGO firmware version 3.0.18 (hex)
  - installation docs



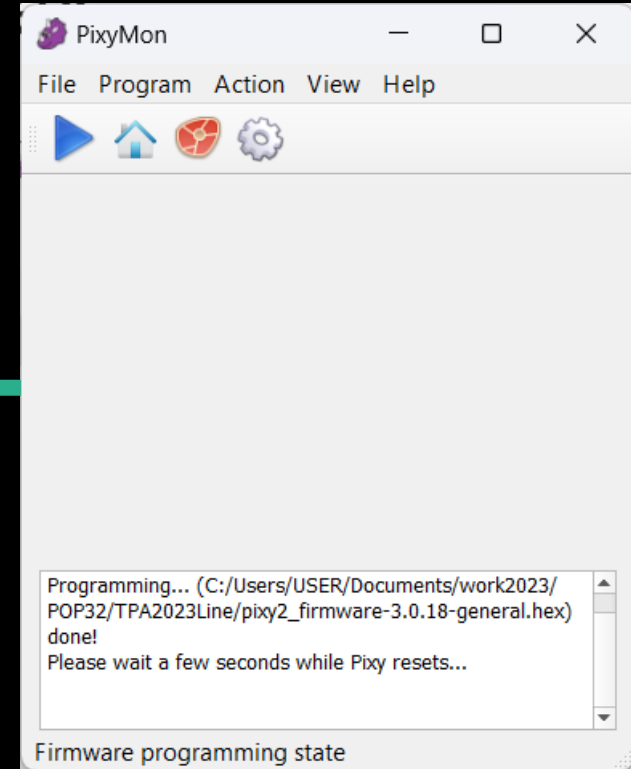
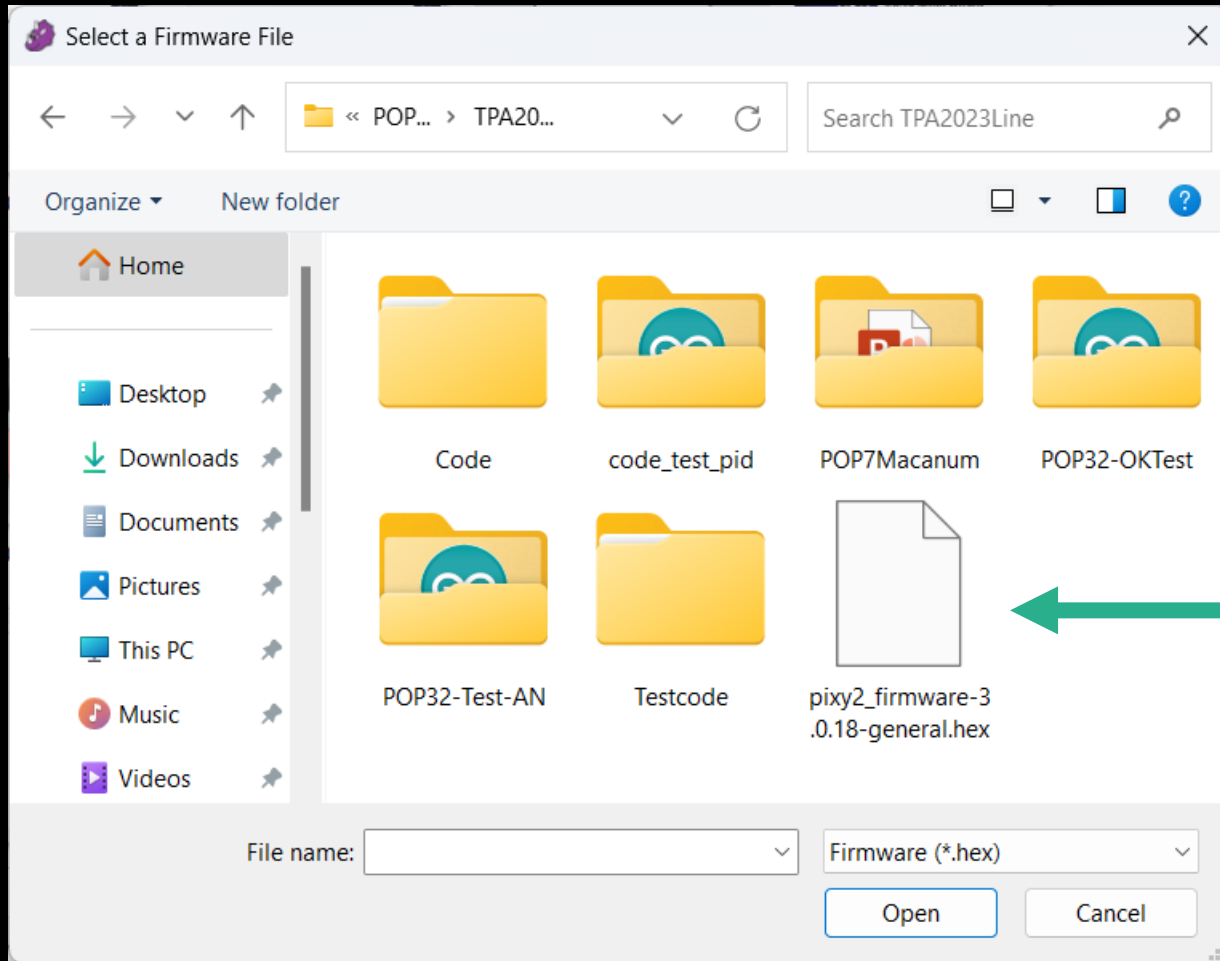
# อัพเกรดเฟิร์มแวร์



กดปุ่ม S1 ก่อนแล้วค่อยเสียบ USB



# เลือกไฟล์ .hex

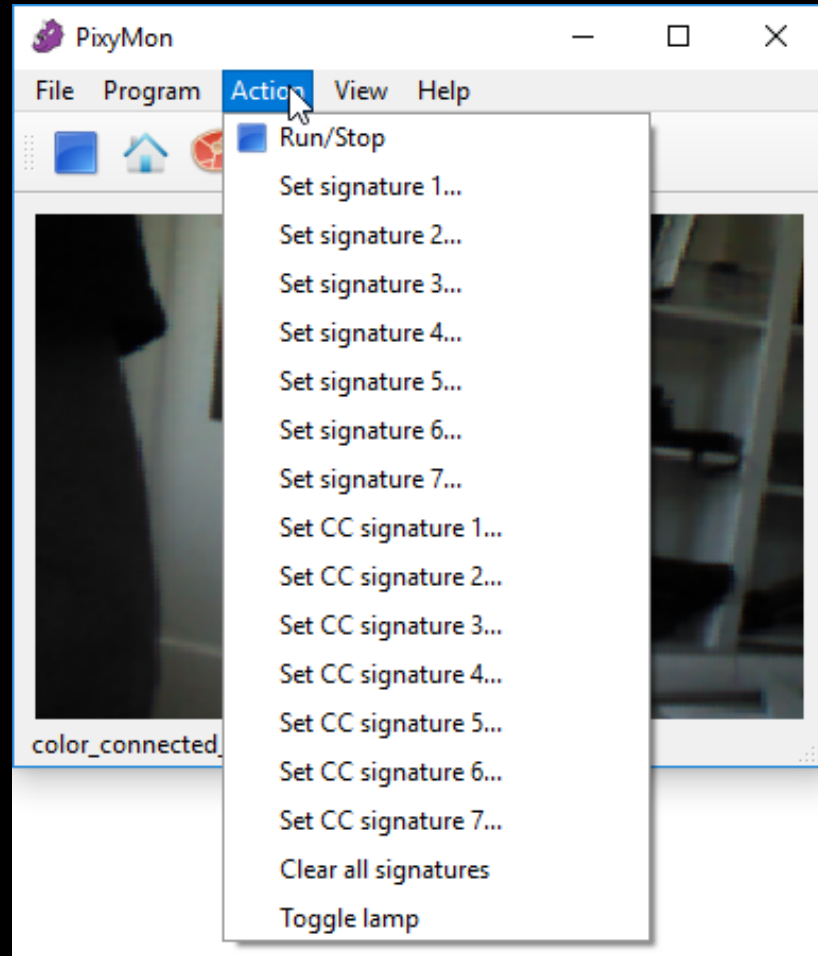


เมื่อเลือกเสร็จแล้ว

โปรแกรมจะเปิดหน้าต่างขึ้นมาให้เลือกไฟล์



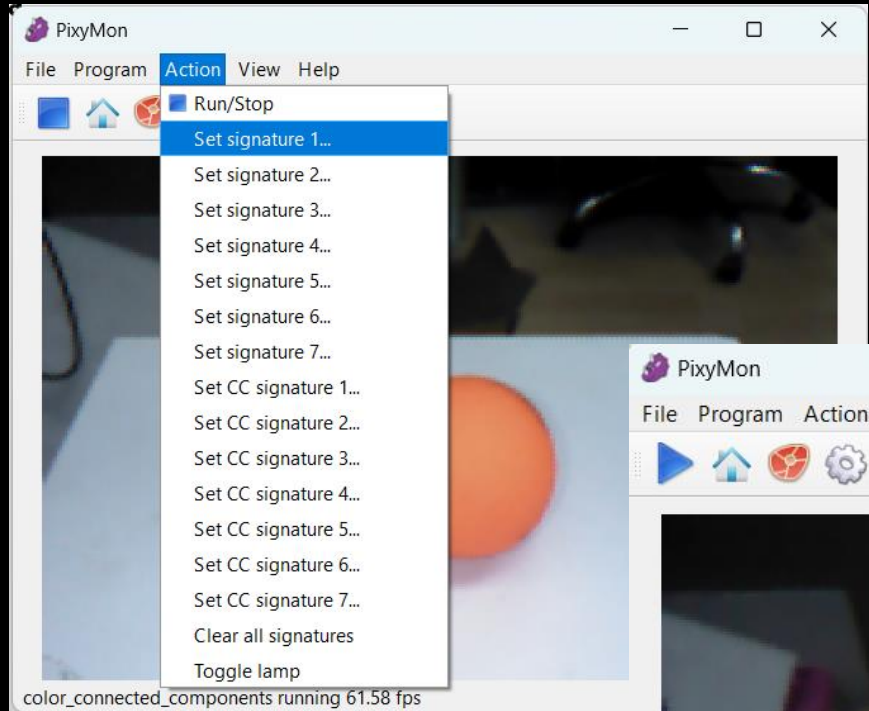
# การตั้งค่าเพื่อจำแนกสี



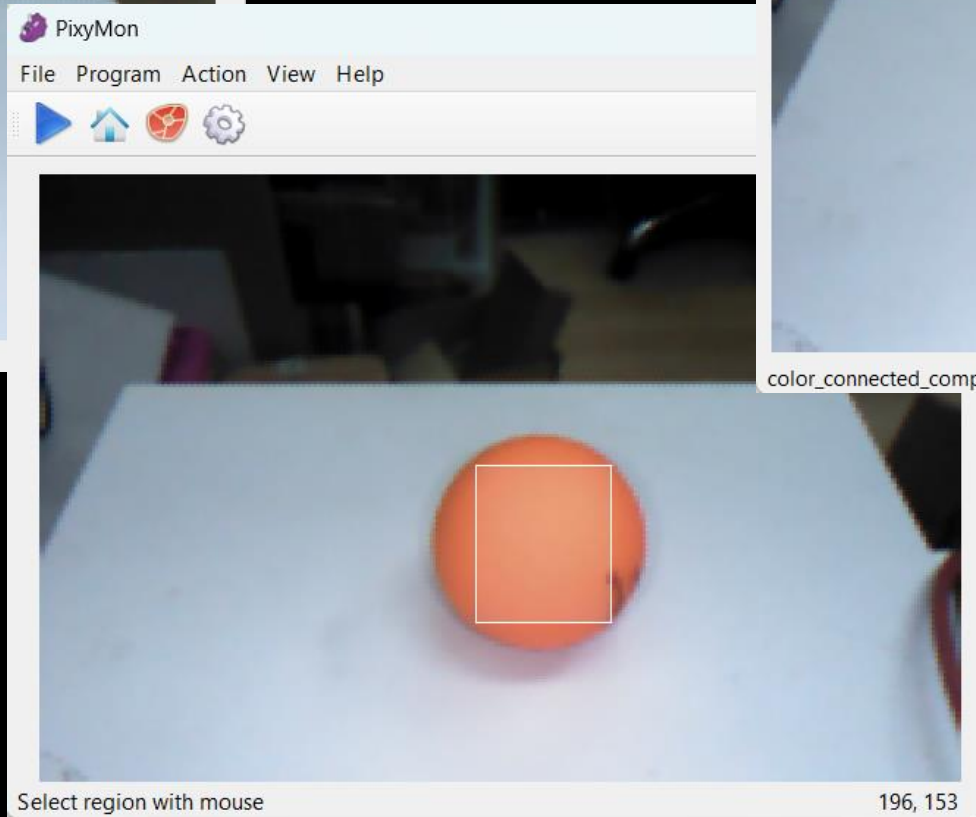
## ตัวเลือกเมนู

1. Run/Stop : สั่งให้หยุดการทำงานหรือทำงาน
2. Set signature : การจำแนกแบบสีเดียว
3. Set CC signature : การจำแนกแบบรหัสสี
4. Clear all signatures : ล้างการจำแนกทั้งหมด
5. Toggle lamp : เปิด-ปิด LED บน Pixy

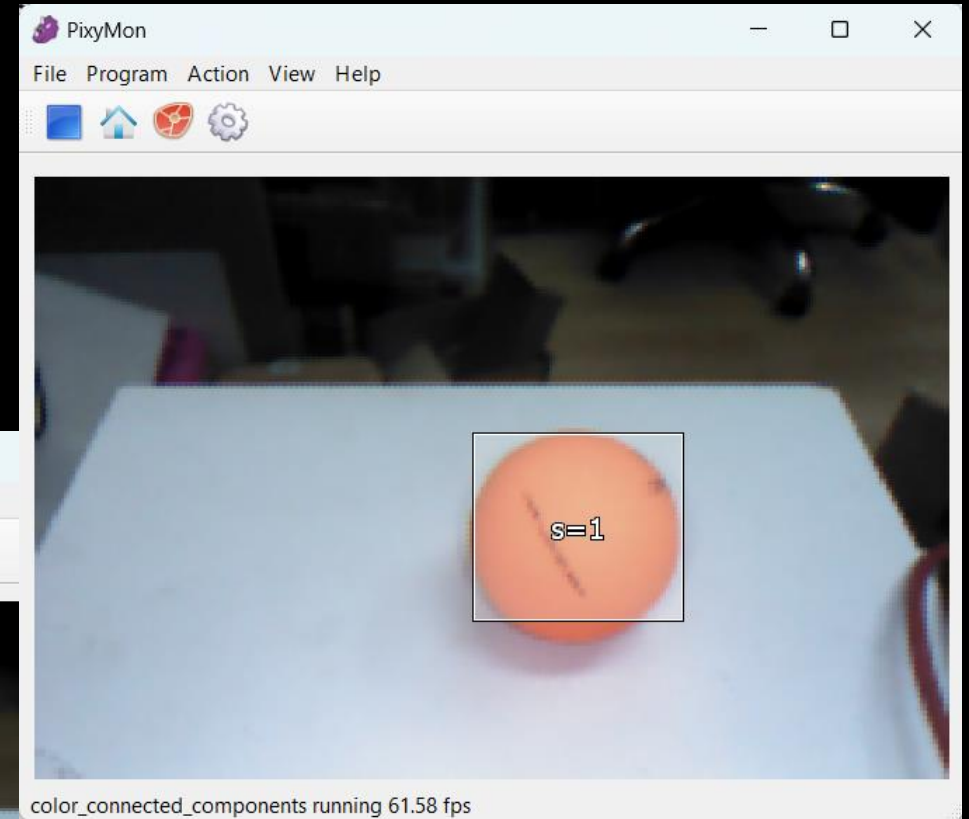
# เลือกการจำแนกแบบสีเดียว



เลือกหมายเลข



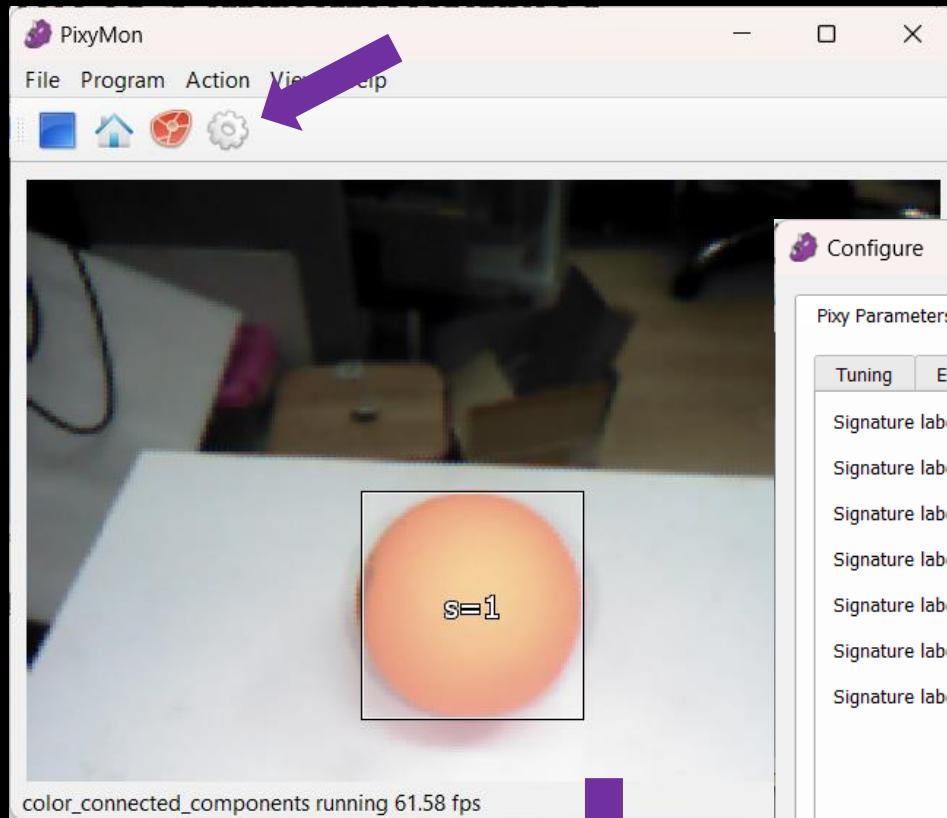
เลือกสี



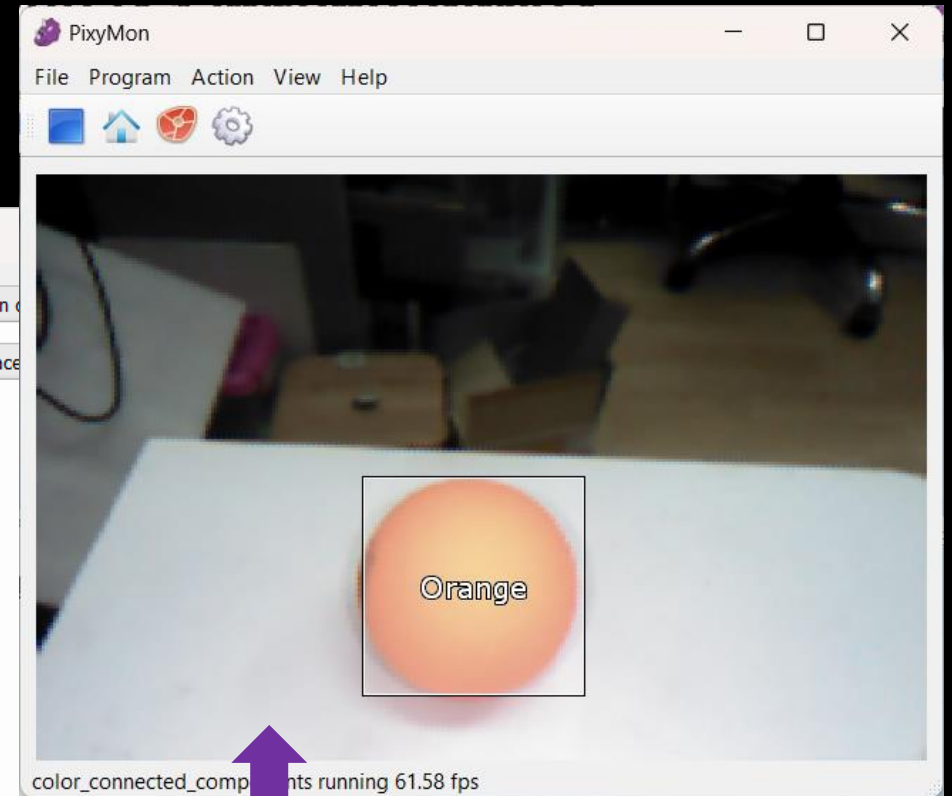
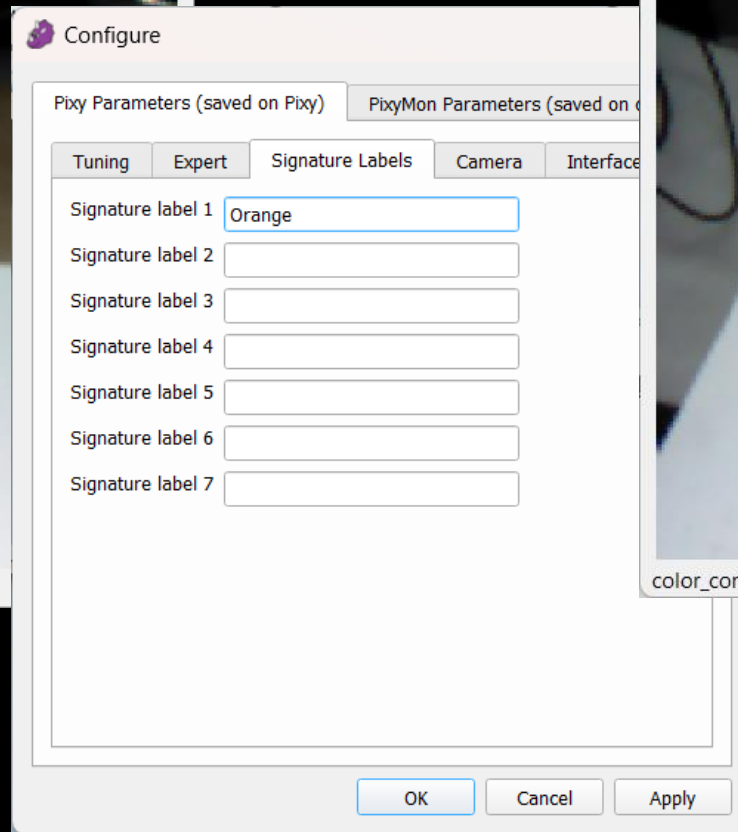
ผลลัพธ์



# ตั้งชื่อวัตถุสีใหม่



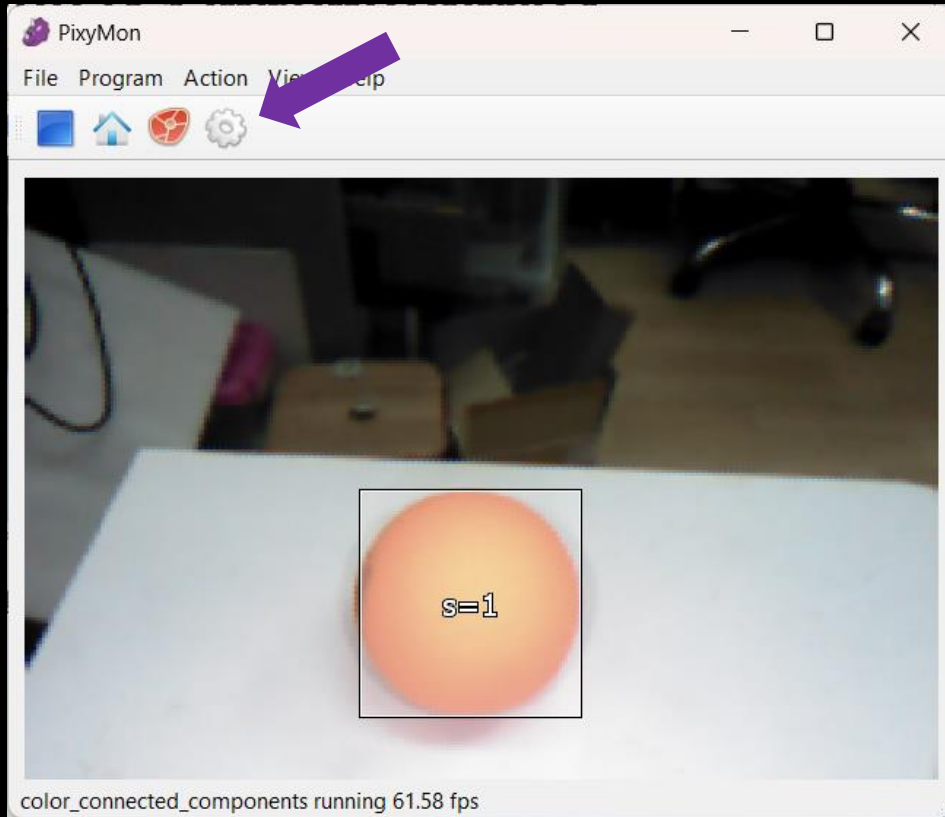
เลือกตั้งค่า



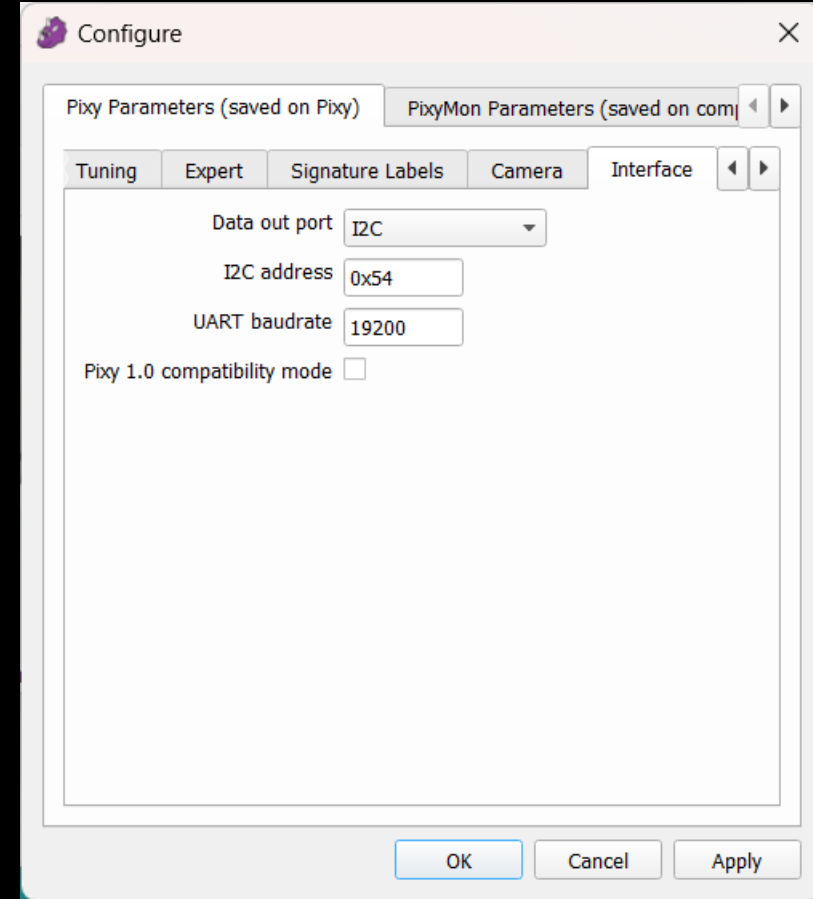
ผลลัพธ์

เลือก Signature Labels เปลี่ยนชื่อแล้วกดปุ่ม OK

# กำหนดรูปแบบการเชื่อมต่อ

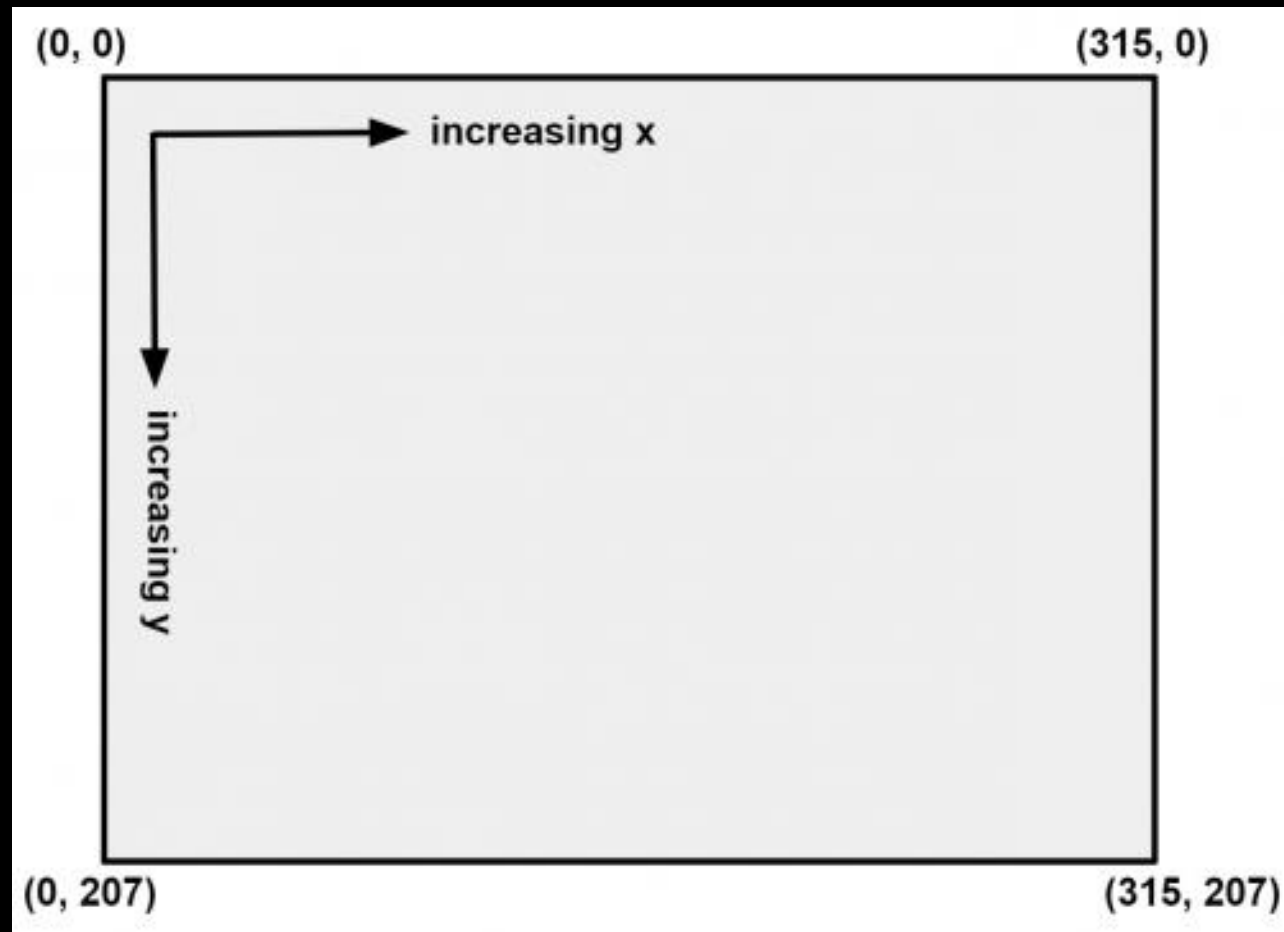


เลือกตั้งค่า



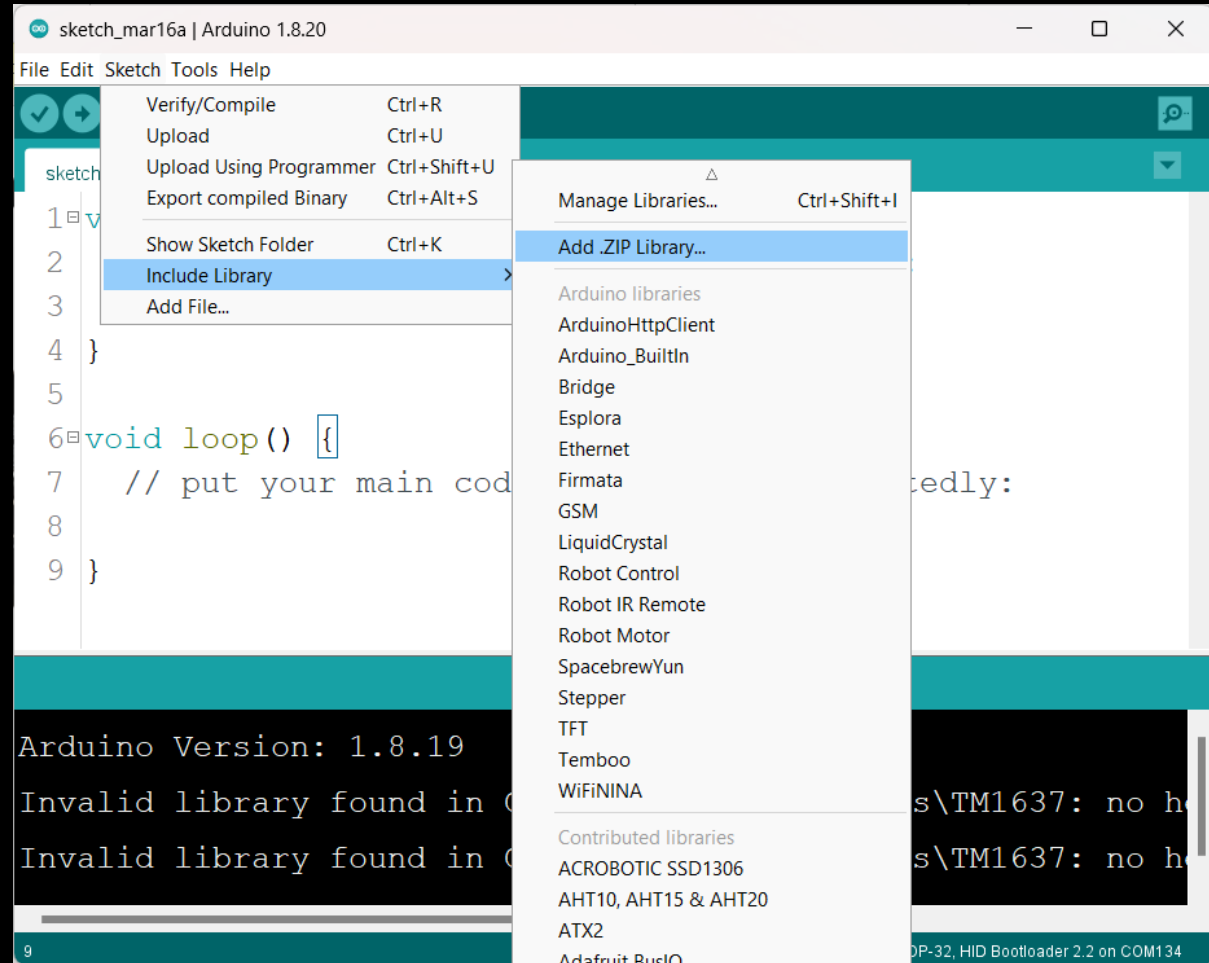
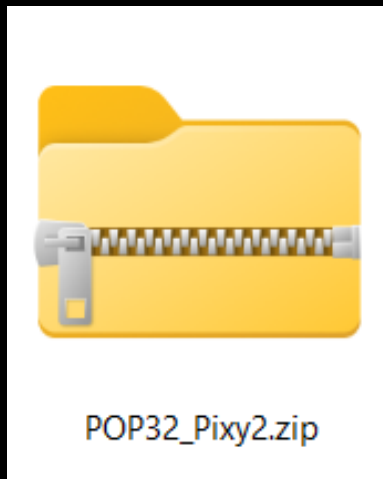
เลือก Interface เปลี่ยน Data out port เป็น I2C แล้วกดปุ่ม OK

# พิกัดภาพจากมุมมองของ Pixy2



# ติดตั้งไลบรารีสำหรับ POP-32

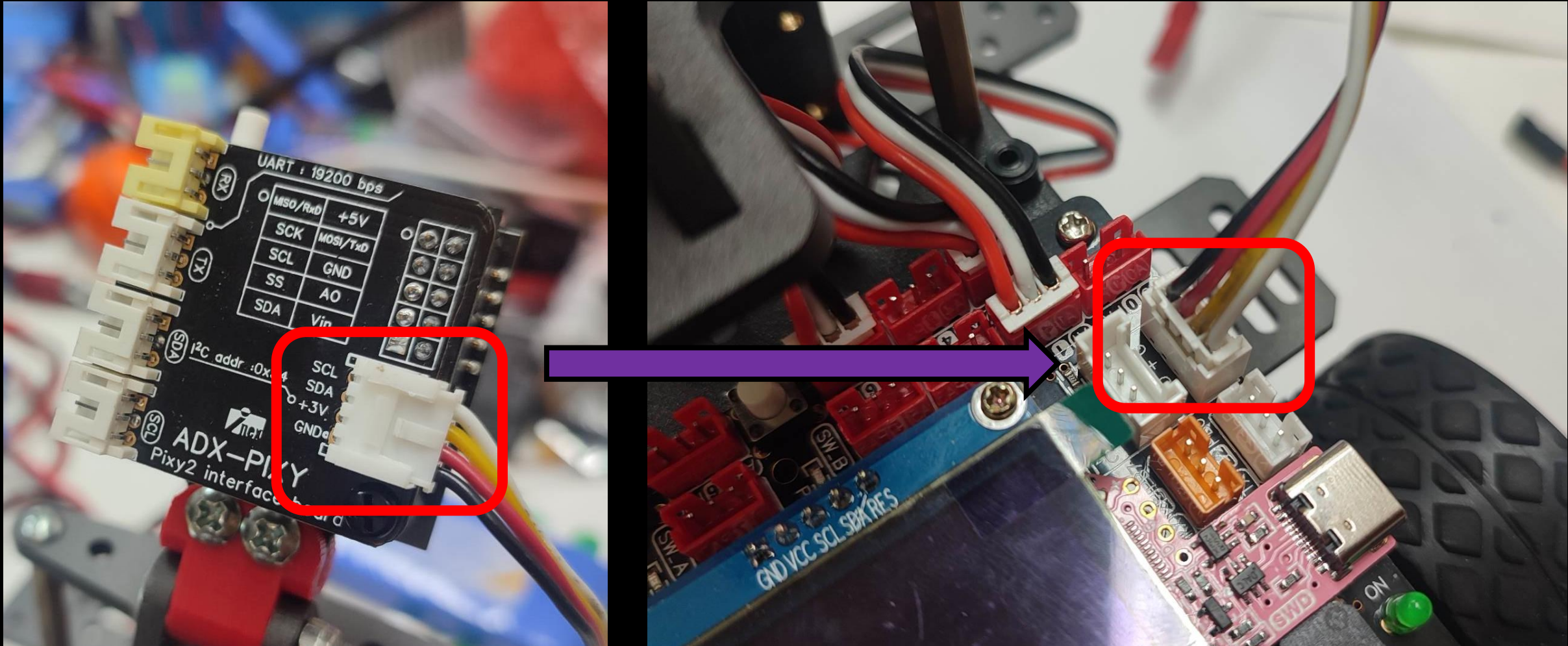
ดาวน์โหลดไลบรารีได้ที่ [https://inex.co.th/store/software/POP32\\_Pixy2.zip](https://inex.co.th/store/software/POP32_Pixy2.zip)



เข้าเมนู Sketch -> Include Library -> Add.Zip Library.. จากนั้นเลือกไฟล์ POP32\_Pixy2.zip

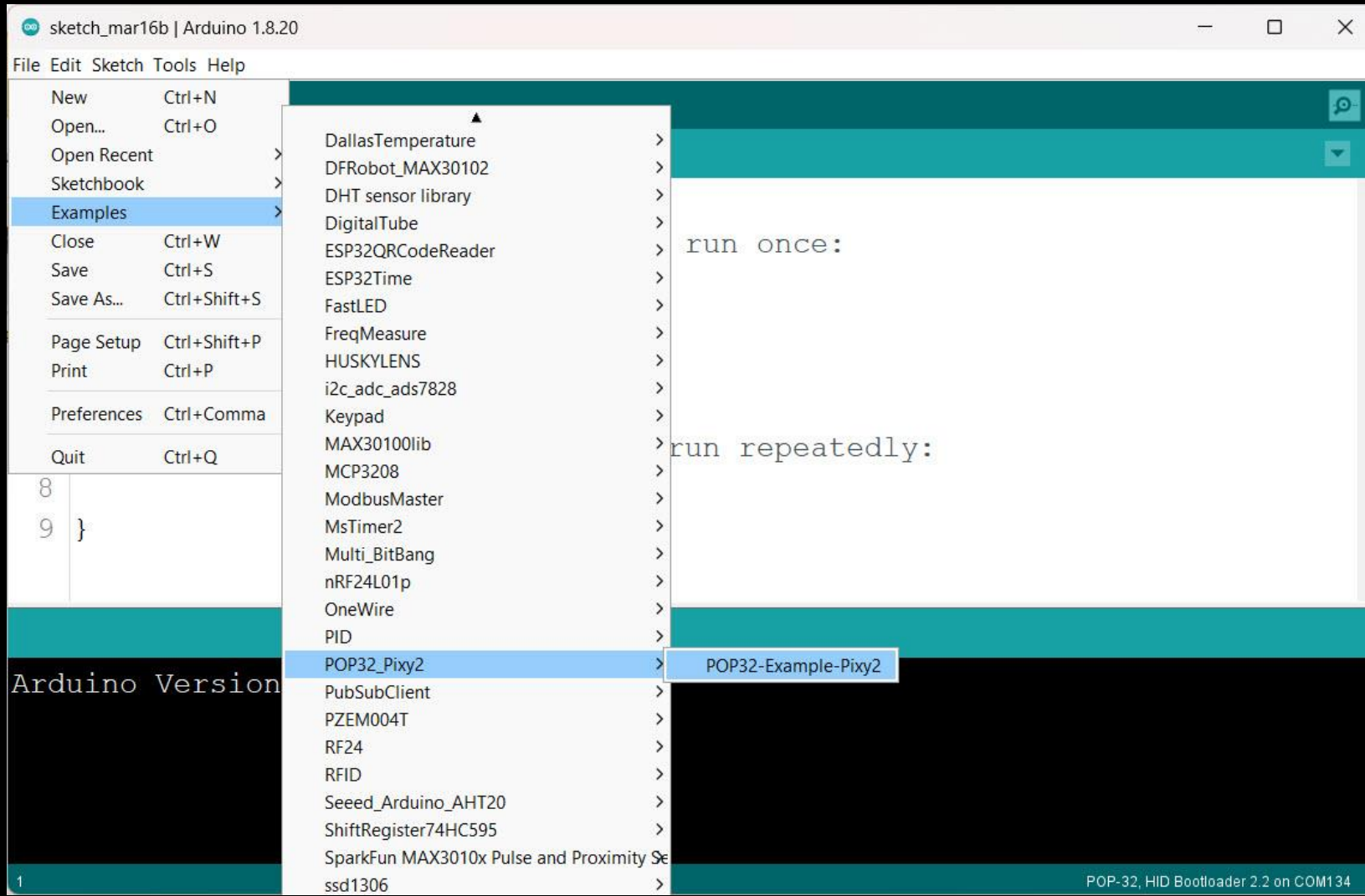


# ติดตั้ง Pixy เข้ากับบอร์ด POP-32



ใช้การเชื่อมต่อแบบ I2C

# ทดสอบไลบรารีสำหรับ POP-32



ไปที่ File -> Examples -> POP32\_Pixy2 -> POP32-Example-Pixy2

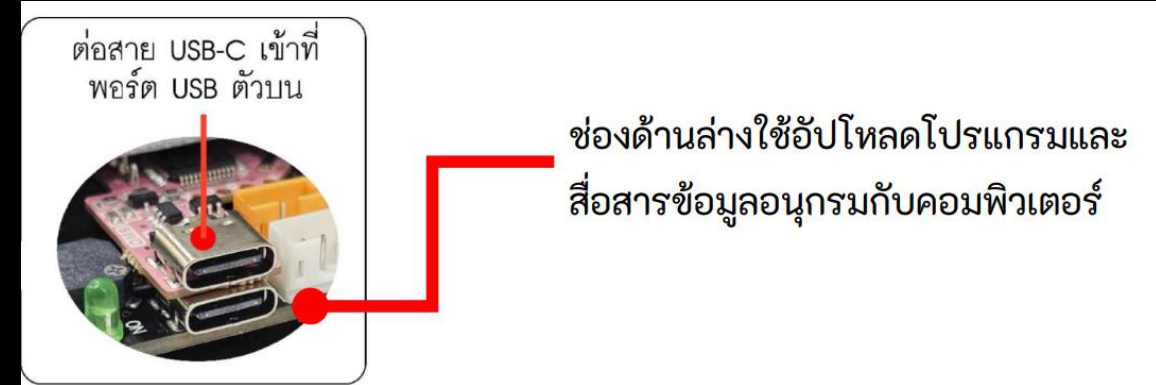


# ทดสอบไลบรารีสำหรับ POP-32

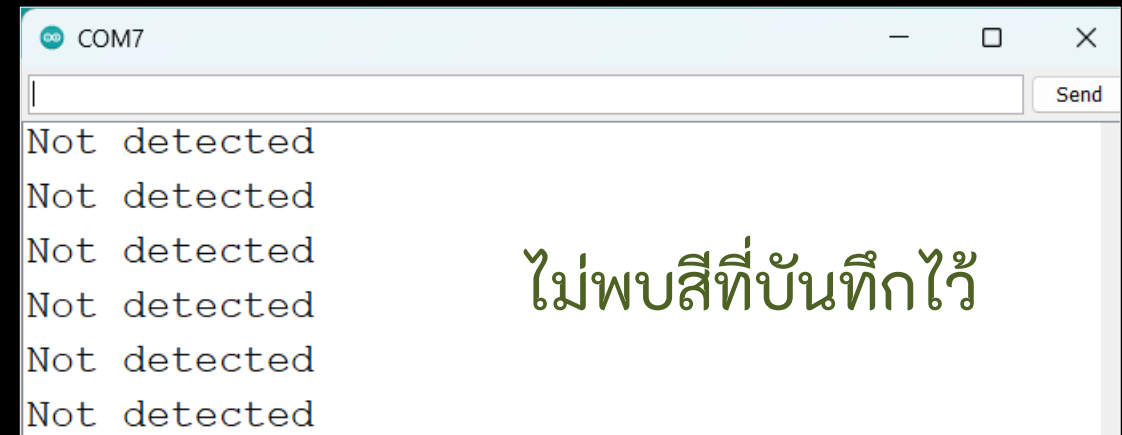
Upload โปรแกรมส่งบอร์ดเพื่อทดสอบ

```
POP32-Example-Pixy2 | Arduino 1.8.20
File Edit Sketch Tools Help

POP32-Example-Pixy2 $
1 #include <POP32.h>
2 #include <POP32_Pixy2.h>
3 POP32_Pixy2 pixy;
4 void setup() {
5     Serial.begin(115200);
6     pixy.init();
7     while (!Serial);|
8     Serial.println("Started!");
9 }
10 void loop() {
11     if (pixy.updateBlocks()) {
12         Serial.print("\nDetected: "); Serial.println(pix
13         Serial.println("x\ty\twidth\theight");
14         for (int i = 1; i <= 7; i++) // for loop signatu
15         {
16             if (pixy.sigSize[i]) // signature not empty
17
Auto Format finished.
> Finish
Invalid library found in C:\Arduinol8\libraries\TM1637:
POP-32, HID Bootloader 2.2 on COM7
```



เปิด Serial Monitor



Detected: 2

| x               | y   | width | height |
|-----------------|-----|-------|--------|
| -> Signature: 1 |     |       |        |
| 161             | 128 | 58    | 47     |
| 292             | 118 | 12    | 3      |

พบสีที่บันทึกไว้

# อธิบายคำสั่งไลบรารีของ Pixy

```
#include <POP32_Pixy2.h>
```

ผนวกไฟล์ไลบรารี

```
POP32_Pixy2 pixy
```

กำหนดตัวแปรที่ชื่อว่า pixy มารองรับคำสั่งทั้งหมด

```
pixy.init()
```

เริ่มต้นใช้งานกล่อง pixy

```
pixy.updateBlocks()
```

ตรวจสอบบล็อกทั้งหมด

```
pixy.getNumBlocks()
```

จำนวนบล็อกทั้งหมด

```
pixy.sigSize[i]
```

นับจำนวนบล็อกทั้งหมดของแต่ละลักษณะ  $i = 1$  ถึง  $7$

# อธิบายคำสั่งไลบรารีของ Pixy

$i$  = คุณลักษณะ 1 ถึง 7

$j$  = ลำดับบล็อกในคุณลักษณะเดียวกัน

`pixy.sigSize[i]`    นับจำนวนบล็อกทั้งหมดของแต่ละคุณลักษณะ

`pixy.sigInfo[i][j].x`    ตำแหน่ง x ตรงกลางบล็อก

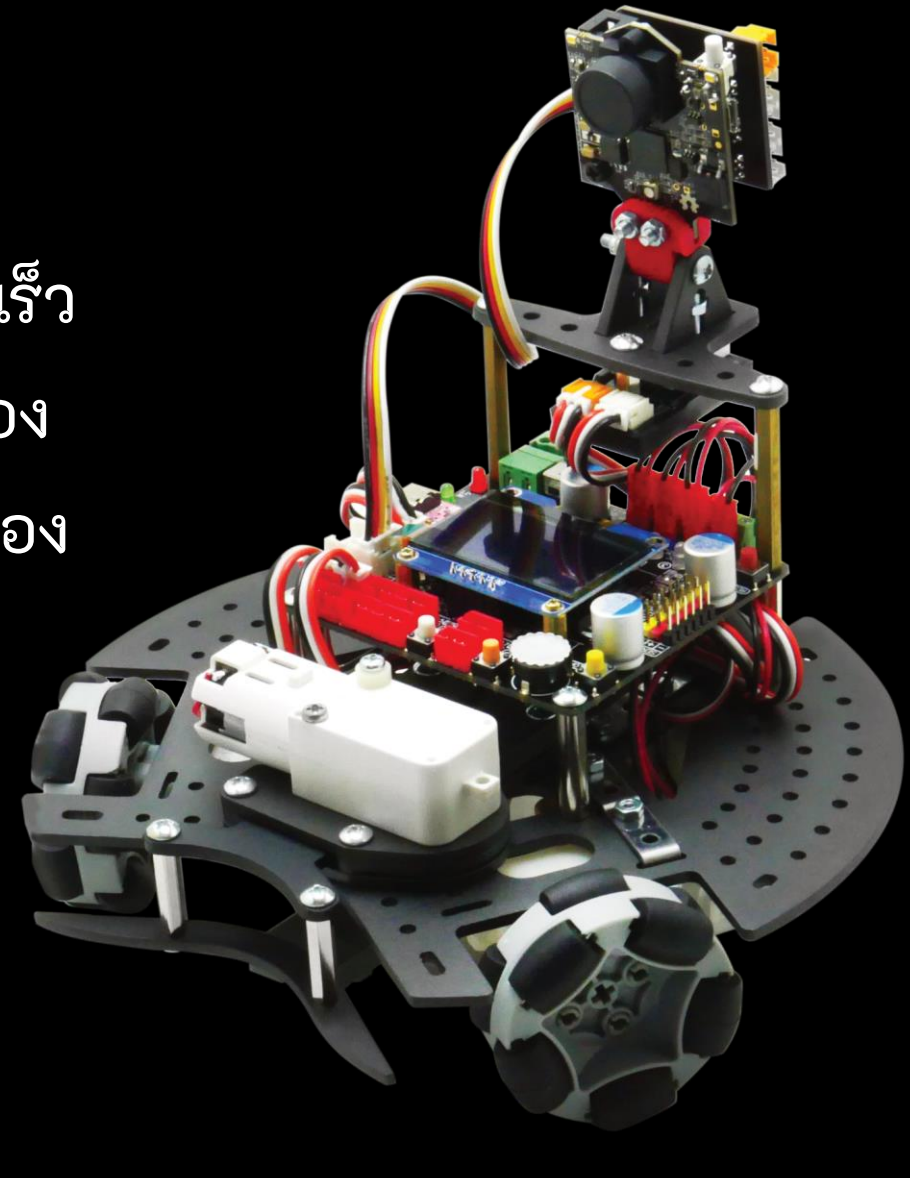
`pixy.sigInfo[i][j].y`    ตำแหน่ง y ตรงกลางบล็อก

`pixy.sigInfo[i][j].width`    ความกว้างของบล็อก

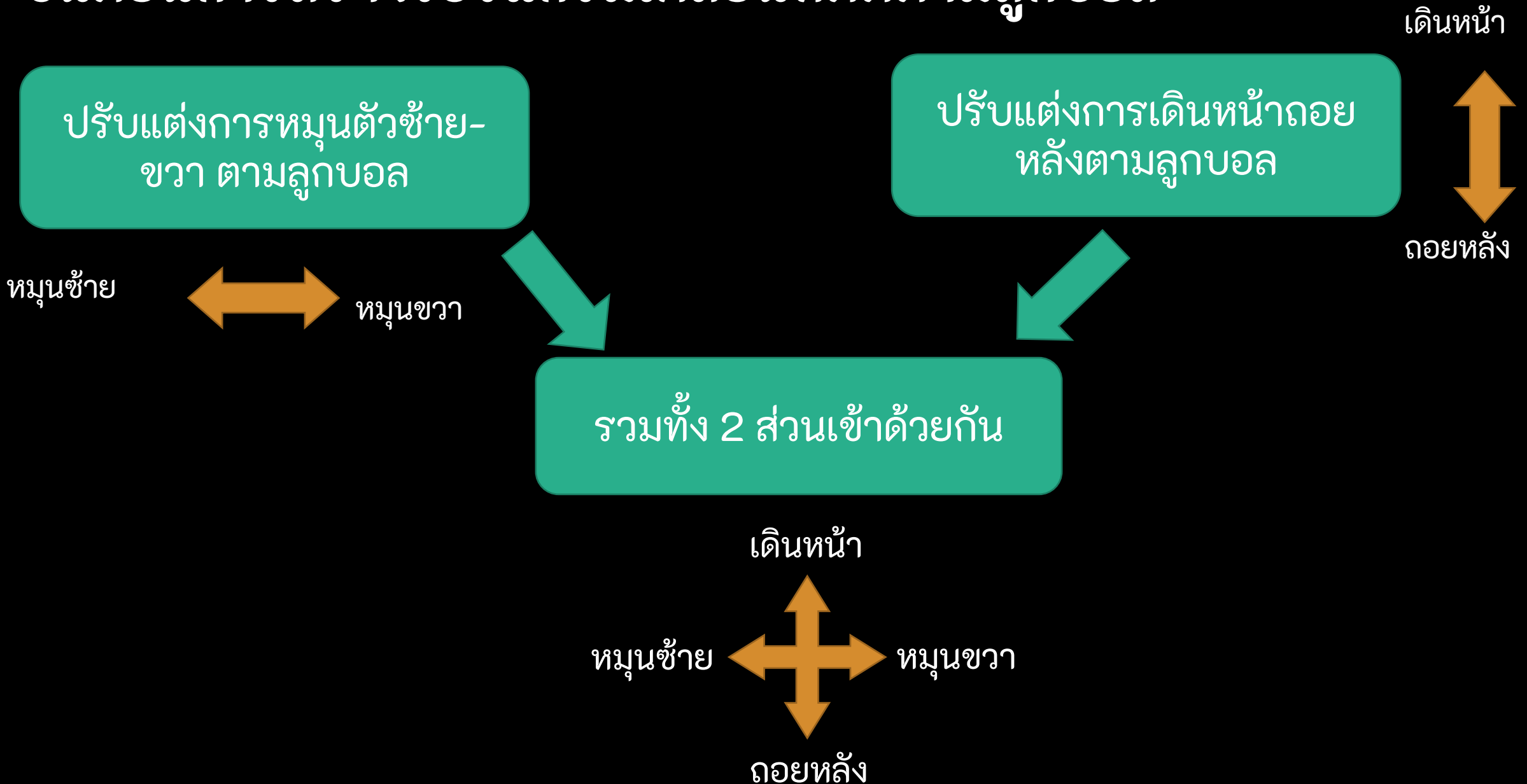
`pixy.sigInfo[i][j].height`    ความสูงของบล็อก

# การเคลื่อนที่ติดตามลูกบอล

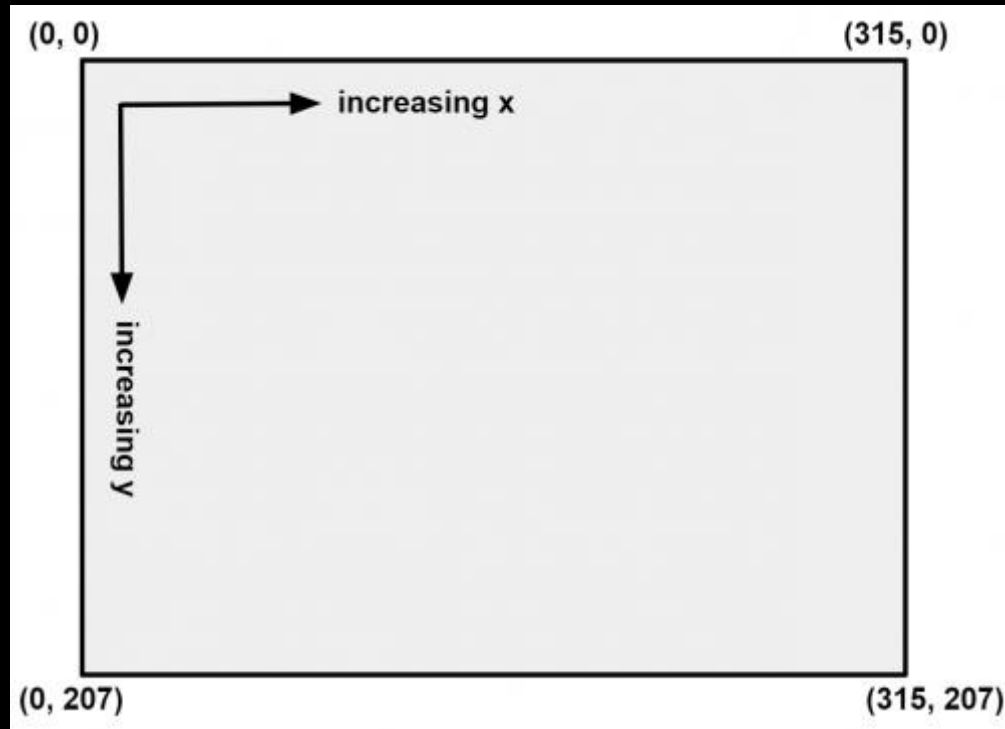
การเคลื่อนที่ติดตามลูกบอลจะต้องอาศัยการควบคุมความเร็ว  
มอเตอร์ โดยที่ความเร็วมอเตอร์จะแปรผันตามระยะทางของ  
ลูกบอลอย่างรวดเร็วมีฉะนั้น จะทำให้ลูกบอลหลุดจากมุมมอง  
กล้องได้



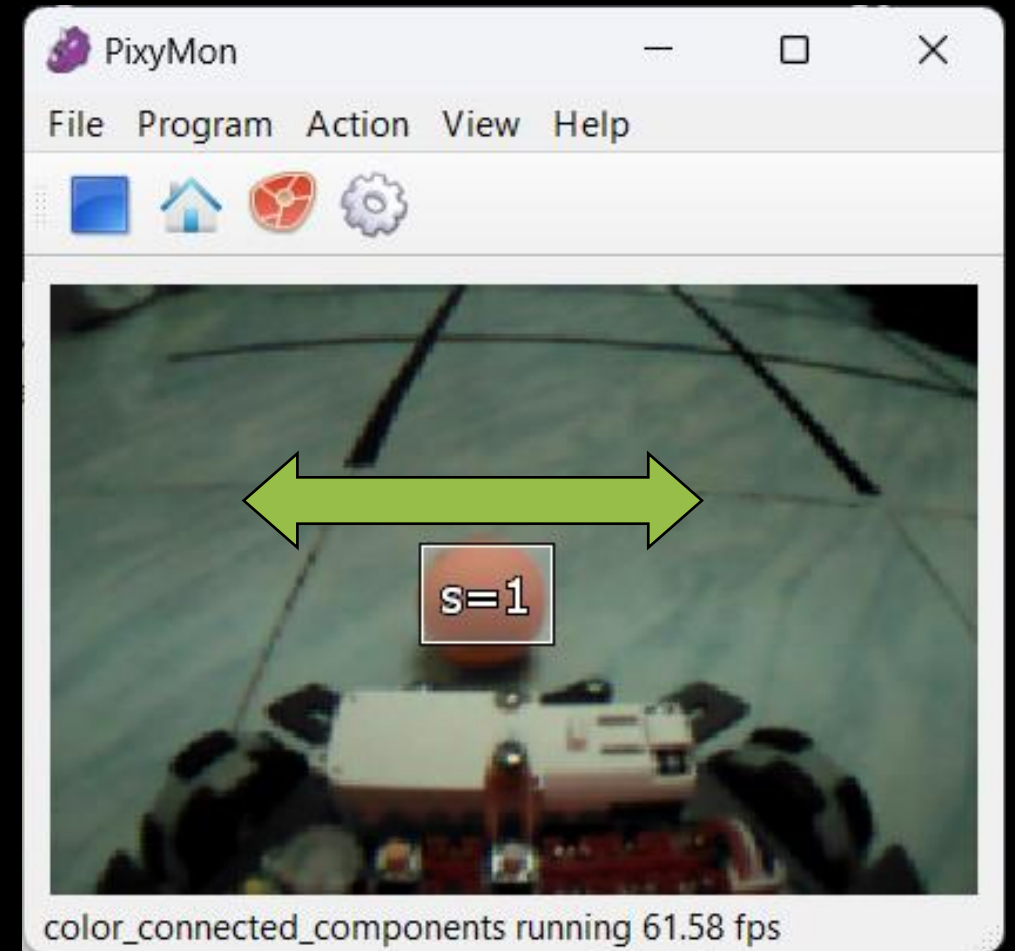
# ขั้นตอนการสร้างโปรแกรมเคลื่อนที่ติดตามลูกบอล



# ปรับแต่งการหมุนตัวซ้าย-ขวา ตามลูกบอล



ใช้แกน x มาพิจารณาการปรับทิศ



# สร้างโปรแกรมเคลื่อนที่ติดตามลูกบอลปรับทิศทางหุ่นยนต์หมุน ช้าย-ขวา

```
#include <POP32.h>
#include <POP32_Pixy2.h>
POP32_Pixy2 pixy;
#define rot_Kp 0.1
#define rot_Ki 0.0
#define rot_Kd 0.0
#define sp_rot 150
#define rotErrorGap 15
float rot_error, rot_pError, rot_i, rot_d, rot_w;
int ballPosX;
```

```
void setup(){
  pixy.init();
  waitSW_A_bmp();
}
```

เริ่มต้นใช้งาน Pixy และ รอกดปุ่ม

เรียกใช้ไลบรารี POP32 และ Pixy  
ประกาศตัวแปรและกำหนดค่า

# สร้างโปรแกรมเคลื่อนที่ติดตามลูกบอลปรับทิศหุ่นยนต์หมุน ช่าย-ขวา

code -> [Test\\_rotate\\_controller.ino](#)

```
void loop() {  
  if (pixy.updateBlocks() && pixy.sigSize[1]) {  
    ballPosX = pixy.sigInfo[1][0].x;  
    rot_error = sp_rot - ballPosX;  
    rot_i = rot_i + rot_error;  
    rot_i = constrain(rot_i,-100,100);  
    rot_d = rot_error - rot_pError;  
    rot_w = (rot_error * rot_Kp) + (rot_i * rot_Ki) + (rot_d * rot_Kd);  
    rot_w = constrain(rot_w,-100,100);  
    holonomic(0, 0, rot_w);  
    rot_pError = rot_error;  
  }  
}
```

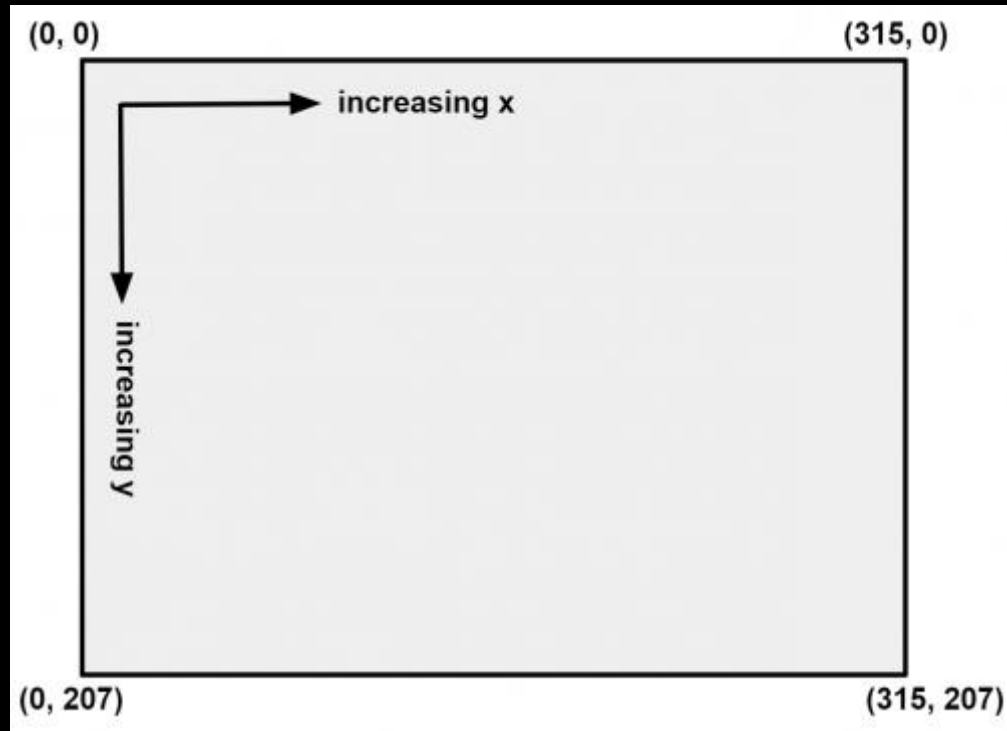


# ปรับแต่งค่าเพื่อให้หุ่นยนต์หมุนตัวซ้าย-ขวา ไปตามลูกบอล

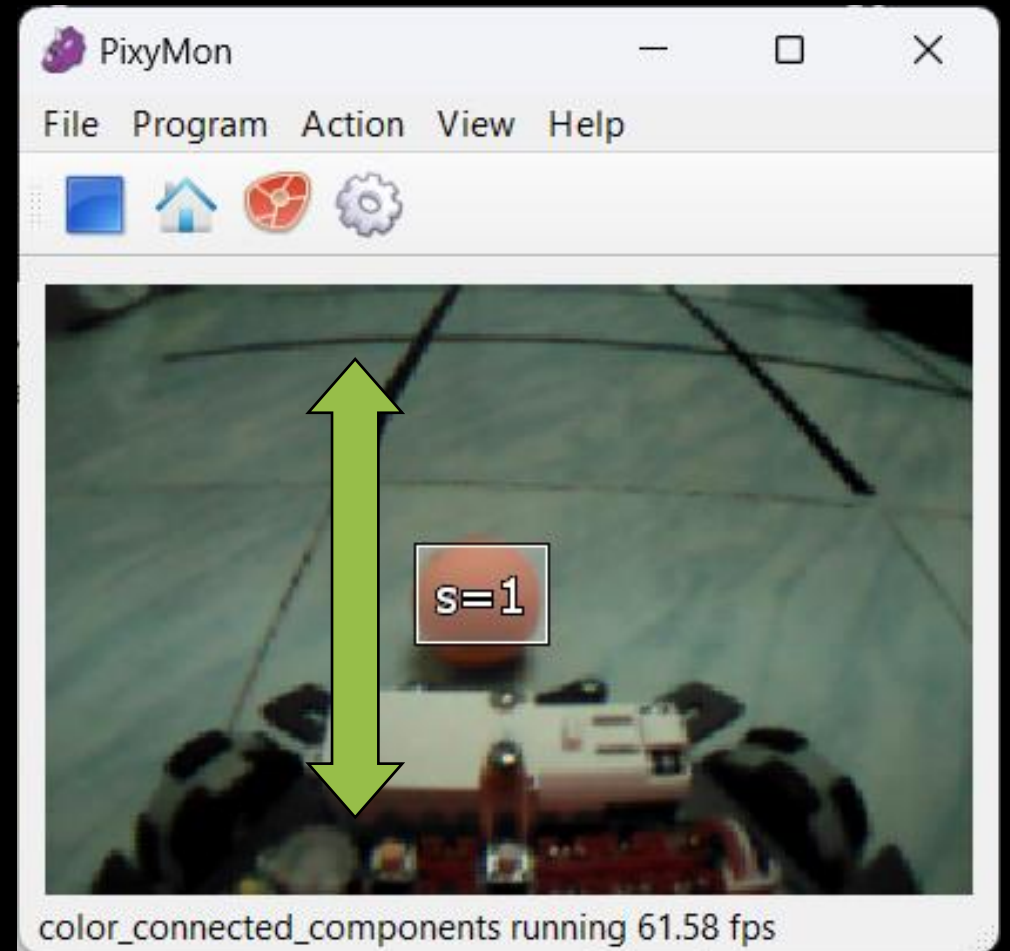
```
#define rot_Kp 0.1  
#define rot_Ki 0.0  
#define rot_Kd 0.0
```

เริ่มจากเพิ่มค่า Kp ก่อนเพื่อให้เข้าสู่จุด Set point ลูกบอลอยู่ตรงกลางหุ่นยนต์พอดี

# ปรับแต่งการเดินทาง-ถอยหลังตามลูกบอล



ใช้แกน y มาพิจารณาการ



# สร้างโปรแกรมเคลื่อนที่ติดตามลูกบอลปรับระยะหุ่นยนต์เข้าใกล้ลูกบอล

```
#define fli_Kp 1.5
#define fli_Ki 0.0
#define fli_Kd 0.0
#define flingErrorGap 15
float spFli = 120;
float fli_error, fli_pError, fli_i, fli_d, fli_spd;
int ballPosY;
```

```
void setup(){
  pixy.init();
  waitAnykey_bmp();
}
```

เริ่มต้นใช้งาน Pixy และ รอกดปุ่ม

เรียกใช้ไลบรารี POP32 และ Pixy  
ประกาศตัวแปรและกำหนดค่า

# สร้างโปรแกรมเคลื่อนที่ติดตามลูกบอลปรับระยะหุ่นยนต์เข้าใกล้ลูกบอล

code -> [Test\\_fling\\_controller.ino](#)

```
void loop() {  
  if (pixy.updateBlocks() && pixy.sigSize[1]) {  
    ballPosY = pixy.sigInfo[1][0].y;  
    fli_error = spFli - ballPosY;  
    fli_i = fli_i + fli_error;  
    fli_i = constrain(fli_i, -100, 100);  
    fli_d = fli_error - fli_pError;  
    fli_spd = fli_error * fli_Kp + fli_i * fli_Ki + fli_d * fli_Kd;  
    fli_spd = constrain(fli_spd, -100, 100);  
    fli_pError = fli_error;  
    holonomic(fli_spd, 90, 0);  
  }  
}
```

# สร้างโปรแกรมเคลื่อนที่ติดตามลูกบอลปรับระยะหุ่นยนต์เข้าใกล้ลูกบอล

```
#define fli_Kp 0.1  
#define fli_Ki 0.0  
#define fli_Kd 0.0
```

เริ่มจากเพิ่มค่า Kp ก่อนเพื่อให้เข้าสู่จุด Set point

# รวมการเคลื่อนที่ในแต่ละแกนเข้าด้วยกัน

code -> [Test\\_Follow-Ball.ino](#)

```
holonomic(fli_spd, 90, rot_w )
```

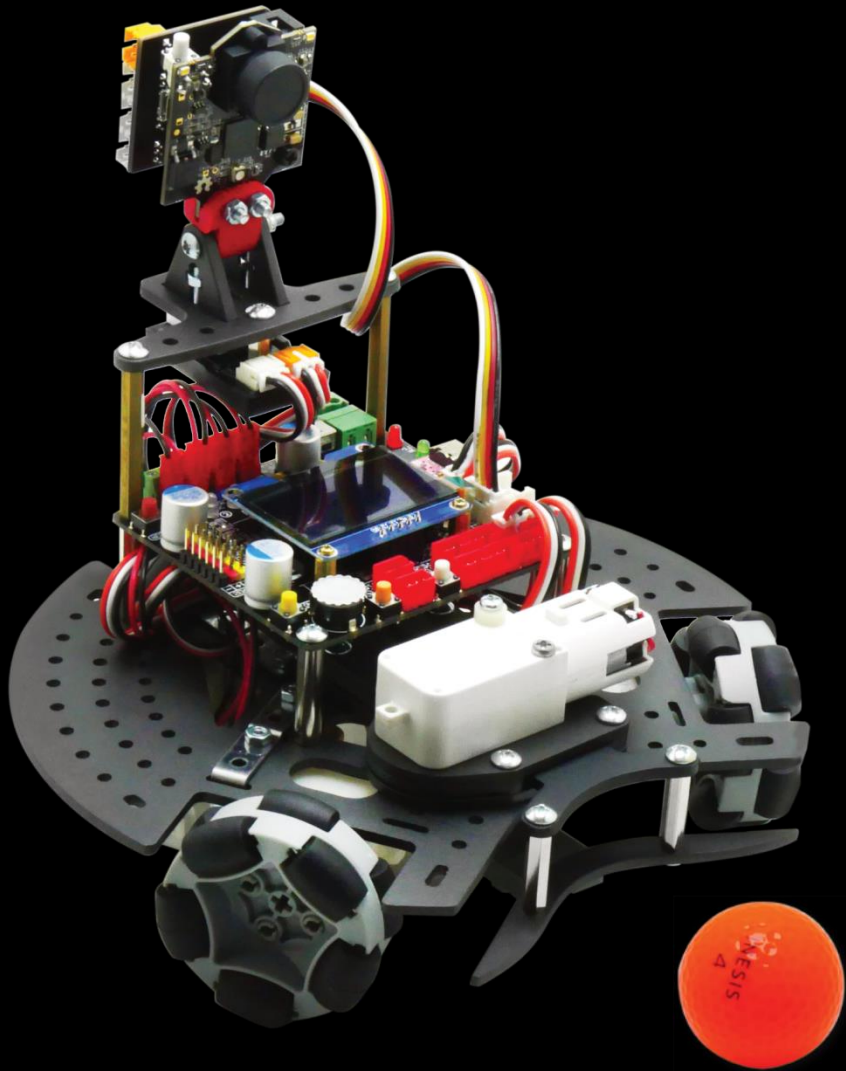
โดยการนำค่า fli\_spd และ rot\_w มาใส่ในฟังก์ชัน holonomic และกำหนดมุมของการเคลื่อนที่ไป 90 องศาเหมือนเดิม

# รวมการเคลื่อนที่ในแต่ละแกนเข้าด้วยกัน

```
if ((abs(rot_error) < rotErrorGap) && abs(fli_error) < flingErrorGap) {  
    wheel(0, 0, 0);  
    beep();  
}
```

เมื่อถึงจุดที่ต้องการ อาจจะหยุดหุ่นยนต์และ  
ส่งเสียงดังออกมา

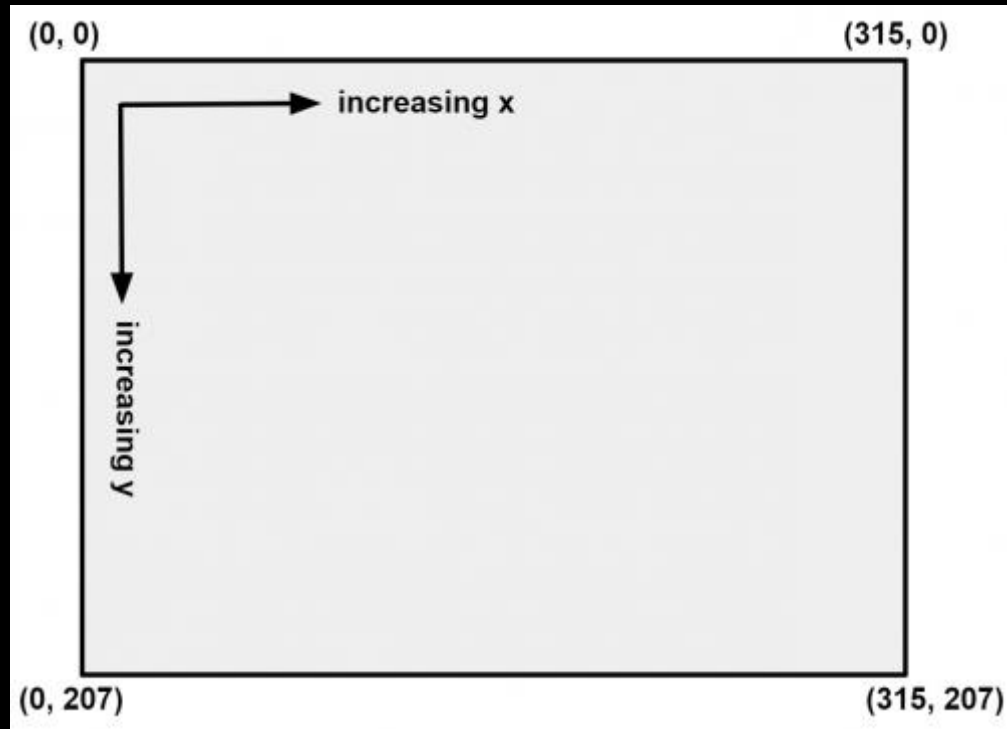
# การเคลื่อนที่ปรับทิศหุ่นยนต์ให้ตรงกับทิศอ้างอิงโดยมีลูกบอลเป็นจุดศูนย์กลาง



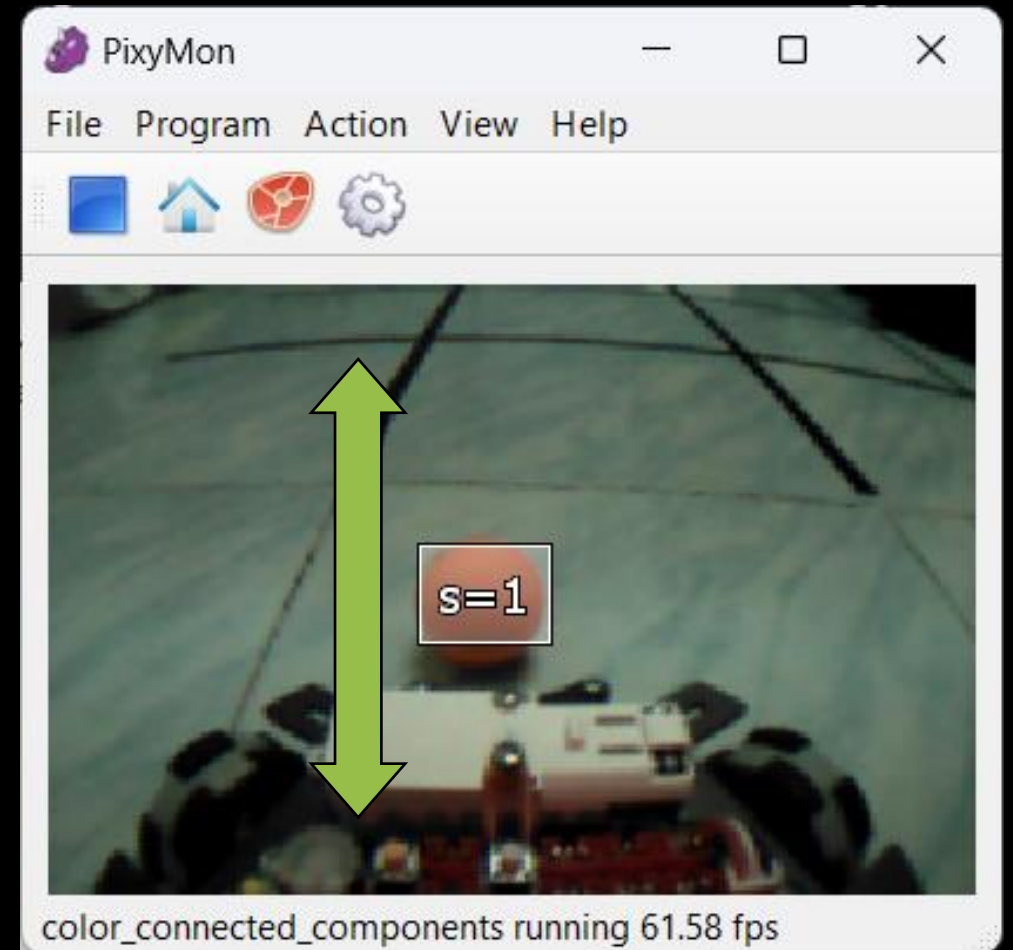
หุ่นยนต์จะเคลื่อนที่หมุนรอบลูกบอลให้ตรงกับทิศ  
อ้างอิงเพื่อให้เตรียมความพร้อมในการยิง



# ปรับแต่งการหมุนตัวรอบลูกบอล



ใช้แกน y มาพิจารณาการ



# การเคลื่อนที่ปรับทิศหุ่นยนต์ให้ตรงกับทิศอ้างอิงโดยมีลูกบอลเป็นจุดศูนย์กลาง

```
// align controller
#define ali_Kp 2.75
#define ali_Kd 0.0
#define alignErrorGap 4
float ali_error, ali_pError, ali_d, ali_vec, lastYaw, vecCurve, radCurve;
float spFli = 130;
int ballPosY;
```

เรียกใช้ไลบรารี POP32 และ Pixy  
ประกาศตัวแปรและกำหนดค่า

เริ่มต้นใช้งาน Pixy และ รอกดปุ่ม B  
รอรีเซตทิศหุ่นยนต์จากนั้นกดปุ่ม A  
เริ่มทำงาน

```
void setup(){
    pixy.init();
    Auto_zero ();
    waitSW_A_bmp();
}
```

```
void loop() {  
  if (pixy.updateBlocks() && pixy.sigSize[1]) {  
    for (int i=0; i<8; i++){ getIMU(); }  
    ballPosY = pixy.sigInfo[1][0].y;  
    ali_error = ballPosY - spFli;  
    ali_d = ali_error - ali_pError;  
    ali_vec = ali_error * ali_Kp + ali_d * ali_Kd;  
    ali_pError = ali_error;  
    if (pvYaw < 0) {  
      vecCurve = -ali_vec;  
      radCurve = 15;  
    }  
    else {  
      vecCurve = 180 + ali_vec;  
      radCurve = -15;  
    }  
    holonomic(40, vecCurve, radCurve);  
    if (abs(pvYaw) < alignErrorGap){  
      holonomic(0,0,0);  
    } } }  
}
```

อ่านค่า ZX-IMU

กำหนดทิศทางหมุนตัว

ตรวจสอบเพื่อหยุดหมุนตัว

การเคลื่อนที่ปรับทิศหุ่นยนต์  
ให้ตรงกับทิศอ้างอิงโดยมีลูก  
บอลเป็นจุดศูนย์กลาง

# รวมการเคลื่อนที่ ติดตามลูกบอลและการปรับทิศเข้าด้วยกัน

```
if (pixy.updateBlocks() && pixy.sigSize[1]) {  
    if (discoveState) {  
        .....กลุ่มคำสั่งเคลื่อนที่ติดตามลูกบอล.....  
        if(setpoint){  
            discoveState =0;  
        }  
    }else {  
        .....กลุ่มคำสั่งเคลื่อนที่ปรับทิศหุ่นยนต์.....  
        if(setpoint){  
            discoveState = 1;  
        }  
    }  
}else{  
    .....กลุ่มคำสั่งเคลื่อนที่หาลูกบอล.....  
}
```

code -> [Test\\_align\\_fling\\_controller.ino](#)

discoveState จะเป็นตัวทำ  
หน้าที่เลือกการเคลื่อนที่

# ชุดคำสั่งเคลื่อนที่หาลูกบอล

หาทิศทางล่าสุดที่กล้องตรวจพบ

```
int sideRot = rot_error;  
holonomic(0, 0, sideRot / abs(sideRot) * idleSpd);
```

ใช้การหมุนรอบตัวเพื่อหาลูกบอล

ความเร็วที่ต้องการหมุนรอบตัว

```
if (pixy.updateBlocks() && pixy.sigSize[1]) {  
    if (discoveState) {  
        .....กลุ่มคำสั่งเคลื่อนที่ติดตามลูกบอล.....  
        if(setpoint){  
            discoveState =0;  
        }  
    }else {  
        .....กลุ่มคำสั่งเคลื่อนที่ปรับทิศหุ่นยนต์.....  
        if(setpoint){  
            discoveState = 1;  
            .....กลุ่มคำสั่งชุดคำสั่งตรวจสอบก่อนยิง....  
        }  
    }  
}else{  
    .....กลุ่มคำสั่งเคลื่อนที่หาลูกบอล.....  
}
```

ชุดคำสั่งตรวจสอบก่อนยิง



# ชุดคำสั่งตรวจสอบก่อนยิง

เมื่อเคลื่อนที่ปรับทิศให้ตรงเรียบร้อยแล้ว จากนั้นจะเป็นขั้นตอนการเตรียมยิงลูกบอล

ตรวจสอบค่า Error ทั้งระยะห่างและทิศทางของลูกบอล

ค่ากึ่งกลางในแนวแกน X

```
if ((abs(150 - ballPosX) < rotErrorGap) && (abs(spFli - ballPosY) < flingErrorGap)) {  
    beep();  
    unsigned long loopTimer = millis();  
    while(1){  
        getIMU();heading(100, 90, 0);  
        if (millis()-loopTimer >= 250)break;  
    }  
    shoot();  
    reload();  
}
```

เดินหน้าตรง เต็มความเร็ว นาน 0.25 วินาที

ทำการยิงและเก็บ

# ชุดคำสั่งฟังก์ชันทั้งหมดที่สร้างขึ้นมาใช้งาน

code -> [Test\\_mission.ino](#)

```
void zeroYaw()  
bool getIMU()
```

เกี่ยวกับปรับทิศใช้ร่วมกับ ZX-IMU

```
void shoot()  
void reload()
```

เกี่ยวกับการยิงและเก็บก้านยิง

```
void wheel(int s1, int s2, int s3)  
void holonomic(float spd, float theta, float omega)  
void heading(float spd, float theta, float spYaw)
```

เกี่ยวกับการเคลื่อนที่